



Geïntegreerde stereo- akoestische-echo-onderdrukking en spraakcodering voor tele- /videoconferencingtoepassingen

Martijn Hemeryck
Hans Van Herck

Proefschrift voorgedragen tot het
behalen van de graad van Master
in de ingenieurswetenschappen:
elektrotechniek, optie Multimedia
en signaalverwerking

Promotor:

Prof. dr. ir. Marc Moonen

Academiejaar 2009 – 2010



Geïntegreerde stereo- akoestische-echo-onderdrukking en spraakcodering voor tele- /videoconferencingtoepassingen

Martijn Hemeryck
Hans Van Herck

Proefschrift voorgedragen tot het
behalen van de graad van Master
in de ingenieurswetenschappen:
elektrotechniek, optie Multimedia
en signaalverwerking

Promotor:

Prof. dr. ir. Marc Moonen

Assessors:

Prof. dr. ir. Hugo Van hamme
Dr. ir. Ann Spriet

Begeleiders:

Dr. ir. Toon van Waterschoot
Ir. Jose Manuel Gil-Cacho
Lorenzo

© Copyright K.U.Leuven

Zonder voorafgaande schriftelijke toestemming van zowel de promotor(en) als de auteur(s) is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen tot of informatie i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, wendt U tot ESAT, Kasteelpark Arenberg 10 postbus 2440, B-3001 Heverlee, +32-16-32 11 30 of via e-mail info@esat.kuleuven.be.

Voorafgaande schriftelijke toestemming van de promotor(en) is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.



Voorwoord

ALGEMEEN DANKWOORD In eerste instantie willen we graag onze promotor, Prof. Marc Moonen bedanken om ons de mogelijkheid te hebben gegeven om dit onderzoek te kunnen doen. Daarbij willen we benadrukken dat zijn lessen met betrekking tot digitale signaalverwerking en audioverwerking voor ons een belangrijke basis zijn geweest om deze thesis aan te vangen.

Daarnaast gaat ook heel veel dank uit naar onze dagelijkse begeleiders, Dr. Ir. Toon van Waterschoot en Ir. Jose Manuel Gil-Cacho Lorenzo (Pepe) voor hun onmisbare hulp. Hun goede raad is voor ons steeds zeer waardevol gebleken, of het nu ging over de concepten achter een bepaald algoritme, de praktische uitwerking er van of het schrijven van de bijhorende technische teksten. De sfeer waarin onze gesprekken verliepen hebben we ook altijd als zeer aangenaam ervaren, waarvoor we ook dankbaar zijn.

Ook willen we de jury bedanken voor hun tijd voor het evalueren van ons werk. Specifiek Prof. Van hamme willen we bedanken voor de vele inzichten die hij ons heeft bijgebracht met betrekking tot spraakverwerking, wat een zeer belangrijk punt is geweest in ons werk.

Onze jaargenoten en tevens ook gemeenschappelijke vrienden verdienen hierbij ook onze dank. Niet alleen zijn we hen dankbaar voor hun serieuze opmerkingen (o.a. in het kader van deze thesis), vooral de niet serieuze opmerkingen hebben er toe bijgedragen dat we steeds in een aangename sfeer hebben kunnen werken. We zullen later nog met plezier terugkijken naar deze periode die we zo samen hebben doorgemaakt en de activiteiten die we samen gedaan hebben.

MARTIJN Ik had graag mijn familie willen bedanken voor al die jaren dat ze me steeds onvoorwaardelijk gesteund hebben op alle mogelijke vlakken, hetzij materieel, hetzij mentaal: altijd kon ik op hen rekenen. Zonder hen was ik waarschijnlijk nooit zo ver geraakt. Uiteindelijk is het ook voor een groot deel dankzij mijn vader dat ik een interesse heb gekregen in wetenschap en techniek en meer specifiek later in audio-en spraakverwerking.

Naast mijn vrienden onder mijn mede-burgies eerder aangehaald wil ik zeker ook mijn vrienden van KAJ Diepenbeek evenals alle andere vrienden die ik heb leren kennen in de jaren dat ik in Leuven verblijf.

Tot slot zijn er waarschijnlijk nog een groot aantal mensen die ik hier niet expliciet vermeld heb, maar aan wie ik ook veel steun heb gehad. Ik hoop dat ze beseffen dat ze eveneens mijn dank verdienen.

HANS Ik zou evenzeer mijn familie en mijn ouders in het bijzonder willen bedanken voor hun steun, zowel in ontspannende als minder ontspannende tijden. Op hun eigen manier hebben ze mij - meer dan ze zelf zullen beseffen - geholpen; dit niet alleen bij het theoretisch samenstellen en schrijven van deze thesis, maar ook bij mijn algemene vorming en opvoeding doorheen de jaren, wat me toegelaten heeft dit punt te bereiken.

Vervolgens dank ik hier ook mijn vriendin, Veerle. Naast haar aangename karakter en opbeurende lach hebben haar aanmaningen en bezorgdheid om mijn goeie thesisresultaten veel bijgedragen aan de succesvolle afwerking van deze masterproef.

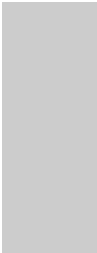
Ten slotte ben ik genoodzaakt specifiek mijn dank te betuigen aan Caroline, Jonas en Arnout. Zij vervulden met glans de rol van trouwe wiespartners tijdens de vele middagen en pauzes die we samen hebben doorgemaakt. Jonas verschaftte ons ook steeds het nodige vervoer, waarvoor dank.

Martijn Hemeryck
Hans Van Herck

Inhoudsopgave

Voorwoord	i
Inhoudsopgave	iii
Samenvatting	v
Lijst van figuren en tabellen	vi
Lijst van afkortingen, symbolen en begrippen	ix
1 Inleiding	1
1.1 Motivatie	1
1.2 Overzicht	2
SAEO 2, Stereo Codering 2, Integratie 2	
2 SAEO	4
2.1 Inleiding	4
2.2 AEO	5
2.3 SAEO fundamenteel probleem	6
SAEC opstelling 6, Niet-uniciteitsprobleem 7, Uitlijningsprobleem 11,	
Impulsresponsie staarteffect 14, Coherentiefunctie 14, Simulaties 15	
2.4 Overzicht van alternatieven	18
Toevoegen van willekeurige ruis 19, Decorrelatiefilters 21, Schuiven van	
de spectra tussen de kanalen 21, Vershoven kamfilters 22, Niet-lineaire	
operaties 22, Orthogonale transformaties 23	
2.5 Lineaire predictie	26
Lineair predictief decorrelatiefilter 27, Gezamenlijke proces schatting –	
Newton/LMS 28	
2.6 Samenvatting	44
3 Stereo codering	46
3.1 Inleiding	46
3.2 Coderingsvormen	47
Som-verschilcodering 47, Subbandcodering 47,	
Woordenboekcodering 48, Differentiële codering 49	

3.3	Spraakcodering via lineaire predictieve codering	50
	<i>Anatomische achtergrond van spraak 50, Voorstelling via bron-filtermodel 51, LPC voor mono spraaksignalen 52</i>	
3.4	Stereo lineaire predictie	57
	<i>Basis: uitbreiding van mono lineaire predictie 58, Eigen implementatie 64</i>	
3.5	Samenvatting	71
4	Integratie	73
4.1	Inleiding	73
4.2	Combinatie LPC en SAEO: decorrelatie via roosterimplementatie	74
	<i>Theorie 74, Praktijk 75</i>	
4.3	Combinatie LPC en SAEO: decorrelatie via LPC-data	77
	<i>Theorie 77, Simulaties 80, Invloed van predictieorde 81, Invloed van venstergrootte 83</i>	
4.4	Niet-ideale transmissie	84
4.5	Samenvatting	85
5	Besluit	87
5.1	Stereo akoestische echo-onderdrukking	87
5.2	Stereo lineaire predictieve codering	88
5.3	Integratie	88
5.4	Verder onderzoek	89
	Bibliografie	91



Samenvatting

Voor toepassingen zoals tele-en videoconferencing is het interessant om over meerdere geluidskanalen te beschikken om het spatiaal effect te kunnen behouden. De problemen met betrekking tot akoestische echo's en eventuele fluittonen ten gevolge van een akoestische terugkoppeling, kunnen worden opgelost met adaptieve filtering.

Het eerste deel omvat een uitbreiding van akoestische echo-onderdrukking naar meerdere kanalen. Een overzicht van het fundamentele probleem wordt gegeven op basis van een overzicht van de literatuur. Uit de beschrijving van het fundamentele probleem zal volgen dat iedere praktische oplossing gebruik maakt van een extra stap, waarbij de twee ingangssignalen voor het adaptieve algoritme eerst gedecorreleerd worden. Een overzicht van een aantal van deze methodes volgt, gebaseerd op een overzicht uit de literatuur. Specifiek wordt er gekeken naar methodes gebaseerd op lineaire predictie, enerzijds omdat kan worden aangetoond dat de predictiefouten orthogonaal zijn ten opzichte van elkaar, anderzijds omdat lineaire predictie interessant is naar (spraak)coderingstoepassingen toe, wat in het latere deel besproken wordt.

Het tweede deel behandelt codering in een ruime context voor stereofone signalen. Een overzicht van verschillende bestaande coderingstechnieken wordt gegeven, waarna in meer detail wordt overgegaan tot een beschrijving van codering van tweekanaals geluid op basis van lineaire predictie. Specifiek voor spraaksignalen wordt het onderliggende anatomische model besproken. Een uitbreiding voor lineaire predictie naar meerdere kanalen, d.i. SLP of stereo lineaire predictie wordt gegeven, met aansluitend resultaten van een eigen implementatie.

Het laatste deel bevat de eigen bijdragen, waar onderzocht wordt hoe een akoestisch echo-onderdrukkingsschema kan geïntegreerd worden met een stereo codering, beiden op basis van lineaire predictie. Hiertoe worden twee geïntegreerde schema's voorgesteld. Eén schema is gebaseerd op het LMS/Newton algoritme. De schatting van de autocorrelatiematrix hier wordt vereenvoudigd doordat ze geschreven wordt als een factorisatie op basis van de predictiefouten. In de beschrijving van het originele algoritme gebruikt men hierbij AR parameters die recursief geschat worden en dus waar voor ieder tijdstip een waarde voor gedefinieerd is. Aangezien het SLP algoritme werkt over vensters van waarden wordt het effect hiervan verder onderzocht. Een ander schema is gebaseerd op het direct toepassen van een standaard NLMS algoritme op de predictiefouten. Ook wordt het effect van de gebruikte predictieorde onderzocht. Tot slot wordt kort de invloed van niet-ideale transmissie voor het voorgesteld geïntegreerd schema bekeken.

Lijst van figuren en tabellen

Lijst van figuren

2.1	Akoestische echo-onderdrukkingsschema	5
2.2	Stereo akoestische echo-onderdrukking schema	7
2.3	Simulatie fundamenteel probleem, gebruikte ingangssignalen	16
2.4	Simulatie fundamenteel probleem foutsignalen, mono	17
2.5	Simulatie fundamenteel probleem foutsignalen, stereo	17
2.6	Simulatie fundamenteel probleem uitlijning, mono	17
2.7	Simulatie fundamenteel probleem uitlijning, stereo	18
2.8	Simulatie fundamenteel probleem uitlijning per kanaal, stereo	18
2.9	Coherentiefuncties voor de verschillende ingangssignalen, stereo	19
2.10	Verre eind signalen na toevoeging van ruis voor decorrelatie, SNR=25dB	20
2.11	Uitlijning na toevoeging van ruis voor decorrelatie	20
2.12	Uitlijning na toevoeging van ruis voor decorrelatie (2 kanalen)	21
2.13	Coherentiefunctie na toevoeging van ruis voor decorrelatie	21
2.14	Verre eind signalen na toepassing niet-lineaire decorrelatie	23
2.15	Uitlijning na toepassing niet-lineaire decorrelatie	23
2.16	Uitlijning per kanaal na toepassing niet-lineaire decorrelatie	24
2.17	Coherentiefunctie na toepassing niet-lineaire decorrelatie (2 kanalen)	24
2.18	Tweekanaals rooster cel	28
2.19	Pool/nulpunt diagramma met AR parameters voor het gegenereerde signaal	29
2.20	Gesynthetiseerd AR signaal met gekende parameters	30
2.21	κ_{11} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters	31
2.22	κ_{12} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters	31
2.23	κ_{22} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters	32
2.24	κ_{11} zoals berekend met de roosterpredictor uitgemiddeld over de tijd, vergeleken met de gegenereerde AR parameters	32
2.25	Gesynthetiseerd AR signaal met gekende parameters voor vier verschillende segmenten	33

2.26	κ_{11} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde tijdsafhankelijke AR parameters	33
2.27	κ_{11} zoals berekend met de roosterpredictor over de tijd voor spraaksignaal man, vergeleken venstergebaseerde LPC-methode	34
2.28	κ_{11} zoals berekend met de roosterpredictor over de tijd voor spraaksignaal vrouw, vergeleken venstergebaseerde LPC-methode	35
2.29	Blokdiagonaal structuur autocorrelatiematrix achterwaartse predictiefouten R_{bb}	36
2.30	Simulatie standaard NLMS-algoritme	43
2.31	Simulatie gezamenlijke proces schatting – Newton/LMS-uitlijning	44
3.1	Schema van een algemene transmissie met codering en kwantisatie.	46
3.2	Boomstructuur bekomen door herhaaldelijk gebruik van een LDF en HDE . .	48
3.3	Algemeen schema van differentiële codering met vertragingselement.	49
3.4	Algemeen schema van differentiële decodering met vertragingselement.	49
3.5	Schematische voorstelling van de componenten van het menselijk spraakkanaal.	50
3.6	Voorstelling van het bron-filtermodel, hier voor een stemloos foneem.	51
3.7	Analysefilter uit vergelijking (3.8) als equivalent voor het stemkanaal: implementatie als rooster.	55
3.8	Synthesefilter gebaseerd op volledig bron-filtermodel.	56
3.9	Coderingsschema van differentiële codering met lineaire predictie gebaseerd op bron-filtermodel.	57
3.10	Analysefilter van een systeem met stereo lineaire predictie.	59
3.11	Originele stereofone ingangssignalen met de twee kanalen in verschillende kleur gegeven.	65
3.12	Gereconstrueerde stereofone ingangssignalen.	66
3.13	Reconstructiefout als verschil tussen originele en gereconstrueerde ingangen.	66
3.14	Residuwaarden voor beide kanalen na analyse voor de ingangssignalen getoond in figuur 3.11.	67
3.15	Residuwaarden voor beide kanalen na analyse en rotatie voor de ingangssignalen getoond in figuur 3.11.	68
3.16	Variatie van predictieorde en invloed hiervan op energie van residu's van beide kanalen.	69
3.17	Coherentie tussen de twee residu's in het frequentiedomein, voor predictieorde 1.	70
3.18	Coherentie tussen de twee residu's in het frequentiedomein, voor predictieorde 20.	70
3.19	Energie van de coherentie tussen de twee stereo kanalen in het frequentiedomein, voor verschillende predictieorden.	71
3.20	Energie van residu's van de twee stereo kanalen voor verschillende venstergroottes, te beginnen bij 5 ms.	72
4.1	Geïntegreerd schema decorrelatie via roosterimplementatie	76
4.2	Geïntegreerd schema decorrelatie via predictiefouten	79
4.3	Uitlijning van mannen- en vrouwenstem met toegevoegde ruis, bekomen via standaard LMS.	81

4.4	Uitlijning van mannen- en vrouwenstem met toegevoegde ruis, bekomen via decorrelatie met stereo lineaire predictie.	81
4.5	Uitlijning van een audiosignaal met toegevoegde ruis, bekomen via decorrelatie met stereo lineaire predictie.	82
4.6	Invloed van de predictieorde op de uitlijning na volledig doorlopen van het ingangssignaal.	83
4.7	Invloed van de venstergrootte op de uitlijning na volledig doorlopen van het ingangssignaal.	84
4.8	Invloed van ruisadditie (in overgebleven SNR van de doorgestuurde predictiecoëfficiënten) op de convergentiesnelheid.	85
4.9	Invloed van ruisadditie (in overgebleven SNR van de doorgestuurde data) op de convergentiesnelheid.	86

Lijst van tabellen

3.1	Recursief Levinson-Durbin algoritme voor monofone spraak.	54
3.2	Recursief blok-Levinson algoritme voor stereofone audio en spraak.	63



Lijst van afkortingen, symbolen en begrippen

Afkortingen

AEO	akoestische echo-onderdrukking
AR	autoregressief
dB	decibel
DCT	discrete cosinus transformatie
DFT	discrete fourier transformatie
FFT	“fast fourier transform”: snelle fourier transformatie
FIR	“finite impulse response”: eindige impulsresponsie
HDF	hoogdoorlaatfilter
Hz	hertz
KLT	karhunen-loève theorema
LAR	“log area ratio”: logaritmische oppervlakteverhouding
LDF	laagdoorlaatfilter
LMS	“least mean squares”: kleinste kwadraten
LPC	lineaire predictie codering/coëfficiënten
NLMS	“normalised least mean squares”: genormaliseerde kleinste kwadraten
PCA	“principal component analysis”: principale componenten analyse
PSD	“power spectral density”: vermogen spectrale dichtheid
QMF	“quadrature mirror filter”: kwadraatuur spiegel filter
RMS	“root mean square”: effectieve waarde
SAEO	stereo akoestische echo-onderdrukking
SLP	stereo lineaire predictie
SNR	“signal to noise ratio”: signaal-tot-ruis verhouding
d.i.	dit is
e.g.	“exempli gratia”: bijvoorbeeld

Symbolen

$s(n)$	Bronsignaal
$g(n)$	Verre eind kamer impulsresponsie
$h(n)$	Dichte eind kamer impulsresponsie
$x(n)$	Bronsignaal, gefilterd door verre eind kamerimpulsresponsie
$d(n)$	Bronsignaal, gefilterd door verre en dichte eind kamerimpulsresponsie
$\hat{h}(n)$	Dichte eind adaptief filter kamerimpulsresponsie
$y(n)$	Bronsignaal, gefilterd door verre en dichte eind adaptief filter kamerimpulsresponsie
$e(n)$	Adaptief filter foutsignaal
$x'(n)$	Niet-lineair vervormd signaal
$v(n)$	Ruisbronsignaal
$J(n)$	Kostfunctie
$\mathbf{R}(n)$	Covariantiematrix
$\mathbf{r}$	Kruiscorrelatievector
L_F	Lengte verre eind kamerimpulsresponsie
L_N	Lengte dichte eind kamerimpulsresponsie
L_M	Lengte adaptief filter
$\mathbf{b}_m(n)$	Achterwaartse predictiefout, orde m
$\mathbf{f}_m(n)$	Voorwaartse predictiefout, orde m
$\mathbf{\kappa}_m(n)$	Reflectiecoëfficiëntenmatrix, orde m
$\mathbf{G}_{m,j}$	Achterwaartse predictieveelterm matrix, orde m , iteratie j
$\mathbf{A}_{m,j}$	Voorwaartse predictieveelterm matrix, orde m , iteratie j
$u(n)$	Adaptieve stapgrootte vector
χ	Eigenwaardeverdeling

Inleiding

1.1 Thesismotivatie

De wet van Moore [1] heeft gemaakt dat de hoeveelheid rekenkracht per oppervlakte steeds maar is kunnen blijven toenemen sinds medio jaren '60. Mobiele toepassingen kunnen bijgevolg steeds meer beroep doen op die toegenomen rekenkracht met tot gevolg dat e.g. GSM toestellen steeds kleiner in grootte kunnen worden gemaakt. Een praktisch probleem dat hier uit voortvloeit is dat de luidspreker en de microfoon steeds dichterbij elkaar komen te staan en dat dit mogelijk aanleiding kan geven tot hinderlijke akoestische echo's, i.e. het luidsprekersignaal dat enkel voor de luisteraar bedoeld is, komt ook gedeeltelijk wordt ook gedeeltelijk terug opgevangen door de microfoon, met tot gevolg dat de andere spreker zichzelf terug hoort.

De meest gangbare oplossing voor dit probleem gebeurt met behulp van adaptieve filtering. Een tijdsafhankelijk model voor de akoestische ruimte wordt opgesteld en periodisch bijgesteld, zodanig dat het mogelijk wordt om een benadering op te stellen voor de akoestische echo, die dan nadien kan verwijderd worden van het doorgestuurde signaal.

Een andere toepassing waar het probleem van de echo's zich voordoet is dat van de tele- en videoconferencing. Net zoals bij de mobiele toepassingen bestaat ook hier de mogelijkheid dat het akoestisch signaal dat wordt gegenereerd door de luidspreker opnieuw wordt opgevangen door de microfoon. Voor tele- en videoconferencing bestaat echter ook de mogelijkheid om het aantal kanalen uit te breiden: het voornaamste voordeel is dat de ruimtelijke informatie in het geluid op deze manier ook kan overgebracht worden. Het probleem met betrekking tot de akoestische echo's wordt hierdoor echter wel complexer. Een logische aanpak zou er in bestaan om het probleem voor ieder van de kanalen afzonderlijk aan te pakken. Spijtig genoeg leidt dit tot andere problemen: doordat de twee kanalen in de praktijk een grote mate van correlatie vertonen, is het moeilijk om ieder van de nodige kanalen te modelleren [2].

Oplossingen voor de problemen gerelateerd met de correlatie die er bestaat tussen de kanalen gaan meestal uit van een extra decorrelerende stap [2, 3]. Vele methodes om een dergelijke decorrelatie toe te passen zijn reeds onderzocht. Eén interessante categorie van methodes is deze waarbij er wordt uitgegaan van het toepassen van modellering met behulp van lineaire predictie, aangezien er in theorie geen vervorming op het originele signaal nodig is. Een ander argument voor het gebruik van lineaire predictie is

dat meeste verliesvrije spraakcoderingstechnieken en bij uitbreiding ook al een aantal audiocoderingstechnieken, ook hierop gebaseerd zijn.

Dit laatste punt maakt dat het interessant wordt om een mogelijke integratie van beide technieken, *akoestische echo-onderdrukking* en *stereo codering* te onderzoeken.

1.2 Overzicht

1.2.1 (Stereo) Akoestische Echo-onderdrukking

Hoofdstuk 2 handelt over het significantere deel als voorbereiding op een performante integratie voor toepassingen zoals tele- en videoconferencing, nl. het neutraliseren van echo's van stereofone spraaksignalen. Een inleiding op het fundamenteel probleem en mogelijke oplossingen hiervoor bestaat uit een uiteenzetting van de theoretische achtergrond, zowel voor monofone als stereofone ingangssignalen. Het verschil tussen beide situaties wordt in dit hoofdstuk uitvoerig behandeld en de wiskundige aspecten verduidelijkt. Daarnaast worden ook een aantal eminente variabelen, nodig voor het verdere verloop van de tekst, gedefinieerd en in functie hiervan enkele simulatieresultaten getoond ter illustratie.

Vervolgens worden een aantal alternatieven uitgewerkt die kunnen gebruikt worden als antwoord op de bestaande problemen van stereofone echo-onderdrukking. Praktische resultaten hiervan zijn bijgevoegd waar relevant.

Uiteindelijk worden dan de verschillende onderdelen van een vrij recent en performant algoritme voor stereo echo-onderdrukking - een LMS/Newton-algoritme - besproken en een eigen implementatie ervan wordt getest aan de hand van een reeks simulaties.

1.2.2 Stereo Codering

De transmissie van de spraak- en/of audiosignalen bij bv. een conferentiesessie gebeurt over het algemeen efficiënter via codering van de doorgezonden data. Dit is dan ook het onderdeel van hoofdstuk 3. Eerst worden een aantal veel voorkomende coderingsschema's kort ingeleid, waarbij telkens een vergelijking wordt gemaakt tussen het beschouwde schema en dat gebaseerd op lineaire predictie. Vervolgens wordt lineaire predictie zelf theoretisch uit de doeken gedaan voor een monofoon ingangssignaal en wordt de link met het gekende model voor spraak benadrukt en uitgelegd. Als laatste geeft dit hoofdstuk een bespreking van de uitbreiding van lineaire predictie, nu toegepast op stereofone spraak en audio, dewelke gepaard gaat met simulaties van een eigen implementatie en de invloed van de voornaamste convergentieparameters op de performantie.

1.2.3 Integratie

Het laatste inhoudelijk hoofdstuk bevat tot slot de theoretische en praktische informatie in verband met twee voorgestelde algoritmes voor de integratie van de begrippen beschouwd in hoofdstukken 2 en 3. De verschillende aanpak in de beide methoden wordt eerst theoretisch naar voor gebracht met de nodige referenties naar hergebruikte formules uit vorige hoofdstukken en duidelijke uiteenzettingen van nieuw geïntroduceerde

vergelijkingen. Daarna worden voor de afzonderlijke algoritmes praktische resultaten weergegeven per sectie, indien mogelijk. Ten slotte wordt, zoals in vorig hoofdstuk, de invloed van een aantal parameters op de finale performantie van het geïntegreerd geheel bekeken en een meer praktische implementatie (m.n. met additie van ruis) gerealiseerd.

Stereo Akoestische Echo-onderdrukking

2.1 Inleiding

In dit hoofdstuk wordt er dieper in gegaan op één van de twee signaalverwerkingstechnieken die aan bod komen in dit werk, m.n. stereo akoestische echo-onderdrukking. In eerste instantie wordt in deel 2.2 het akoestisch echo-onderdrukkingprobleem in algemene termen besproken, door te beginnen met een vereenvoudigd geval, nl. akoestische echo-onderdrukking voor monofone signalen.

Vervolgens wordt het probleem verruimd in deel 2.3 en volgt een theoretische bespreking van akoestische echo-onderdrukking voor het stereofone geval, wat hier het doel is. Hoewel gerelateerd aan het monofone geval, zal blijken dat er een aantal fundamentele verschillen zijn die een uitbreiding naar meerdere kanalen bemoeilijken. Aanvankelijk wordt het probleem opnieuw geschetst in relatief algemene termen door middel van een intuïtieve benadering, daarna wordt er dieper ingegaan op de onderliggende theorie. Een overzicht van de voornaamste literatuur wordt geschetst. Om de theorie tot slot wat bevattelijker te maken worden in deel 2.3.6 een aantal eenvoudige simulaties gegeven.

In deel 2.4 wordt dan een aantal oplossingen die voorgesteld zijn in de literatuur kort besproken. Deze lijst is niet exhaustief, maar geeft een idee van de verschillende soorten van oplossingen die mogelijk zijn. Dit ook om het contrast aan te tonen met de methodes gebaseerd op lineaire predictie in een later deel besproken.

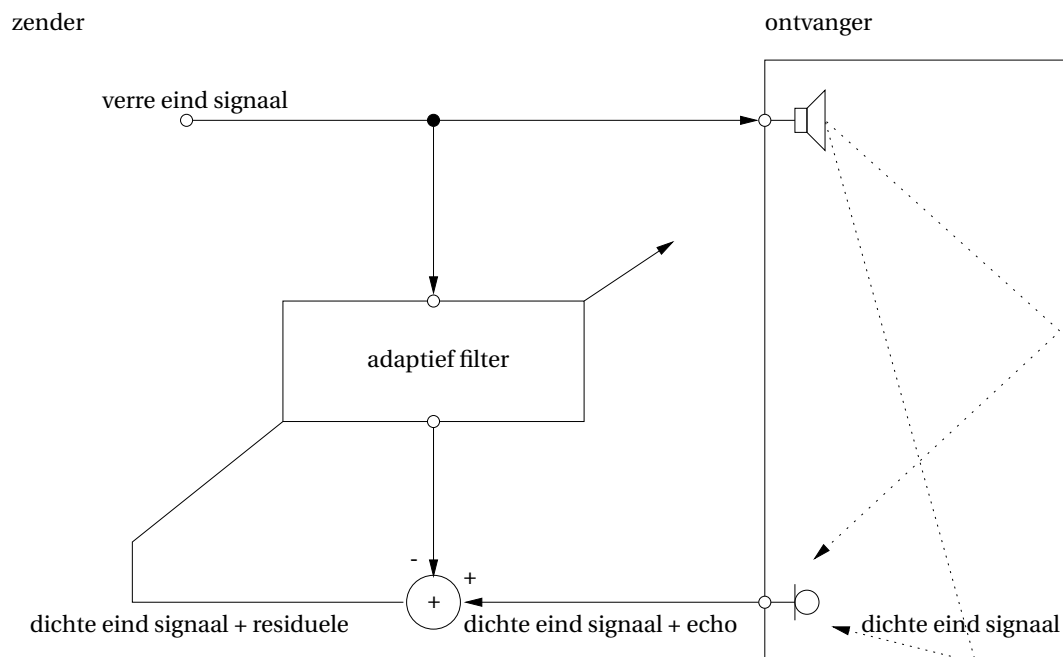
Betreffende lineaire predictie wordt in een deel 2.5 een uiteenzetting gedaan van twee adaptieve algoritmes die hier gebruik van maken. De redenen waarom dit een interessant alternatief is worden uitvoeriger in dit deel zelf besproken, maar één van de voornaamste redenen is dat veel van deze principes opnieuw voorkomen in het deel over stereo (spraak) codering, in hoofdstuk 3, wat van belang is voor de uiteindelijke integratie in hoofdstuk 4. Niet alleen wordt de hele theorie besproken, eigen simulaties worden ook gegeven en bediscussieerd.

Het hoofdstuk wordt uiteindelijk besloten in deel 2.6 met een samenvatting en bijhorende bedenkingen.

2.2 (Mono) Akoestische Echo-onderdrukking

In de inleiding zijn een aantal concrete toepassingen aangehaald die het probleem van akoestische echo's illustreren. Een eerste punt is dat het effect van de echo's hinderlijk kan zijn in een gewone conversatie. Daarnaast bestaat ook de mogelijkheid dat er op deze manier een gesloten lus systeem ontstaat, dat mogelijk onstabiel wordt met hinderlijke fluittonen tot gevolg. Om met behulp van signaalverwerking tot een oplossing te komen wordt er beroep gedaan op adaptieve signaalverwerkingstechnieken [4, 5]. De aanpak bestaat er in om het effect van de echo's te modelleren zodanig dat deze achteraf kan verwijderd worden.

Om te beginnen zullen hier de basisprincipes opnieuw uitgelegd worden met behulp van het eenvoudigste geval: mono akoestische echo-onderdrukking. Het principeschema hier voor is te zien in figuur 2.1. De linkerkant toont de zendkant oftewel het *verre eind signaal*. De rechterkant toont de ontvangerkant of het *dichte eind signaal*. Aan de ontvangerkant wordt expliciet de akoestische ruimte aangegeven, een gelijkaardig schema geldt voor de zendkant. Het probleem met de echo's komt voort uit het feit dat de luidspreker voor het verre eind signaal en de microfoon voor het dichte eind signaal zich in de zelfde akoestische ruimte bevinden en dat het verre eind signaal voor een deel terug komt op het dichte eind signaal. Aangezien het verre eind signaal gekend is zou een mogelijke oplossing er in kunnen bestaan dit gewoon terug af te trekken van het dichte eind signaal. Door het effect van de akoestische ruimte wordt het verre eind signaal echter vervormd tot een *echo* en is dit niet mogelijk.



Figuur 2.1: Akoestische echo-onderdrukkingsschema

Om toch te kunnen compenseren voor het effect van de akoestische ruimte wordt een

model opgesteld voor het *echopad*. Het model komt dan neer op een filter dat hetzelfde effect zou hebben als het echopad op het signaal, d.i. de gehele akoestische ruimte van luidspreker naar microfoon wordt gekarakteriseerd door een impulsresponsie. Praktisch worden hier altijd *FIR* filters gebruikt omwille van hun gegarandeerde stabiliteit.

Aangezien de eigenschappen van de akoestische ruimte en de bijhorende impulsresponsie variabel zijn in functie van de tijd wordt gebruik gemaakt van adaptieve filters, dit zijn filters waarvan de coëfficiënten veranderen met de tijd. In strikte zin zijn filterkarakteristieken invariant in functie van de tijd. Adaptieve signaalverwerking gaat er van uit dat voor deze specifieke toepassingen de eigenschappen van het signaal stationair zijn binnen het tijdsvenster waarin gewerkt wordt [6].

Gegeven een adaptief filter dat het echopad benadert over de tijd, kan de echo gegenereerd door het echopad gereconstrueerd worden en bijgevolg ook afgetrokken worden van het vervormde dichte eind signaal, wat het oorspronkelijk opzet is. Daar het adaptief filter nooit volledig overeen komt met het echopad zal er zich op het dichte eind signaal steeds een *residu* bevinden. Deze informatie kan dan gebruikt worden voor het algoritme dat het adaptief filter gebruikt om de filtercoëfficiënten bij te stellen.

Voor een overzicht van de meest gangbare adaptieve algoritmes wordt opnieuw gerefereerd naar Haykin [4].

2.3 Uitbreiding naar stereo: het fundamentele probleem

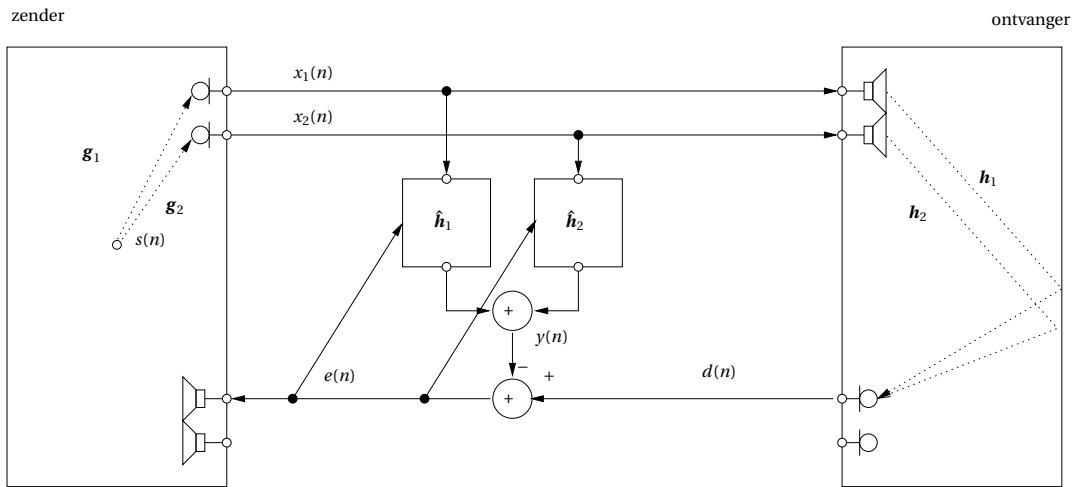
In dit deel wordt overgegaan tot het tweekanaalsgeval en de bijhorende problemen. Hoewel dit op het eerste zicht triviaal lijkt, zal snel blijken dat stereo akoestische echo-onderdrukking een aantal fundamentele problemen met zich mee brengt in vergelijking met het mono geval.

2.3.1 Opstelling Stereo Akoestische Echo-onderdrukking

Het principeschema voor de uitbreiding naar twee kanalen wordt getoond in figuur 2.2. Qua opstelling is dit vrij gelijkaardig aan dat voor het éénkanaals geval. In de volgende korte uiteenzetting wordt het schema opnieuw behandeld, hetzij iets meer expliciet, met verwijzing naar de variabelen zoals deze voor de rest zullen gebruikt worden. De linkerkant toont weer het verre eind, de kant waar het bronsignaal $s(n)$ zich bevindt. In plaats van één microfoon, bevinden er zich nu twee microfoons om dit signaal op te vangen. Voor ieder van de microfoons wordt een tijdsveranderlijk echopad g_1, g_2 verondersteld naar het bronsignaal toe. In tegenstelling tot het mono geval, zal blijken dat de echopaden van het verre eind ook hun invloed hebben op het schatten van de echopaden van het dichte eind. De resulterende signalen aan de zendkant $x_1(n), x_2(n)$ kunnen nu als volgt uitgeschreven worden:

$$x_i(n) = s(n) * g_i, \quad i = 1, 2 \quad (2.1)$$

Aan de ontvangkant bevinden er zich gelijkaardig twee luidsprekers, waarbij voor iedere luidspreker twee echopaden h_1, h_2 verondersteld worden. Weergegeven op de figuur zijn slechts de twee echopaden voor één microfoon, een vereenvoudiging die in de literatuur



Figuur 2.2: Stereo akoestische echo-onderdrukking schema

vaak gedaan wordt [2, 3]. In de rest van de bespreking zal deze vereenvoudiging ook aangehouden worden.

Het gewenste signaal $d(n)$ kan in functie van de verre eind kamerimpulsresponsies uitgeschreven worden, gelijkaardig aan vergelijking (2.1)

$$d(n) = x_1(n) * h_1(n) + x_2(n) * h_2(n) \quad (2.2)$$

De adaptieve filters $\hat{h}_1, \hat{h}_2$ volgen de overeenkomstige dichte eind kamerimpulsresponsies, hun uitgangssignaal is

$$y(n) = x_1(n) * \hat{h}_1(n) + x_2(n) * \hat{h}_2(n) \quad (2.3)$$

Het residu $e(n)$ wordt nu berekend als het verschil

$$e(n) = d(n) - y(n) \quad (2.4)$$

en vervolgens terug doorgegeven naar de oorspronkelijke zendkant.

2.3.2 Niet-uniciteitsprobleem: het probleem van niet unieke echopaden

In tegenstelling tot het éénkanaals geval, hebben de echopaden van zowel de zend- als ontvangkant een invloed op het convergentiegedrag van de adaptieve filters. Een eerste, intuïtieve manier om dit in te zien is als volgt [2].

Intuïtieve benadering

Definieer eerst de vectoren $\mathbf{x}$ en $\hat{\mathbf{h}}$ als de concatenatie van ieder van de signaalvectoren, e.g.:

$$\mathbf{x} = [x_1(n) \ x_2(n)]^T \quad (2.5)$$

en gelijkaardig voor $\hat{\mathbf{h}}$. Definieer voor de dichte eind kamerimpulsresponsie de *uitlijningsvector*

$$\tilde{\mathbf{h}} \equiv \mathbf{h} - \hat{\mathbf{h}} \quad (2.6)$$

Het foutsignaal $e(n)$ kan dan geschreven worden als

$$e(n) = y(n) - \hat{\mathbf{h}}_1^T \mathbf{x}_1 - \hat{\mathbf{h}}_2^T \mathbf{x}_2 \quad (2.7)$$

en aangezien

$$y(n) = \mathbf{h}_1^T \mathbf{x}_1 + \mathbf{h}_2^T \mathbf{x}_2 \quad (2.8)$$

geldt met de eerder ingevoerde notatie

$$e(n) = (\mathbf{h} - \hat{\mathbf{h}})^T \mathbf{x} = \tilde{\mathbf{h}}^T \mathbf{x} \quad (2.9)$$

Het doel is nu om de fout $e(n)$ steeds zo klein mogelijk te houden. Onafgezien van het gebruikte algoritme om deze fout te minimaliseren, veronderstel dat $e(n) \rightarrow 0$. Dan geldt

$$\tilde{\mathbf{h}}_1^T \mathbf{x}_1 + \tilde{\mathbf{h}}_2^T \mathbf{x}_2 = 0 \quad (2.10)$$

In de veronderstelling dat er één enkel bronsgignaal is, kan dit herschreven worden als

$$(\tilde{\mathbf{h}}_1 * \mathbf{g}_1 + \tilde{\mathbf{h}}_2 * \mathbf{g}_2) * s(n) = 0 \quad (2.11)$$

In het Fourier domein wordt dit

$$(\tilde{H}_1(f)G_1(f) + \tilde{H}_2(f)G_2(f))S(f) = 0 \quad (2.12)$$

waarbij de symbolen in hoofdletter de Fourier getransformeerden voorstellen van de corresponderende variabelen met kleine letter. De voorwaarde voor complete uitlijning, d.i. de adaptieve filters corresponderen volledig met de kamerimpulsresponsies, is deze waar de volgende voorwaarde voor de uitlijningsvector aan voldaan is

$$\tilde{\mathbf{h}}(n) \rightarrow 0 \quad (2.13)$$

of

$$\mathbf{h}(n) - \hat{\mathbf{h}}(n) \rightarrow 0 \quad (2.14)$$

In het éénkanaals geval, d.i. $G_2(f) = 0$ wordt de voorwaarde van vergelijking (2.13) herleid tot $\tilde{H}_1(f) = 0$ en afgezien van de nulpunten van $G_1(f)S(f)$ wordt uitlijning bereikt.

Voor twee kanalen is dit niet het geval: in de veronderstelling dat $S(f)$ geen nullen in het frequentiedomein bevat, is het beste geval dat kan bereikt worden

$$\tilde{H}_1(f)G_1(f) + \tilde{H}_2(f)G_2(f) = 0 \quad (2.15)$$

Bemerk dat vergelijking (2.15) over het algemeen *niet inhoudt* dat $\tilde{H}_1(f) = \tilde{H}_2(f) = 0$, wat hier de voorwaarde is voor volledige uitlijning. Verder toont deze vergelijking duidelijk de afhankelijkheid van de verre eind kamerimpulsresponsies voor het bereiken van volledige uitlijning. Stel dat kamerimpulsresponsies aan het dichte eind onveranderlijk zouden zijn, dan maakt een verandering aan het verre eind, e.g. doordat een spreker zich in de akoestische ruimte verplaatst, dat er niet meer voldaan is aan de toestand van volledige uitlijning.

Invloed lengte kamerimpulsresponsies

Uit de vorige bespreking is duidelijk geworden dat het tweekanaals echo- onderdrukingsprobleem inherent slecht geconditioneerd is. Om dit beter te kwantificeren moet gekeken worden naar de lengtes van de verschillende kamerimpulsresponsies L_F , L_N en L_M , respectievelijk de lengtes voor de zendkant, de ontvangkant en de adaptieve filters. Het effect op de conditionering van het probleem wordt hier verder uitgezet, uitgaande van een kleinste kwadraten kostfunctie $J(n)$ [3]

$$J(n) = \sum_{p=1}^n \lambda^{n-p} e^2(p) \quad (2.16)$$

waarbij $0 < \lambda \leq 1$ een exponentiële vergeetfactor is en $e(n)$ het foutsignaal uit vergelijking (2.4). Qua notatie betekent voor een echopad $\mathbf{h}_L(n)$, dat het echopad $\mathbf{h}(n)$ een lengte L heeft.

Minimalisatie van de kostenfunctie (2.16) leidt tot de normaalvergelijking

$$\mathbf{R}(n) \begin{bmatrix} \hat{\mathbf{h}}_{1,L_M}(n) \\ \hat{\mathbf{h}}_{2,L_M}(n) \end{bmatrix} = \mathbf{r}(n) \quad (2.17)$$

waarbij

$$\mathbf{R}(n) = \sum_{p=1}^n \lambda^{n-p} \begin{bmatrix} \hat{\mathbf{x}}_{1,L_M}(p) \\ \hat{\mathbf{x}}_{2,L_M}(p) \end{bmatrix} \begin{bmatrix} \hat{\mathbf{x}}_{1,L_M}(n) & \hat{\mathbf{x}}_{2,L_M}(n) \end{bmatrix} \quad (2.18)$$

$$= \begin{bmatrix} \mathbf{R}_{11}(n) & \mathbf{R}_{12}(n) \\ \mathbf{R}_{21}(n) & \mathbf{R}_{22}(n) \end{bmatrix} \quad (2.19)$$

een benadering is van de autocorrelatie van het ingangssignaal en

$$\mathbf{r}(n) = \sum_{p=1}^n \lambda^{n-p} d(p) \begin{bmatrix} \hat{\mathbf{x}}_{1,L_M}(n) \\ \hat{\mathbf{x}}_{2,L_M}(n) \end{bmatrix} \quad (2.20)$$

$$= \begin{bmatrix} \mathbf{r}_1(n) \\ \mathbf{r}_2(n) \end{bmatrix} \quad (2.21)$$

een benadering is van de kruiscorrelatievector van de ingangs- en uitgangssignalen. Er wordt verondersteld dat de autocorrelatiematrices voor de ingangssignalen zelf inverteerbaar zijn. Het probleem van de niet unieke echopaden komt mathematisch nu overeen met de vraag of de autocorrelatiematrix $\mathbf{R}(n)$ singulier is of niet, wat bepaald wordt door de lengtes van de verschillende beschouwde impulsresponsies. Twee gevallen worden besproken

$L_M \geq L_F$ Dit komt overeen met het geval waarin de impulsresponsies van de adaptieve filters groter zijn dan deze van de verre eind echopaden. Veronderstel de vector

$$\mathbf{w}(n) = [\mathbf{g}_{2,L_N} \ 0 \ \dots \ 0 - \mathbf{g}_{1,L_N} \ 0 \ \dots \ 0]^T \quad (2.22)$$

en $\mathbf{w}(n)$ bevat $L_M - L_N$ nullen. Dit invullen in de normaalvergelijking (2.17) geeft

$$\mathbf{R}(n) \mathbf{w}(n) = 0 \quad (2.23)$$

Een manier om dit te zien is door terug te gaan naar de eerdere intuïtieve benadering waar als bronsignaal $\mathbf{s}(n)$ één enkele spreker verondersteld werd. Aangezien in dit geval

$$\begin{aligned} X_1(f) &= G_1(f)S(f) \\ X_2(f) &= G_2(f)S(f) \end{aligned} \quad (2.24)$$

geldt ook

$$X_1(f)G_2(f) = X_2(f)G_1(f) \quad (2.25)$$

of in het tijdsdomein

$$\mathbf{x}_{1,L_F}^T(n)\mathbf{g}_{2,L_F} = \mathbf{x}_{2,L_F}^T(n)\mathbf{g}_{1,L_F} \quad (2.26)$$

en gezien de definitie van $\mathbf{R}(n)$ in vergelijking (2.18) gaat de normaalvergelijking (2.23) met $\mathbf{w}(n)$ op.

Hier blijkt direct uit dat $\mathbf{R}(n)$ singulier is. Voor $L_M \geq L_F$ gelden voor de adaptieve filters volgende niet-unieke oplossingen

$$\begin{aligned} \hat{\mathbf{h}}_{1,L_M} &= \mathbf{h}_{1,L_M} + \beta \left[\mathbf{g}_{2,L_F}^T \ 0 \ \dots \ 0 \right]^T \\ \hat{\mathbf{h}}_{2,L_M} &= \mathbf{h}_{2,L_M} + \beta \left[-\mathbf{g}_{1,L_F}^T \ 0 \ \dots \ 0 \right]^T \end{aligned} \quad (2.27)$$

waarbij β een arbitraire constante is. Bemerk dat net zoals in de eerdere intuïtieve benadering de oplossing voor de adaptieve filters impliciet afhankelijk is van de kamerimpulsresponsies van het verre eind. Vergelijking (2.23) kan ook gezien worden als een eigenwaardenprobleem; bijgevolg wordt er in vergelijking (2.27) expliciet van uit gegaan dat de covariantiematrix $\mathbf{R}(n)$ slechts één eigenwaarde heeft gelijk aan 0 en dat $\mathbf{w}(n)$ de bijhorende eigenvector is, wat juist is als $\mathbf{g}_{1,L_F}$ en $\mathbf{g}_{2,L_F}$ geen samenvallende nulpunten hebben en als het bronsignaal van volledige rang is. Als $\mathbf{R}(n)$ meerdere eigenwaarden gelijk heeft aan nul zullen de oplossingen voor $\mathbf{h}_{1,L_M}$ en $\mathbf{h}_{2,L_M}$ nog steeds expliciet afhankelijk zijn van $\mathbf{g}_{1,L_F}$ en $\mathbf{g}_{2,L_F}$.

$L_M < L_F$ Dit is het geval waarbij de lengte van de gemodelleerde kamerimpulsresponsies korter zijn dan de echopaden van het verre eind. In de praktijk gaat dit altijd het geval zijn, aangezien fysische echopaden in realiteit altijd van oneindige lengte zijn [7]. De lineaire afhankelijkheid tussen de $\mathbf{x}_i(n)$ beschreven in vergelijking (2.26) kan geschreven worden als

$$\mathbf{x}_{1,L_M}^T(n)\mathbf{g}_{2,L_M} + q_1(n-L_M) = \mathbf{x}_{2,L_M}^T(n)\mathbf{g}_{1,L_M} + q_2(n-L_M) \quad (2.28)$$

waarbij

$$q_1(n-L_M) = \sum_{i=L_M}^{L_F-1} x_1(n-i)g_{2,i} \quad (2.29)$$

$$q_2(n-L_M) = \sum_{i=L_M}^{L_F-1} x_2(n-i)g_{1,i} \quad (2.30)$$

Uit vergelijking (2.26) volgt dat de $\mathbf{x}_{i,L_F}$ lineair afhankelijk zijn, maar uit vergelijking (2.28) volgt dat de $\mathbf{x}_{i,L_M}$ dat *niet* zijn, tenzij in het uitzonderlijke geval dat $q_1(n-L_M) = q_2(n-L_M)$,

wat enkel voorkomt als $\mathbf{g}_{1,L_F}$ en $\mathbf{g}_{2,L_F}$ minstens $L_F - L_M$ gemeenschappelijke nulpunten delen. Bijgevolg is de covariantiematrix $\mathbf{R}(n)$ in principe van volledige rang, doch slecht geconditioneerd aangezien $q_1(n - L_M)$ en $q_2(n - L_M)$ over het algemeen zeer klein zijn.

Een maatgetal om de lineaire relatie tussen de x_{i,L_M} aan te geven is

$$\zeta^2 = \frac{\sum_{i=1}^2 \mathbf{g}_{i,L_M}^T \mathbf{g}_{i,L_M}}{\sum_{i=1}^2 \mathbf{g}_{i,L_F}^T \mathbf{g}_{i,L_F}} \quad (2.31)$$

in het geval dat het niet eindig zijn van de kamerimpulsresponsies in rekening wordt gebracht. In het geval dat $L_M = L_F$ geldt dat $\zeta = 1$ en m.a.w. is er voldaan aan vergelijking (2.26) en zijn de $\mathbf{x}_{i,L_M}$ lineair afhankelijk van elkaar. Algemeen geldt $0 < \zeta \leq 1$. Hoe lager ζ , hoe beter het conditiegetal van de covariantiematrix.

2.3.3 Uitlijningsprobleem: maat voor niet unieke echopaden

In de intuïtieve benadering van deel 2.3.2 is de uitlijningsvector $\tilde{\mathbf{h}}$ aangehaald als een maat voor het afwijken van de adaptieve filters $\hat{\mathbf{h}}$ t.o.v. de echopaden $\mathbf{h}$ die ze proberen te modelleren. Een maatgetal gerelateerd aan de uitlijningsvector is de *uitlijning* ϵ [3]

$$\epsilon = \frac{\|\mathbf{h} - \hat{\mathbf{h}}\|}{\|\mathbf{h}\|} \quad (2.32)$$

Bemerk dat in dit geval de lengte van $\mathbf{h}$, $\hat{\mathbf{h}}$ verondersteld wordt L_M te zijn. In dit deel wordt het effect van de kruiscorrelatie van de ingangssignalen bekeken op de uitlijning [3].

Eénkanaalsgeval

In het mono geval worden de normaalvergelijkingen (2.17) herleid tot het eenvoudiger geval

$$\mathbf{R}_{11}(n) \hat{\mathbf{h}}_{1,L_M}(n) = \mathbf{r}_1(n) \quad (2.33)$$

en waarbij $\mathbf{R}_{11}(n)$, $\mathbf{r}_1(n)$ af te leiden zijn volgens vergelijking (2.19), respectievelijk vergelijking (2.21). Om het effect van het verschil in lengte tussen het adaptief filter $\hat{\mathbf{h}}_i$ en het echopad dat ze probeert te modelleren $\mathbf{h}_i$ uit te drukken, d.i. het verschil tussen L_M en L_N , kan men de kamerimpulsresponsie voor het dichte eind opsplitsen

$$\mathbf{h}_{1,L_N} = \begin{bmatrix} \mathbf{h}_{1,L_M} \\ \mathbf{h}_{1,t} \end{bmatrix} \quad (2.34)$$

d.i. een gedeelte $\mathbf{h}_{1,L_M}$ dat gemodelleerd kan worden door het adaptief filter en een zgn. “staart” die hier buiten valt. Het microfoonsignaal van vergelijking (2.3) kan dan verder worden uitgeschreven als

$$\begin{aligned} \mathbf{y}_1(n) &= \mathbf{h}_{1,L_N}^T \mathbf{x}_{1,L_N}(n) \\ &= \mathbf{h}_{1,L_M}^T \mathbf{x}_{1,L_N}(n) + \mathbf{h}_{1,t}^T \mathbf{x}_{1,t}(n - L_M) \end{aligned} \quad (2.35)$$

De oplossing van de normaalvergelijkingen is dan, in de veronderstelling dat $\mathbf{R}_{11}(n)$ niet singulier is

$$\hat{\mathbf{h}}_{1,L_M}(n) = \mathbf{h}_{1,L_M} + \mathbf{R}_{11}^{-1}(n) \mathbf{R}_{11,t}(n) \mathbf{h}_{1,t} \quad (2.36)$$

waarbij $\mathbf{R}_{11,t}(n)$ gelijkaardig is aan $\mathbf{R}_{11}(n)$

$$\mathbf{R}_{11,t}(n) = \sum_{p=1}^n \lambda^{n-p} \mathbf{x}_{1,L_M}(p) \mathbf{x}_{1,t}^T(p - L_M) \quad (2.37)$$

Interessant om op te merken aan de oplossing voor $\hat{\mathbf{h}}_1$ is dat deze naast het gezochte echopad $\mathbf{h}_1$ ook nog een extra bijdrage bevat, afhankelijk van de “staart” van het echopad, die in een reële situatie er altijd zal zijn, wegens het niet eindig zijn van de kamerimpuls-responsie.

In termen van het maatgetal uitlijning, is de minimale uitlijning $\epsilon_{\min}$ die volgens de kleinste kwadraten kostenfunctie kan bereikt worden

$$\begin{aligned} \epsilon_{\min}^2(n) &= \frac{\left[\mathbf{h}_{1,L_M} - \hat{\mathbf{h}}_{1,L_M}(n) \right]^T \left[\mathbf{h}_{1,L_M} - \hat{\mathbf{h}}_{1,L_M}(n) \right]}{\mathbf{h}_{1,L_M}^T \mathbf{h}_{1,L_M}} \\ &= \frac{\mathbf{h}_{1,t}^T \mathbf{Q}_{11,t}(n) \mathbf{h}_{1,t}}{\mathbf{h}_{1,L_M}^T \mathbf{h}_{1,L_M}} \end{aligned} \quad (2.38)$$

waarbij de hulpvariabele $\mathbf{Q}_{11,t}(n)$

$$\mathbf{Q}_{11,t}(n) = \mathbf{R}_{11,t}^T(n) \mathbf{R}_{11}^{-2}(n) \mathbf{R}_{11,t}(n) \quad (2.39)$$

Gelijkaardig aan het niet-uniciteitsprobleem kunnen twee gevallen verondersteld worden, afhankelijk van de relatieve lengtes van de echopaden en hun model, d.i. L_M in vergelijking met L_N .

$L_M \geq L_N$ Dit is het geval waarin de lengte van het adaptief filter groter is dan dat van het echte echopad. In dit geval bestaat er niet zoiets als een staart en geldt $\epsilon_{\min} = 0$. Bemerkt dat dit in de praktijk niet kan voorkomen.

$L_M < L_N$ Dit is het geval waarin het adaptief filter een kleinere lengte heeft dan het echte echopad. In dit geval wordt verondersteld dat voor een L_M die groot genoeg is de autocorrelatie voor de staart verwaarloosbaar wordt, d.i. $\mathbf{R}_{11,t}(n) \approx \mathbf{0}_{L_M \times (L_N - L_M)}$ waardoor $\mathbf{Q}_{11,t}(n)$ en $\epsilon_{\min}$ zeer klein worden.

Uit deze uiteenzetting kan besloten worden dat een slechte uitlijning voor het éénkanaalsgeval vrij uitzonderlijk is.

Tweekanaalsgeval

De normaalvergelijking (2.18) komt hier terug voor in zijn originele vorm, waarbij de kamerimpulsresponsies (cfr. het éénkanaalsgeval) opnieuw kunnen worden uitgeschreven in twee delen

$$\mathbf{h}_{i,L_N} = \begin{bmatrix} \mathbf{h}_{i,L_M} \\ \mathbf{h}_{i,t} \end{bmatrix} \quad i = 1, 2 \quad (2.40)$$

Het microfoonsignaal wordt in dit geval

$$\begin{aligned} \mathbf{y}(n) &= \sum_{i=1}^2 \mathbf{h}_{i,L_N}^T \mathbf{x}_{i,L_N}(n) \\ &= \sum_{i=1}^2 \mathbf{h}_{i,L_M}^T \mathbf{x}_{i,L_M}(n) + \sum_{i=1}^2 \mathbf{h}_{i,t}^T \mathbf{x}_{i,t}(n - L_M) \end{aligned} \quad (2.41)$$

In de veronderstelling dat $\mathbf{R}(n)$ niet singulier is, is de oplossing te schrijven als

$$\begin{bmatrix} \hat{\mathbf{h}}_{1,L_M}(n) \\ \hat{\mathbf{h}}_{2,L_M}(n) \end{bmatrix} = \begin{bmatrix} \mathbf{h}_{1,L_M} \\ \mathbf{h}_{2,L_M} \end{bmatrix} + \mathbf{R}^{-1}(n) \mathbf{R}_t(n) \begin{bmatrix} \mathbf{h}_{1,t} \\ \mathbf{h}_{2,t} \end{bmatrix} \quad (2.42)$$

waarbij de $\mathbf{R}_t(n)$ gelijkaardig is aan de blokmatrix $\mathbf{R}(n)$

$$\mathbf{R}_t(n) = \sum_{p=1}^n \lambda^{n-p} \begin{bmatrix} \mathbf{x}_{1,L_M}(p) \\ \mathbf{x}_{2,L_M}(p) \end{bmatrix} \begin{bmatrix} \mathbf{x}_{1,L_M}(p - L_M) & \mathbf{x}_{2,L_M}(p - L_M) \end{bmatrix} \quad (2.43)$$

$$= \begin{bmatrix} \mathbf{R}_{11,t}(n) & \mathbf{R}_{12,t}(n) \\ \mathbf{R}_{21,t}(n) & \mathbf{R}_{22,t}(n) \end{bmatrix} \quad (2.44)$$

De minimale uitlijning bekomen onder een recursieve kleinste kwadraten kostenfunctie is hier

$$\begin{aligned} e_{\min}^2(n) &= \frac{\sum_{i=1}^2 \left[\mathbf{h}_{i,L_M} - \hat{\mathbf{h}}_{i,L_M}(n) \right]^T \left[\mathbf{h}_{i,L_M} - \hat{\mathbf{h}}_{i,L_M}(n) \right]}{\mathbf{h}_{1,L_M}^T \mathbf{h}_{1,L_M}} \\ &= \frac{\mathbf{h}_t^T \mathbf{Q}_t(n) \mathbf{h}_t}{\mathbf{h}_{L_M}^T \mathbf{h}_{L_M}} \end{aligned} \quad (2.45)$$

met vectoren $\mathbf{h}$, $\mathbf{h}_t$

$$\begin{aligned} \mathbf{h} &= \begin{bmatrix} \mathbf{h}_{1,L_M}^T & \mathbf{h}_{2,L_M}^T \end{bmatrix}^T \\ \mathbf{h}_t &= \begin{bmatrix} \mathbf{h}_{1,t}^T & \mathbf{h}_{2,t}^T \end{bmatrix}^T \end{aligned}$$

en

$$\mathbf{Q}_t(n) = \mathbf{R}_t^T(n) \mathbf{R}^{-2}(n) \mathbf{R}_t(n) \quad (2.46)$$

In de behandeling van het niet-uniciteitsprobleem is gebleken dat een nodige voorwaarde voor het niet singulier zijn van $\mathbf{R}$ $L_M < L_F$ is, een voorwaarde waar in de realiteit altijd aan voldaan is en dit wordt hier dan ook impliciet aangenomen.

Twee gevallen worden behandeld:

$L_M \geq L_N$ De adaptieve filters zijn langer dan de kamerimpulsresponsies. In dit geval zijn er geen staarten in de kamerimpulsresponsies en bijgevolg $e_{\min} = 0$.

$L_M < L_N$ De reële situatie waarin de adaptieve filters korter zijn dan de echte echopaden. Opnieuw kan voor grote L_M verondersteld worden dat $\mathbf{R}_{11,t}(n) \approx \mathbf{0}_{L_M \times (L_N - L_M)}$ en eveneens $\mathbf{R}_{22,t}(n) \approx \mathbf{0}_{L_M \times (L_N - L_M)}$. Echter, over de termen $\mathbf{R}_{12,t}(n)$, $\mathbf{R}_{21,t}(n)$ kan niets gezegd worden aangezien hun waarde afhankelijk is van de kruiscorrelatie en de vertraging tussen de signalen x_1 en x_2 . Daarbij komt dan nog dat uit de bespreking van het niet-uniciteitsprobleem volgt dat $\mathbf{R}$ over het algemeen slecht geconditioneerd is en dat de factor $\mathbf{Q}_t(n)$, functie van $\mathbf{R}^{-2}$, dan ook niet verwaarloosbaar is.

2.3.4 Impulsresponsie staarteffect: samenvatting niet-uniciteitsprobleem, uitlijningsprobleem

Uit de discussie van het niet-uniciteitsprobleem in deel 2.3.2 en van de uitlijning in deel 2.3.3 blijkt dat de zogenaamde staarten van kamerimpulsresponsies steeds een belangrijke rol spelen. Voor het niet-uniciteitsprobleem blijkt dat het bestaan van een staart ten gevolge van het verre eind er voor zorgt dat, ondanks dat het probleem nog steeds slecht geconditioneerd blijft, een unieke oplossing mogelijk blijft. Voor het probleem van de uitlijning blijkt dat volledige uitlijning moeilijk te bereiken is omwille van het bestaan van een staart ten gevolge van het dichte eind. In de praktijk kan men veronderstellen dat hier steeds aan voldaan is, d.i. $L_M < L_F$ respectievelijk $L_M < L_N$. Om het probleem met de uitlijning te verminderen zou men de lengte van de adaptieve filters langer kunnen maken. In de praktijk is dit niet aantrekkelijk omwille van convergentiesnelheid, geheugenvereisten, numerieke problemen, etc. Daarbij komt ook het feit dat het bestaan van een unieke oplossing net gegarandeerd wordt door het feit dat de $L_M < L_F$. Een andere aanpak bestaat er dan ook in om de kruiscorrelatie die inherent aanwezig is tussen de twee kanalen te verminderen. Immers, in de praktijk zullen stereo signalen inherent een grote mate van kruiscorrelatie vertonen, aangezien de twee kanalen vaak informatie over een zelfde bronsignaal aanleveren. In video-/teleconferencing is dit vrijwel altijd het geval aangezien de twee kanalen de stem van een zelfde spreker bevatten, enkel de spatiale informatie verschilt. Quasi alle voorgestelde oplossingen voor dit probleem zullen dan ook een extra soort van decorrelerende stap bevatten in vergelijking met het éénkanaalsgeval, zoals zal blijken in deel 2.4.

2.3.5 Coherentiefunctie: maat voor decorrelatie in het frequentiedomein

Daar alle oplossingen voor het stereo akoestisch echo-onderdrukingsprobleem een extra decorrelerende stap zullen bevatten, is het interessant om een maat te kunnen invoeren die hier rekening mee houdt. Eén mogelijkheid is om te gaan kijken naar de kruiscorrelatie zelf. In de literatuur wordt echter vaak een andere maat gehanteerd, nl. de coherentiefunctie, die ook een maat is voor (de)correlatie, hetzij in het frequentiedomein [3, 8, 9].

De coherentie zelf is gedefinieerd als

$$\gamma(f) = \frac{S_{x_1 x_2}(f)}{\sqrt{S_{x_1 x_1}(f) S_{x_2 x_2}(f)}} \quad (2.47)$$

waarbij

$$\begin{aligned} S_{x_k x_l}(f) &= \sum_{\tau=-\infty}^{\infty} E\{x_k(n) x_l(n-\tau)\} e^{-j2\pi f\tau} \\ &= \sum_{\tau=-\infty}^{\infty} R_{x_k x_l}(\tau) e^{-j2\pi f\tau}, k, l = 1, 2 \end{aligned} \quad (2.48)$$

gekend is als de *vermogenspectraaldensiteit*, power spectral density of kortweg PSD. De berekening van de PSD als de Fourier getransformeerde van de correlatiefunctie komt voort uit het Wiener-Khinchin theorema [10, p 408-411].

In [3, app. A] wordt een verband tussen de eigenwaarden van de covariantiematrix en de coherentiefunctie verder in detail uitgewerkt. In de praktijk kan de coherentiefunctie vooral gezien worden als een maat voor decorrelatie.

2.3.6 Simulaties

Achtergrond

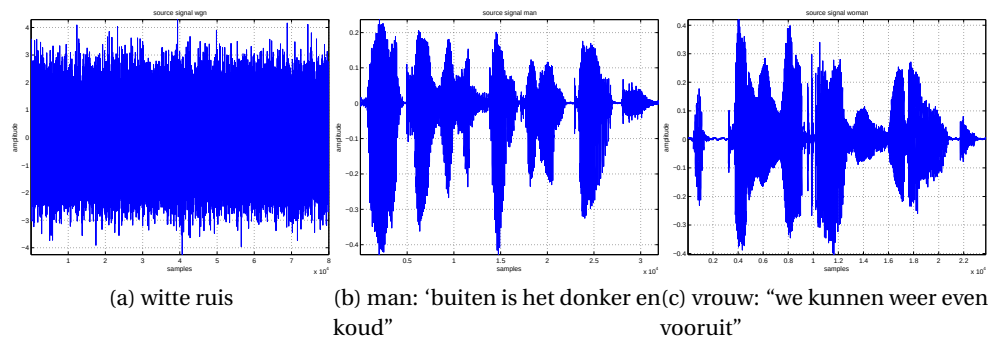
In dit deel worden een aantal simulaties gedaan, voornamelijk om het fundamentele verschil tussen mono en stereo akoestische echo-onderdrukking te benadrukken. De gebruikte signalen, parameters en adaptieve filter algoritmes worden expliciet eenvoudig gehouden in dit deel, om in een later deel over te gaan tot meer geavanceerde oplossingen.

GEBRUIKT ADAPTIEF ALGORITME Het algoritme om het adaptief filter bij te stellen is hier NLMS, genormaliseerde kleinste kwadraten [4]. De reden waarom dit algoritme gebruikt wordt is omdat het eenvoudig is qua opzet doch, althans voor het mono geval, vaak een efficiënt algoritme. Zoals eerder aangegeven zal blijken dat dit voor het stereo geval niet zo efficiënt meer is. De uitbreiding naar stereo wordt gedaan door de opstelling van het mono geval te ontdebellen, d.i. afzonderlijke kamerimpulsresponsies en bijhorende adaptieve filters voor ieder kanaal. De enige parameters die gebruikt worden in het NLMS-algoritme zijn de stapgrootte μ en een term ϵ om versterking van gradiënt ruis te vermijden. Deze zijn hier voor alle gevallen arbitrair gekozen als $\mu = 0.1$ en $\epsilon = 0.001$. Ter herinnering wordt hier de vergelijking gegeven voor het aanpassen van de adaptieve filtercoëfficiënten

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \frac{\mathbf{x}(n)e(n)}{\mathbf{x}^T(n)\mathbf{x}(n) + \epsilon} \quad (2.49)$$

GEBRUIKTE KAMERIMPULSRESPONSIES Voor de kamerimpulsresponsies hier beschouwd wordt gebruik gemaakt van een model: voor zowel de zend- als ontvangkant worden de kamerimpulsresponsies gemodelleerd als onafhankelijke normaal verdeelde sequenties met gemiddelde 0 en een variantie die omgekeerd evenredig afneemt in functie van de tijd $1/n$ [11]. Als lengtes is er voor het verre eind gekozen voor een lengte $L_F = 2048$ en voor het dichte eind voor een lengte $L_N = 1024$. In het geval van stereo zijn er per kamer twee zulke sequenties gegenereerd. Uit een informele luistertest is gebleken dat deze benadering niet volledig overeenstemt met de werkelijkheid. Andere alternatieven zijn dan ook onderzocht geweest, zoals echte kamerimpulsresponsies, evenals een reeks kamerimpulsresponsies die gesynthetiseerd zijn [12]. Bij vergelijking van de resultaten wat betreft het gemiddeld foutsignaal $e(n)$ en de uitlijning blijkt dit echter geen al te grote verschillen te geven, wat deze benadering rechtvaardigt.

GEBRUIKTE SIGNALEN Drie verschillende signalen worden gebruikt als ingangssignaal: een wit normaal verdeeld ruissignaal en twee spraaksignalen. Het ruissignaal is te zien in figuur 2.3a. Beide spraaksignalen zijn bemonsterd aan 44.1kHz, 16 bit resolutie. Het eerste signaal is een mannenstem die de Nederlandse woorden “buiten is het donker en koud” uitspreekt: figuur 2.3b. Het tweede signaal is een vrouwenstem die de zin “we kunnen weer even vooruit” vormt: figuur 2.3c. Uiteindelijk zijn alle signalen herbemonsterd naar een bemonsteringsfrequentie van 16kHz, aangezien de kamerimpulsresponsies deze frequentie veronderstellen.

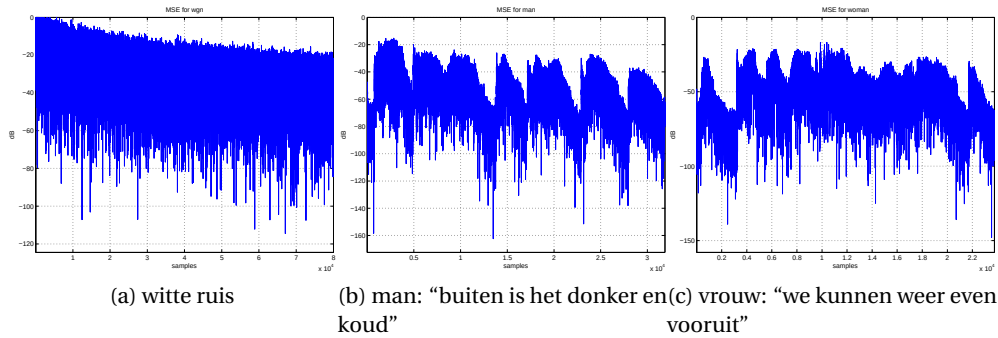


Figuur 2.3: Simulatie fundamenteel probleem, gebruikte ingangssignalen

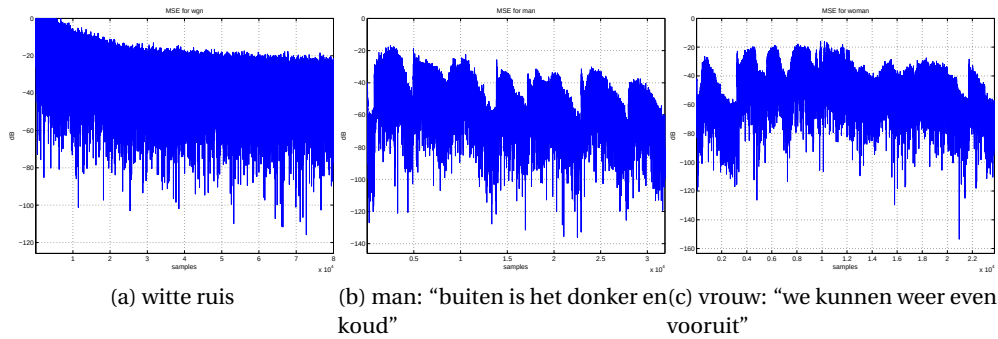
Resultaten

FOUTSIGNALEN De foutsignalen $e(n)$ zijn voor het mono, respectievelijk stereo geval te zien in figuren 2.4 en 2.5. Bemerkt dat voor alle ingangssignalen een relatief goede echo-onderdrukking wordt bekomen, ondanks de eenvoud van het NLMS-algoritme. Specifiek voor de spraaksignalen varieert het foutsignaal mee met het ingangssignaal; voor het ruissignaal is dit niet het geval aangezien het een stationair signaal is. Bij de vergelijking voor het mono en het stereo geval lijkt de performantie nagenoeg gelijk. Zoals eerder gesteld is het ook mogelijk om een goede echo-onderdrukking te hebben, zonder een goede uitlijning te hebben [3]. Het foutsignaal zegt immers niets over hoe ieder van de afzonderlijke adaptieve filters hun overeenkomstig echopad benaderen. Dit wordt echter duidelijk als men de uitlijning bekijkt.

UITLIJNING De uitlijning voor mono en stereo is te zien in figuren 2.6 en 2.7. Over het algemeen is het duidelijk dat de uitlijning steeds dalend is, wat betekent dat over het verloop van de tijd de adaptieve filters steeds beter het echopad benaderen. Hierbij moet wel opgemerkt worden dat voor deze simulatie de echopaden geen functie zijn van de tijd, wat een geïdealiseerde situatie is. Voor zowel mono als stereo verloopt de uitlijning voor het ruissignaal heel continu, wat opnieuw kan verklaard worden door het stationair karakter van het signaal. Het belangrijkste punt is hier de algemene vergelijking tussen het mono en het stereo geval: beiden hebben een gelijkaardig verloop, alleen dat de uitlijning sneller daalt voor het mono geval, wat te verwachten is uit de voorgaande theoretische discussie. De uitlijning zoals hier voorgesteld is slechts een maat die voor beide kanalen

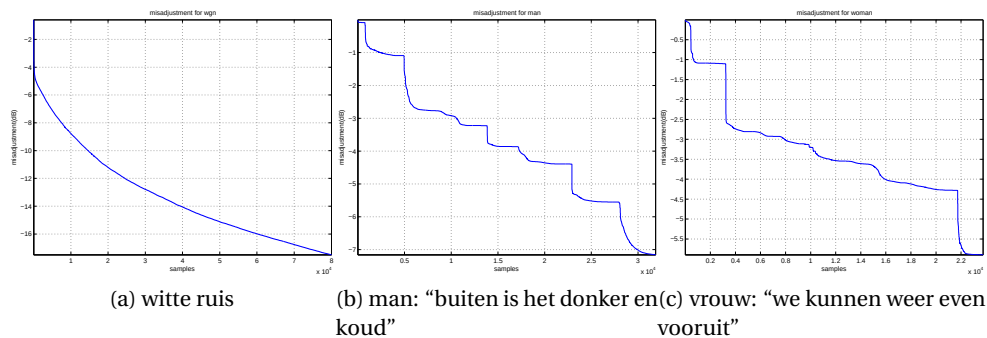


Figuur 2.4: Simulatie fundamenteel probleem foutsignalen, mono

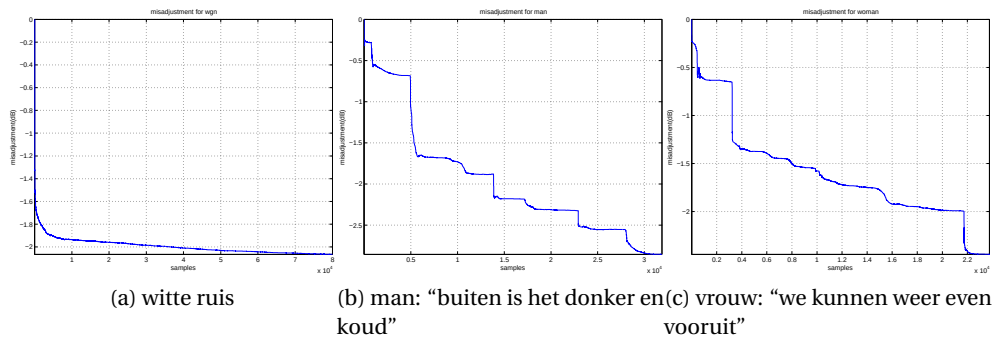


Figuur 2.5: Simulatie fundamenteel probleem foutsignalen, stereo

tegelijk bepaald wordt. Om het fundamentele probleem duidelijker te illustreren is de uitlijning ook nog voor ieder kanaal afzonderlijk berekend. Ter verduidelijking: onder de uitlijning wordt steeds vergelijking (2.32) bedoeld, waarbij in het stereo geval de monsters van de twee kanalen afwisselend worden geplaatst in de vectoren $\mathbf{h}, \hat{\mathbf{h}}$.

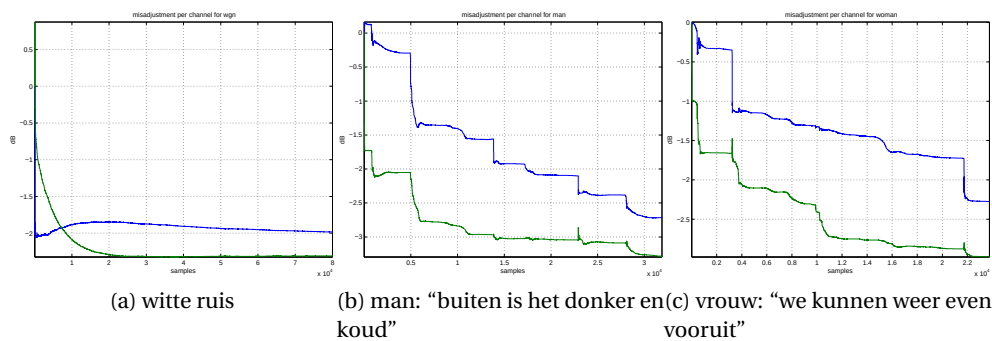


Figuur 2.6: Simulatie fundamenteel probleem uitlijning, mono



Figuur 2.7: Simulatie fundamenteel probleem uitlijning, stereo

UITLIJNING PER KANAAL Figuur 2.8 toont de uitlijning per kanaal, waar ieder van de kleuren voor een ander kanaal staat. Voor het ruissignaal blijkt zelfs dat voor het *ene kanaal de uitlijning verslechtert terwijl deze voor het andere kanaal verbetert*. Dit maakt duidelijk dat het foutsignaal niet het enige criterium kan zijn om te garanderen dat het adaptief filter convergeert naar de kamerimpulsresponsies.

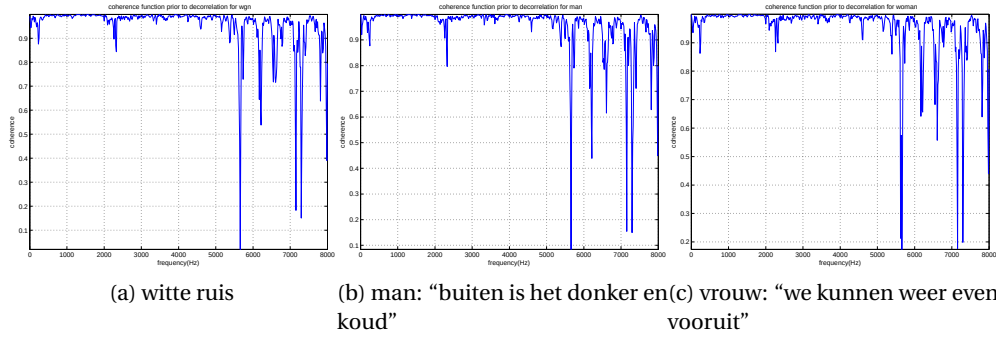


Figuur 2.8: Simulatie fundamenteel probleem uitlijning per kanaal, stereo

COHERENTIEFUNCTIE Tot slot worden voor ieder van de gebruikte ingangssignalen de coherentiefuncties getoond in figuur 2.9. Voor ieder van de signalen blijkt dat er een grote correlatie is tussen de twee kanalen, vooral voor de lagere frequenties.

2.4 Overzicht van alternatieven

Uit de theoretische bespreking en de bijhorende simulaties uit deel 2.3 volgt dat een goede oplossing voor het stereo akoestisch echo-onderdrukingsprobleem altijd een extra decorrelerend stap gaat invoeren vooraleer een adaptief algoritme toe te passen. In dit deel zullen een aantal alternatieven die in de literatuur zijn terug te vinden worden overlopen; voor een aantal zijn enkele simulaties bijgevoegd.



Figuur 2.9: Coherentiefuncties voor de verschillende ingangssignalen, stereo

2.4.1 Toevoegen van willekeurige ruis

Door onafhankelijk ruis toe voegen uit twee ruisbronnen met ieder verschillende stochastische eigenschappen is het evident dat de verschillende signalen kunnen gedecorreleerd worden [2]. Mathematisch geeft dat

$$x'_i(n) = x_i(n) + v_i(n) \quad i = 1, 2 \quad (2.50)$$

In [3] wordt dit verder besproken in termen van de coherentiefunctie: er wordt gesteld dat toevoegen van willekeurige ruis significant de uitlijning kan verbeteren. Dit leidt echter wel tot de volgende afweging: meer ruis toevoegen verbetert de uitlijning, doch zorgt uiteraard ook voor een degradatie van het signaal.

Gerelateerd hier is het idee voorgesteld in [13], om ieder van de ingangssignalen te moduleren met onafhankelijke willekeurige ruis

$$x'_i(n) = [1 + \epsilon_i(n)]x_i(n) \quad (2.51)$$

waarbij ϵ_i twee onafhankelijke laagdoorlaat ruisprocessen zijn

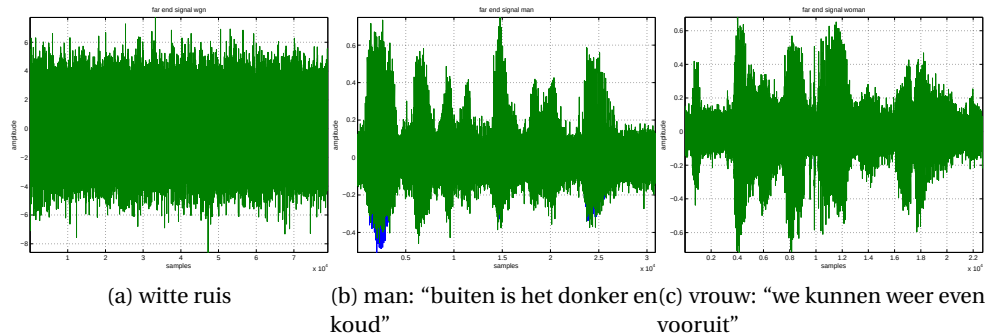
$$\epsilon_i(n) = \alpha \epsilon_i(n-1) + (1 - \alpha)v_i(n) \quad (2.52)$$

en de v_i zijn opnieuw twee willekeurige witte ruisbronnen. Deze methode is opnieuw onderzocht in [3] en niet goed bevonden, aangezien het gemoduleerd ruissignaal in de praktijk altijd hoorbaar is.

Simulaties

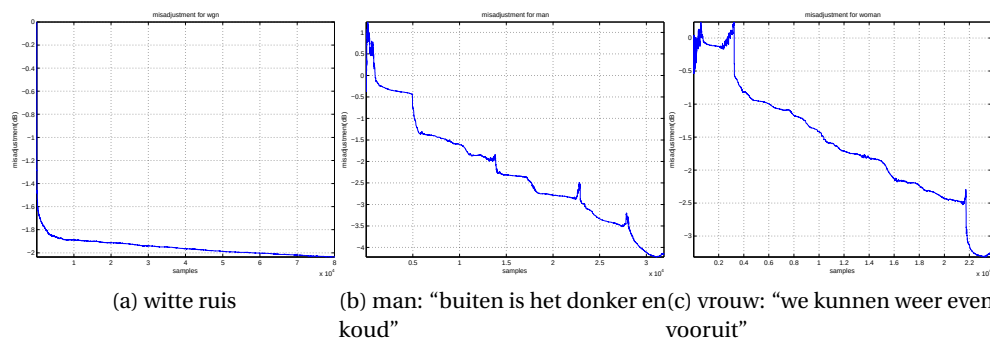
Om deze methode te testen wordt er verder gegaan vanuit de opstelling¹ eerder voorgesteld in deel 2.3.6. Om de signalen te decorreleren wordt er aan het tweede kanaal ruis toegevoegd zodanig dat een bepaald niveau van signaal-tot-ruis vermogenverhouding (SNR) wordt behaald. In figuur 2.10 zijn de verre eind signalen te zien na toevoeging van ruis zodanig dat een SNR = 25dB behaald wordt voor het kanaal waar de ruis is aan toegevoegd.

¹ d.i. de signalen, parameters, adaptief algoritme, etc.



Figuur 2.10: Verre eind signalen na toevoeging van ruis voor decorrelatie, SNR=25dB

Het effect op de uitlijning wordt zichtbaar in figuur 2.11 en in figuur 2.12 per kanaal. Bemerk dat de uitlijning in dit geval beter is dan voor het geval waar er geen ruis wordt toegevoegd, zoals aangetoond in figuren 2.7 en 2.8. Er wordt echter geen grote daling waargenomen en uit simulaties waar de SNR nog lager ligt, d.i. meer ruis wordt toegevoegd, blijkt zelfs dat de uitlijning terug begint te stijgen.

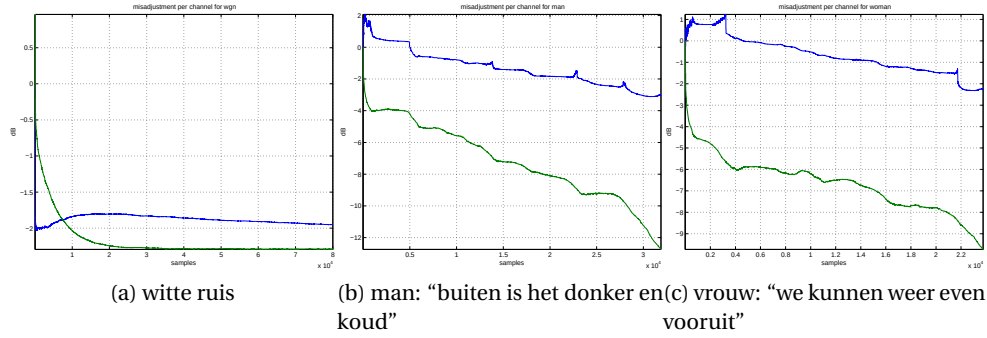


Figuur 2.11: Uitlijning na toevoeging van ruis voor decorrelatie

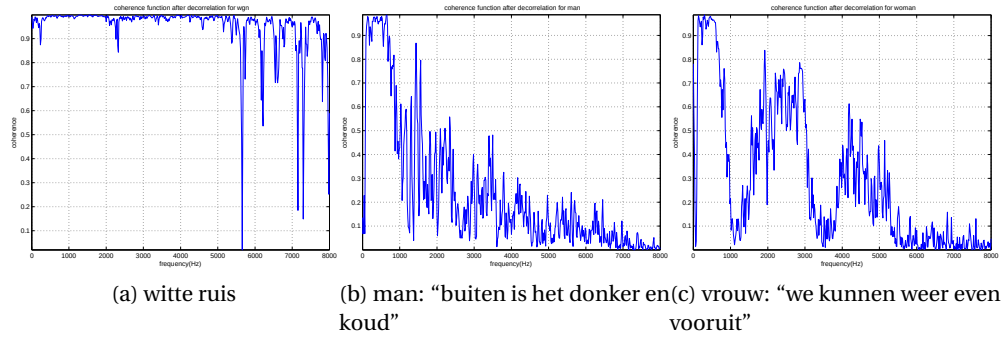
Dat er een grote decorrelatie gebeurt is te zien aan de coherentiefuncties: de originelen komen overeen met deze in figuur 2.9, deze na decorrelatie zijn te zien in figuur 2.13. Bij vergelijking van deze figuren blijkt dat er een duidelijke decorrelatie is voor de twee spraaksignalen, voor het ruissignaal is dit niet geval. Dit is te verklaren doordat de ruis die toegevoegd is witte ruis is en dus voor het eerste kanaal gecorreleerd met het signaal zelf.

Deze methode is in de praktijk echter niet toe te passen aangezien het een ontoelaatbare distortie van het signaal vereist. Testen zijn gedaan waarbij de hoeveelheid ruis heel laag is gehouden, wat quasi geen decorrelerend effect heeft noch een positieve invloed op de uitlijning.

Een mogelijke verbetering hierin is het toevoegen van de ruis zodanig uit te voeren dat ze psychoakoestisch gemaskeerd kan worden [14]. Zo heeft het gebruik van psychoakoestische maskers zich al bewezen in de perceptuele audio codering [15]. Het



Figuur 2.12: Uitlijning na toevoeging van ruis voor decorrelatie (2 kanalen)



Figuur 2.13: Coherëntiefunctie na toevoeging van ruis voor decorrelatie

toepassen van psychoakoestische maskering is hier echter niet zo evident, enerzijds omdat het berekenen van de spectrale maskering computationeel vrij intensief is. Anderzijds is de toepasbaarheid specifiek binnen spraakverwerking beperkt, aangezien spraak in bandbreedte beperkt is in vergelijking met algemene audio/muziek.

2.4.2 Decorrelatiefilters

Bij deze methode komt het er op neer een vervorming toe te passen in het frequentiedomein op beide signalen x_i . Veronderstel de filters $F_i(f)$

$$X'_1(f) = X_1(f) - F_2(f)X_2(f) \quad (2.53)$$

$$X'_2(f) = X_2(f) - F_1(f)X_1(f) \quad (2.54)$$

zodanig dat $X'_1(f)$, $X'_2(f)$ niet gecorreleerd zijn. In [2] wordt reeds aangegeven dat deze methode minder aantrekkelijk is.

2.4.3 Schuiven van de spectra tussen de kanalen

Deze methode bestaat er in het frequentiespectrum van een kanaal relatief ten opzichte van het andere te verschuiven om een decorrelatie te bekomen [2]. Dit wordt opnieuw

niet als doeltreffend ondervonden. Enkel in het geval het spectrum uit aparte frequenties bestaat zou dit een echte invloed kunnen hebben, wat voor spraak niet het geval is. Daarbij gaat de spatiale informatie afkomstig uit het feit dat er twee verschillende signalen zijn verloren door deze operatie, wat uiteraard te vermijden is.

2.4.4 Vershoven kamfilters

Een andere methode die een decorrelatie probeert te bekomen met behulp van informatie in het frequentiedomein is deze van verschoven kamfilters [2]. Het idee houdt het volgende in: stel dat alle energie in een bepaalde frequentieband voor één van de twee signalen verwijderd zou worden, zodanig dat enkel ongecorreleerde achtergrondruis overblijft. Voor die frequentieband wordt de coherentie gereduceerd tot nul. Men kan redeneren dat men op die manier verschillende frequentiecomponenten kan verdelen over de twee kanalen. In [2] wordt geargumenteed dat op basis van perceptuele modellen de spatiale informatie in de twee kanalen niet al te zeer vervormd wordt voor kamfilters werkend boven de 1000 Hz; voor frequenties onder de 1000 Hz is dit niet het geval. Spijtig genoeg is het net in dat frequentiegebied dat de echo's de meeste invloed hebben. [2]

Eigen simulaties bevestigen dit idee: het is inderdaad mogelijk om de twee kanalen te decorreleren, alleen is dit altijd hoorbaar in het uiteindelijke resultaat. Hier is verder dan ook geen onderzoek meer naar verricht.

2.4.5 Niet-lineaire operaties

In [3] gaat men uit van het idee dat, om de hoorbare degradatie binnen de perken te houden, het aan te raden is om iets van het oorspronkelijke signaal toe te voegen. Hetgene wat men toevoegt mag dan niet langer linear afhangen van het origineel signaal. Algemeen wordt dit als volgt voorgesteld

$$x'_i(n) = x_i(n) + \alpha f[x_i(n)], i = 1, 2 \quad (2.55)$$

waarbij f algemeen staat voor een niet-lineaire functie zodanig dat er geen lineaire relatie bestaat tussen de $x_i(n)$. Betreffende het soort niet-lineaire operatie bestaan er heel veel mogelijkheden, waar een aantal van worden besproken in [3]. In [8] wordt dit verder onderzocht en wordt voor een aantal niet-lineaire functies het effect op de coherentiefunctie analytisch bepaald.

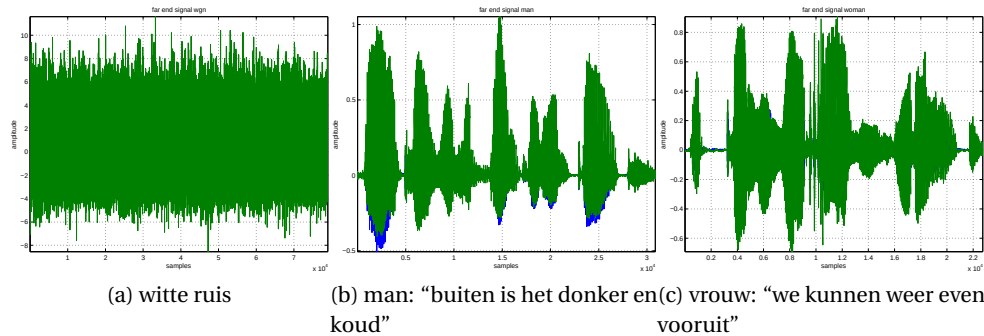
Simulaties

Een aantal niet-lineaire functies en hun effect op de coherentiefunctie zijn onderzocht. Bij wijze van illustratie is hier het resultaat van slechts één van de niet-lineaire functies gegeven, nl. de *half-wave rectifier*. Deze functie ziet er als volgt uit

$$f(x) = \begin{cases} x, & \text{als } x \geq 0 \\ 0, & \text{elders} \end{cases} \quad (2.56)$$

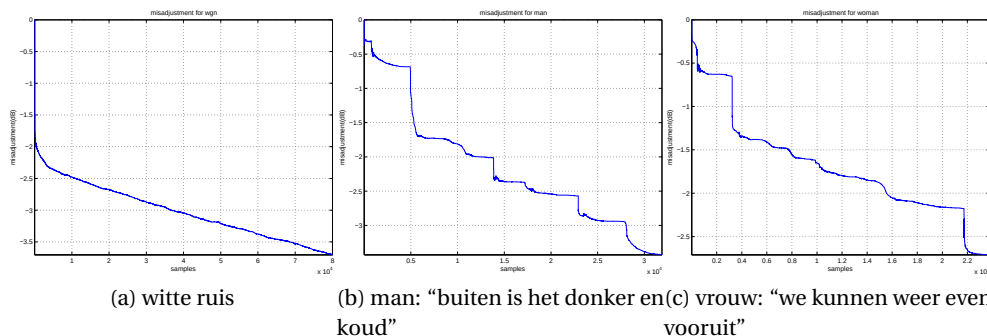
Ook voor deze simulaties wordt weer uitgegaan van de vereenvoudigde opstelling uit deel 2.3.6. Als parameter α zijn verschillende waarden geprobeerd, uiteindelijk is er

gekozen voor de waarde $\alpha = 0.5$. Allereerst wordt het effect van de decorrelerende operatie bekeken in figuur 2.14, d.i. het verre eind signaal. Uit de figuur blijkt dat de vervorming op het (spraak)signaal beperkt blijft. Een informele luistertest bevestigt dit.



Figuur 2.14: Verre eind signalen na toepassing niet-lineaire decorrelatie

Betreffende de uitlijning, blijkt uit figuur 2.15 dat deze sneller daalt dan voor het geval waarbij er geen decorrelatie wordt toegepast, zichtbaar in figuur 2.7. Figuur 2.15 toont hetzelfde, maar dan per kanaal.

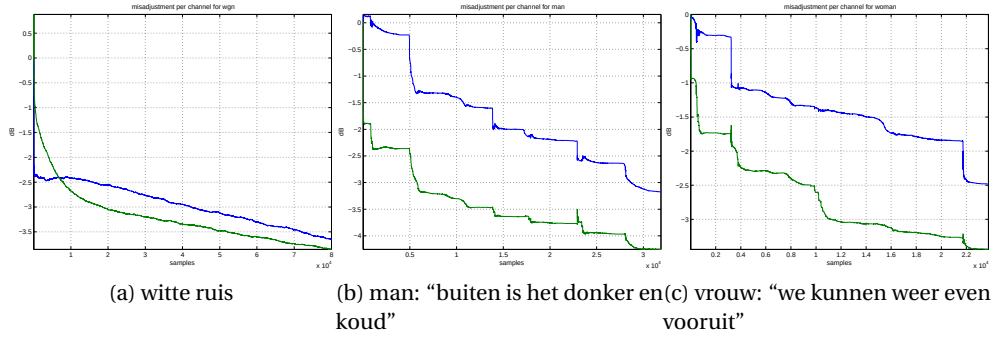


Figuur 2.15: Uitlijning na toepassing niet-lineaire decorrelatie

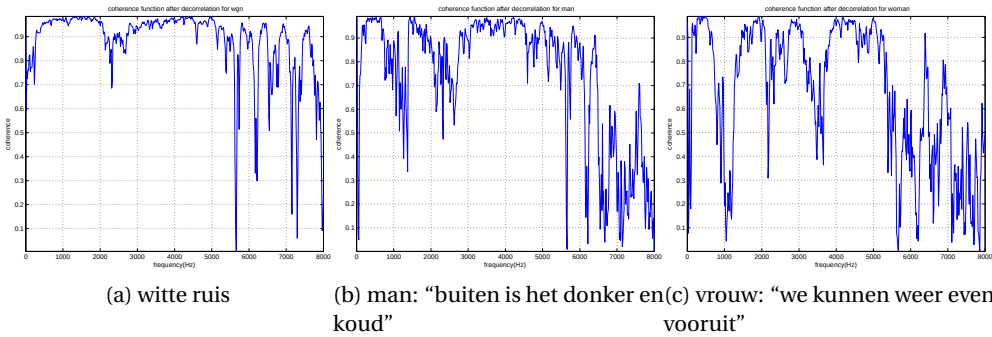
De verklaring hiervoor ligt in het feit dat de niet-lineaire functie inderdaad een goede decorrelatie doorvoert tussen beide kanalen, zoals blijkt uit de coherentiefunctie, te zien in figuur 2.17. Deze methode kan een goede decorrelatie uitvoeren op de twee signalen, zonder dat in de praktijk hoorbaar is, wat dit een interessant alternatief maakt. Uiteraard wordt het signaal wel vervormd, zodat dit uiteindelijk nog geen optimale oplossing is.

2.4.6 Orthogonale transformaties

Het idee van signaaldecorrelatie komt niet alleen voor in een context waar een adaptief algoritme op meerdere kanalen wordt toegepast, ook voor het mono geval zijn zulke transformaties interessant om een snellere convergentie te bekomen voor een stochastische gradiënt adaptief filter algoritme, zoals e.g. het LMS-algoritme. Er kan immers



Figuur 2.16: Uitlijning per kanaal na toepassing niet-lineaire decorrelatie



Figuur 2.17: Coherëntiefunctie na toepassing niet-lineaire decorrelatie (2 kanalen)

aangetoond worden dat het convergentiegedrag van het LMS-algoritme gerelateerd is aan de eigenstructuur van de autocorrelatiematrix van het ingangssignaal. In deze context definieert men het begrip *eigenwaardenverdeling* χ als de verhouding tussen de hoogste en laagste eigenwaarde

$$\chi = \lambda_{\max} / \lambda_{\min} \quad (2.57)$$

In termen van de eigenwaardenverdeling wordt in [4] aangetoond dat voor een lage waarde χ van de autocorrelatiematrix van het ingangssignaal een betere convergentie bekomen wordt voor het LMS-algoritme. In het ideale geval zal deze autocorrelatiematrix dan ook de eenheidsmatrix benaderen.

Om deze orthogonalisatie door te voeren kan men voor het LMS-algoritme rekening houden met de autocorrelatiematrix in het aanpassen van het adaptief filter [16],[17]

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \alpha \mathbf{R}_{xx}^{-1} \mathbf{x}(n) e(n) \quad (2.58)$$

Vergelijking 2.58 staat ook bekend als het LMS/Newton adaptief algoritme, in analogie met het Newton algoritme dat gebruikt wordt om de extrema van een tweedegraads kostfunctie te zoeken [18]. Hierbij staat $\mathbf{R}_{xx}$ voor de autocorrelatiematrix van het ingangssignaal; in het geval dat het ingangssignaal voortkomt uit een stationair proces kan deze constant

worden gehouden, in een praktische toepassing zal deze schatting mee moeten variëren over de tijd. Concreet betekent dit dan ook dat het rechtstreeks toepassen van deze methode niet evident is, aangezien een veranderlijke $\mathbf{R}_{xx}$ impliceert dat voor iedere iteratie een schatting moet worden gedaan van de stochastische parameters van het proces, d.i. de autocorrelatiematrix (en de inversie daarvan).

Om de matrixinversie te vereenvoudigen zou men kunnen gebruik maken van een signaaltransformatie waarbij de autocorrelatiematrix $\mathbf{R}_{xx}$ vervangen kan worden door een diagonaalmatrix. Op die manier is enkel een deling door een scalar vereist per element van het adaptief filter

$$\hat{h}_i(n+1) = \hat{h}_i(n) + \frac{\alpha}{\lambda_i} v_i(n) e(n) \quad (2.59)$$

waarbij de $\hat{h}_i$ staan voor de individuele elementen van het adaptief filter. De $v_i(n)$ en λ_i staan voor de eigenvectoren en hun overeenkomstige eigenwaarden uit een eigenwaardenontbinding van de autocorrelatiematrix $\mathbf{R}_{xx}$. De voorstelling van een signaal in termen van de eigenwaardenontbinding van zijn autocorrelatiematrix staat ook bekend als de Karhunen-Loève transformatie (KLT) [19, 20] of als principale componentenanalyse (PCA) [21], al naargelang de discipline. Aangezien het een voorstelling van het signaal is ten opzichte van een stelsel dat gebruik maakt van een eigenwaardenontbinding, is de basis een orthogonale basis [22]. Daarbij is de basis ook van het signaal zelf afgeleid wat in de praktijk maakt dat het signaal in deze basis vaak kan voorgesteld worden met een laag aantal indices [4].

Het probleem verschuift zich nu naar het berekenen van de KLT zelf. Opnieuw moet voor ieder venster waarin stationariteit geldt een nieuwe autocorrelatiematrix $\mathbf{R}_{xx}$ geschat worden, waar vervolgens een eigenwaardenontbinding op moet worden uitgevoerd om een factorisatie in termen van de λ_i en $v_i(n)$ te bekomen. De complexiteit waarmee eigenwaardenontbindingen gebeuren is te hoog om praktisch bruikbaar te zijn.

Om tegemoet te komen aan het computationeel probleem dat de KLT stelt, wordt in [4] voorgesteld om van een signaalafhankelijke transformatie over te gaan op een signaalonafhankelijke orthogonale transformatie. In [23, 24, appendix E] stelt men voor om gebruik te maken van een discrete cosinus transformatie (DCT) of een discrete Fourier transformatie (DFT) als orthogonalisatiestap. Om het computationeel probleem verder te verzachten wordt een recursief algoritme voorgesteld voor het berekenen van de DCT/DFT-coëfficiënten.

Voor zover gekend bij de auteurs, vermeldt de literatuur nergens een algoritme waarbij een uitbreiding wordt gedaan naar meerdere kanalen. Ook houdt het algoritme niet expliciet rekening met de stochastische eigenschappen die specifiek zijn voor een spraaksignaal, waardoor dit algoritme ook niet interessant blijkt. Zoals zal blijken uit deel 2.5, biedt een oplossing gebaseerd op lineaire predictie een interessanter alternatief. Wel belangrijk hier is de uiteenzetting over de invloed van de eigenstructuur van de autocorrelatiematrix $\mathbf{R}_{xx}$ op het convergentiegedrag van een stochastische gradiënt methode, aangezien dit concept later opnieuw zal terugkomen.

2.5 Lineaire predictie

MOTIVATIE Het idee achter lineaire predictie is het voorstellen van een monster van een signaal op basis van een lineaire gewogen combinatie van andere monsters van dat signaal. Een volledige beschrijving van de theorie achter lineaire predictie en o.a. de fysische achtergrond voor het gebruik er van in spraakcodering worden hier niet verder behandeld. Voor een meer uitgebreide beschrijving wordt er verwezen naar hoofdstuk 3. Om het gebruik van lineaire predictie te motiveren in de context van stereo akoestische echo-onderdrukking worden hier kort wel een aantal eigenschappen aangehaald. Een veel gebruikte eigenschap is dat, voor een stationair stochastisch proces, de coëfficiënten van het autoregressief model overeen komen met de coëfficiënten bekomen door lineaire predictie [4]. Voor de extractie van deze coëfficiënten definieert men analysefilters. Een structuur van analysefilters die voor signaaldecorrelatie en bijgevolg voor het probleem van stereo akoestische echo-onderdrukking interessant is, is het analysefilter in de zgn. *roostervorm*.

Een analyse met dit soort filter geeft naast de LPC's ook de voorwaartse en achterwaartse predictiefouten, waarvan geweten is dat deze van verschillende orde orthogonaal zijn ten opzichte van elkaar. Deze eigenschap van orthogonaliteit kan e.g. gebruikt worden in een *gezamenlijke proces schatting* [4, deel 6.9], waarbij de LPC's direct gebruikt worden door een Wiener filter.

Het verband dat tussen de LPC's bestaat en de coëfficiënten van het overeenkomstige AR model maken ook dat lineaire predictie vaak gebruikt wordt in spraakverwerking. Spraaksignalen zijn nl. te benaderen als quasi-stationaire signalen, d.i. binnen een kort tijdsvenster van om en bij de 20 ms gedraagt een spraaksignaal zich stationair. Bijgevolg is het mogelijk het spraaksignaal voor ieder van deze segmenten te parameteriseren met behulp van een beperkt aantal LPC's, d.i. een orde van 5 tot 10 volstaat in de praktijk [25]. Deze ordeverlaging, zal later blijken, heeft ook zijn voordelen in de praktische implementatie van adaptieve algoritmes. Ook kan er aangetoond worden dat de reflectiecoëfficiënten van de roostervorm van het analysefilter een verband hebben met de fysische achtergrond van de menselijke spraakproductie.

Wat zeker ook in het voordeel speelt van het gebruik van deze methode is dat er geen vervorming van het originele signaal nodig is, in tegenstelling tot veel van de andere methodes reeds besproken.

TOEPASSING IN STEREO AKOESTISCHE ECHO-ONDERDRUKKING Specifiek voor stereo akoestische echo-onderdrukking is vooral de eigenschap van orthogonaliteit tussen de predictiefouten interessant. Omwille van deze orthogonaliteit kunnen de predictiefouten gebruikt worden als een manier om de ingang van het adaptief filter te decorreleren. Specifiek worden in dit deel twee verschillende technieken voorgesteld: een eerste, die rechtstreeks werkt op de predictiefout van een lineaire predictie en een tweede die de predictiefouten integreert in een schema van gezamenlijke proces schatting.

De eerste techniek wordt besproken in deel 2.5.1 en komt in essentie neer op het toepassen van een lineair predictiefilter. Het resultaat van deze operatie is een predictiefoutsignaal dat gebruikt wordt als ingang voor een adaptief filter. Dit concept wordt besproken voor het monofone geval in [26], om de correlatie tussen de verschillende

monsters van hetzelfde kanaal te verkleinen². Een uitbreiding naar het stereofone geval wordt hier voorgesteld op basis van stereofone lineaire predictie, zoals uitgewerkt in hoofdstuk 3.

De andere mogelijkheid wordt besproken in deel 2.5.2 en maakt gebruik van het principe van gezamenlijke proces schatting, besproken voor het monofone geval in [4]. Het schatten van LPC's gebeurt hierbij recursief over de verschillende ordes en verschillende tijdstippen met behulp van een zogenaamd roosterpredictor. In [27] wordt deze roosterpredictor besproken voor meerdere kanalen. Dit is het principe dat gebruikt wordt in het adaptief algoritme voorgesteld in [28]. Het nadeel is dat voor grote lengtes van het adaptief filter het gebruik van deze methode niet meer opportuun is.

Een andere methode die gebruik maakt van lineair predictieve codering is deze zoals voorgesteld in [29]. Hier wordt de orthogonaliteit van de achterwaarde predictiefouten gecombineerd in een LMS/Newton adaptief algoritme, d.i. een stochastische gradient adaptief algoritme. Bemerkt dat dit algoritme slechts voor éénkanaalstoepassingen uitgewerkt is.

Het algoritme hier beschouwd [11] is een tweekanaalsversie van het LMS/Newton, LPC-gebaseerde algoritme in [29]. De principes voor de meerkanaalsuitbreiding van de roosterpredictor in [28] komen hier dan ook in terug. Voor de bespreking van algoritme wordt in deel 2.5.2 de roosterpredictor beschouwd. Daarna wordt er in deel 2.5.2 ingegaan op de gezamenlijke proces schatting, uitgebreid naar meerdere kanalen. Tot slot wordt in deel 2.5.2 het LMS/Newton algoritme gebaseerd op AR modellering verder besproken.

2.5.1 Lineair predictief decorrelatiefilter

PRINCIPE Net zoals de alternatieven besproken in deel 2.4, betekent deze oplossing opnieuw het toepassen van een extra decorrelerende stap alvorens een adaptief filter algoritme uit te voeren. In analogie met de beschrijving gebruikt in [26] wordt hier ook de term *decorrelatiefilter* aangewend om deze decorrelerende stap aan te geven. Deze benaming is niet analoog aan de decorrelatiefilters besproken onder 2.4.2 waar men filters voorstelt die rechtstreeks in het frequentiedomein werken om een decorrelatie van de verschillende kanalen te bekomen. Een andere reden voor het gebruik van deze term is het principe van de analyse- en synthesefilters uit de theorie van lineaire predictie [4]. Wat hier van belang is, is dat een analysefilter voortkomend uit lineaire predictie kan gezien worden als een soort *wittend filter*, d.i. het transformeert de ingangssignalen tot andere signalen waarvan de correlatie tussen de monsters van een zelfde kanaal en tussen monsters van de verschillende kanalen gereduceerd is.

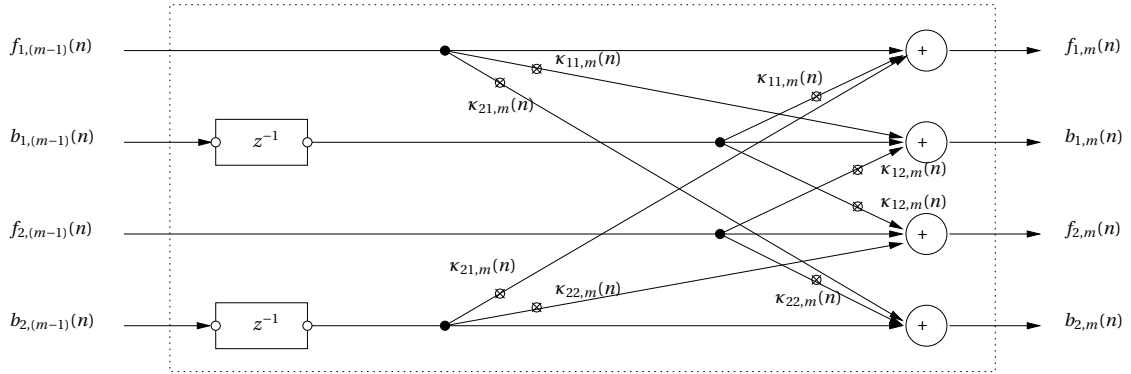
Een theoretische beschouwing evenals een aantal simulaties in termen van de coherentiefunctie worden meer in detail besproken in deel 3; verdere details worden hier dan ook achterwege gelaten.

² en uiteindelijk de eigenwaardeverdeling χ van de autocorrelatiematrix R_{xx} te verlagen

2.5.2 Gezamenlijke proces schatting – Newton/LMS

Tweekanaals roosterpredictor

BEREKENING De uitbreiding van de éénkanaals roosterpredictor [4, deel 6.8] naar meerdere kanalen is voor het eerst afgeleid in [27] in kleinste kwadraten zin. Hier wordt de overeenkomstige tweekanaals roosterpredictor besproken, e.g. gebruikt in [28] in context van een adaptief algoritme. De structuur van een dergelijke cel is te zien in figuur 2.18. Uit de figuur kunnen volgende vergelijkingen worden afgeleid voor respectievelijk de



Figuur 2.18: Tweekanaals roostercel

achterwaartse en voorwaartse predictiefouten [11]

$$\mathbf{b}_m(n) = \mathbf{b}_{m-1}(n-1) + \boldsymbol{\kappa}_m \mathbf{f}_{m-1}(n) \quad (2.60)$$

$$\mathbf{f}_m(n) = \mathbf{f}_{m-1}(n) + \boldsymbol{\kappa}_m^T \mathbf{b}_{m-1}(n-1) \quad (2.61)$$

waarbij $\mathbf{b}_m(n) = [b_{1,m}(n) \ b_{2,m}(n)]^T$, $\mathbf{f}_m(n) = [f_{1,m}(n) \ f_{2,m}(n)]^T$ en de reflectiecoëfficiëntenmatrix $\boldsymbol{\kappa}_m(n)$

$$\boldsymbol{\kappa}_m(n) = \begin{bmatrix} \kappa_{11,m}(n) & \kappa_{12,m}(n) \\ \kappa_{21,m}(n) & \kappa_{22,m}(n) \end{bmatrix} \quad (2.62)$$

De roosterpredictor wordt geïnitieëerd door telkens de nieuwe waarden in te voegen in de predictiefouten

$$\mathbf{b}_0(n) = \mathbf{f}_0(n) = [x_1(n) \ x_2(n)]^T \quad (2.63)$$

Het aanpassen van de reflectiecoëfficiënten kan gemakkelijk recursief gebeuren met behulp van een gradiënt adaptief algoritme [28, 11]

$$\kappa_{ij,m}(n+1) = \kappa_{ij,m}(n) - \frac{2\mu_r}{\bar{P}_{ij,m-1}(n) + \epsilon} (f_{j,m}(n)b_{i,m-1}(n-1) + f_{j,m-1}(n)b_{i,m}(n)) \quad (2.64)$$

voor $(i, j) = \{(1, 1), (1, 2), (2, 1), (2, 2)\}$

met μ_r een constante om de stapgrootte te regelen, ϵ een constante om te compenseren voor eventuele numerieke instabiliteit en de $\bar{P}_{ij,m}$ een schatting van het vermogen in het

signaal. De vermogens zelf kunnen ook recursief berekend worden

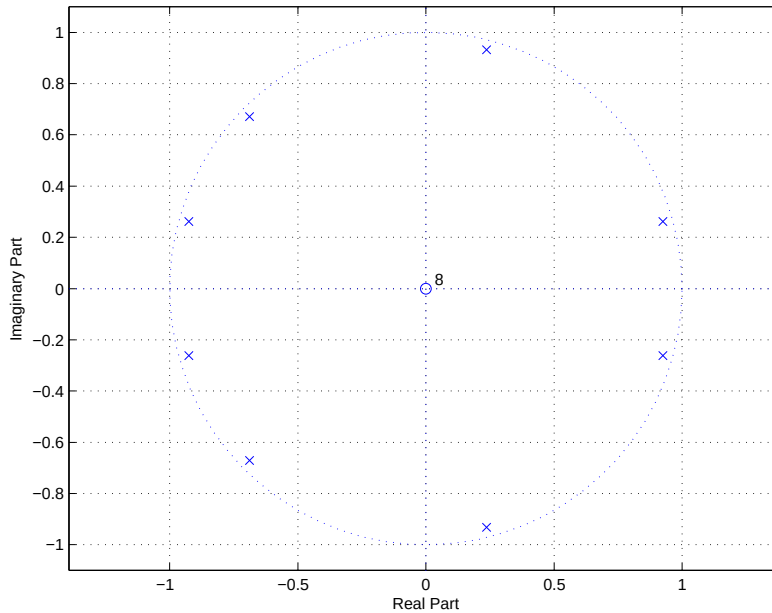
$$\tilde{P}_{ij,m}(n) = \beta \tilde{P}_{ij,m}(n-1) + 0.5(1-\beta) \left(b_{i,m-1}^2(n-1) + f_{j,m-1}^2(n) \right) \quad (2.65)$$

voor $(i, j) = \{(1, 1), (1, 2), (2, 1), (2, 2)\}$

met $0 \leq \beta \leq 1$.

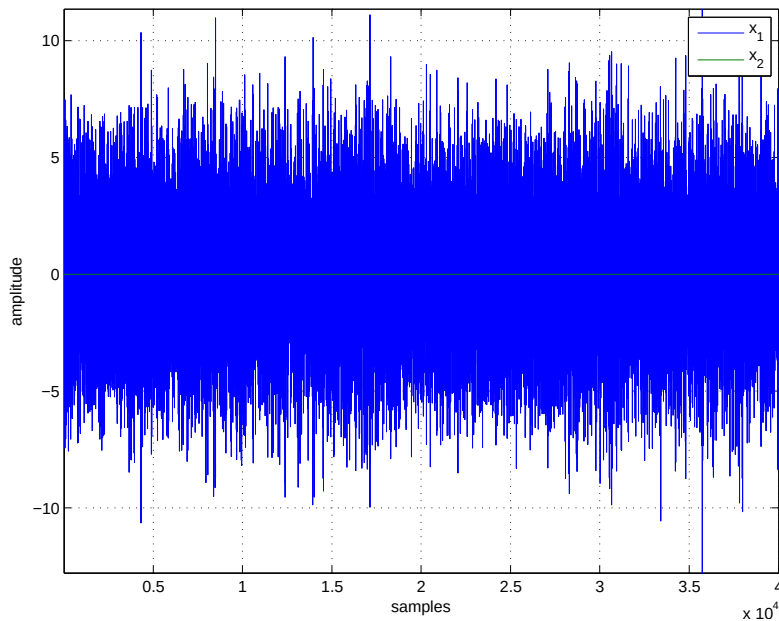
IMPLEMENTATIE Eerder is het verband tussen lineaire predictie en autoregressieve modellering al aangehaald; om na te gaan in welke mate de roosterpredictor een lineaire predictie doet, is het interessant om te kijken naar de analyse die het uitvoert op een gesynthetiseerd signaal met gekende AR parameters en deze achteraf te vergelijken.

Constante AR parameters Het gebruikte AR signaal wordt gegenereerd door standaard normaal verdeelde, witte ruis door een AR synthesefilter te sturen, met gekende AR parameters [4]. Praktisch is er voor volgende parameters gekozen. Het ingangssignaal is standaard normaal verdeelde witte ruis van 5s lang; om een tijdsreferentie te hebben is er een bemonsteringsfrequentie f_s van 8kHz gekozen. Het synthesefilter is van orde $M = 8$, de polen zijn zodanig gekozen dat ze allen binnen de éénheidscirkel vallen, allen op gelijke afstand van de oorsprong. Bemerk dat deze polen overeen komen met de LPC's van het signaal. In figuur 2.19 worden deze parameters getoond. Op basis van deze LPC's kunnen de reflectiecoëfficiënten worden afgeleid met behulp van het Levinson-Durbin algoritme [30]. Om twee kanalen te bekomen wordt als eerste kanaal het ruissignaal door



Figuur 2.19: Pool/nulpunt diagramma met AR parameters voor het gegenereerde signaal

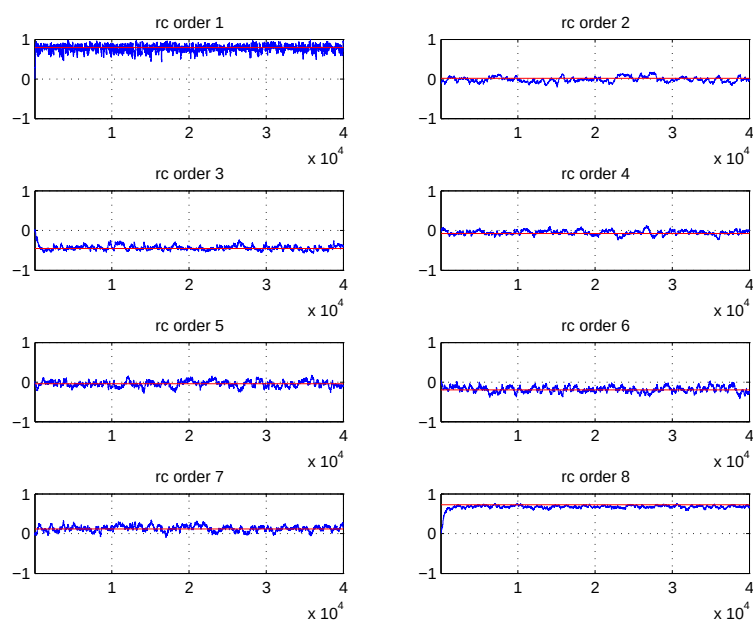
het synthesefilter gestuurd, als tweede kanaal wordt het ruissignaal gekozen, vermenigvuldigd met een constante zodanig dat het tweede kanaal een zeer lage amplitude heeft. Het signaal uiteindelijk zo bekomen is te zien in figuur 2.20.



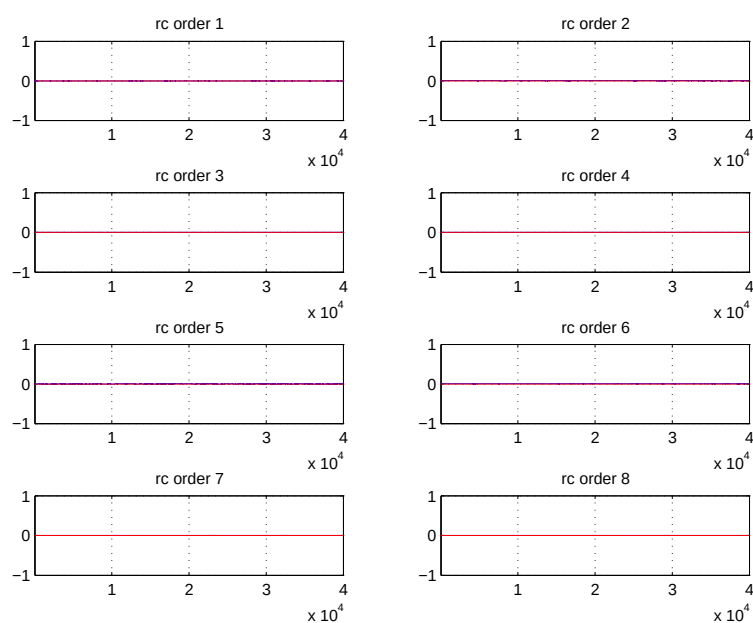
Figuur 2.20: Gesynthetiseerd AR signaal met gekende parameters

De functie van de roosterpredictor is nu om de reflectiecoëfficiënten κ uit het signaal te halen. Voor het gesynthetiseerde signaal zal enkel voor het eerste kanaal, d.i. κ_{11} , een bijdrage zichtbaar mogen zijn, meer bepaald de reflectiecoëfficiënten die eerder gegenereerd waren. Het tweede kanaal is nl. witte, normaal verdeelde ruis (κ_{22}) en bijgevolg is er ook geen correlatie tussen de twee kanalen (κ_{12} en κ_{21}). Figuren 2.21, 2.22 en 2.23 tonen respectievelijke de κ_{11} , κ_{12} en κ_{22} . Hieruit blijkt dat de roosterpredictor inderdaad in staat is, mits enige gradiënt ruis, om de opgegeven AR parameters terug uit het signaal te halen. Figuur 2.24 toont ter illustratie een uitmiddeling over de tijd voor de κ_{11} .

Variabele AR parameters: gesynthetiseerd Verder is ook nog getest hoe de roosterpredictor werkt wanneer de stochastische eigenschappen van het signaal wijzigen over de tijd, zoals het geval is voor e.g. een spraaksignaal. Hiertoe zijn vier sequenties gegenereerd, elk met verschillende stochastische eigenschappen, d.i. verschillende reflectiecoëfficiënten κ . Opnieuw wordt hier een enkel kanaal beschouwd. Het gegenereerde signaal is te zien in figuur 2.25. Het gedrag van de roosterpredictor voor dit signaal is te zien in figuur 2.26. De verschillende lijnen staan voor de gegenereerde, gekende AR parameters. Hieruit blijkt dat de roosterpredictor ook om kan gaan met variaties op het signaal. Voor het verkrijgen van een goed resultaat is wel gebleken dat een grotere stapgrootte μ_r nodig is in de vergelijking (2.64), aangezien er relatief gezien een grotere variatie is in de stochastische eigenschappen van het signaal.

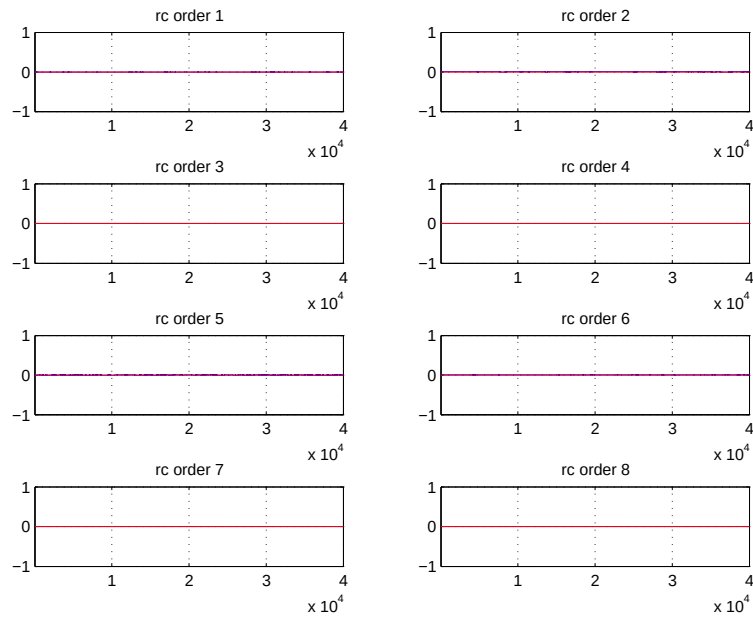


Figuur 2.21: κ_{11} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters

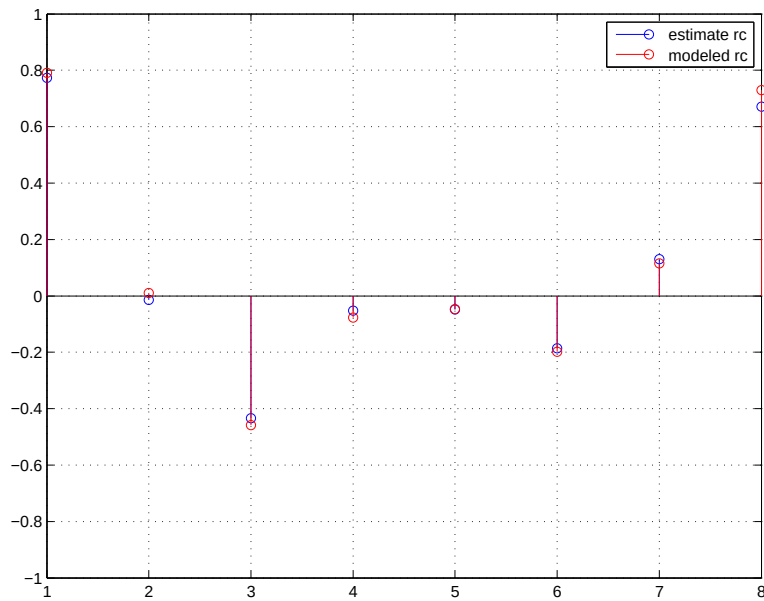


Figuur 2.22: κ_{12} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters

Variabele AR parameters: spraaksignaal Tot slot is er ook gekeken of de roosterpredictor om kan gaan met echte spraaksignalen, aangezien dit uiteindelijk de bedoeling

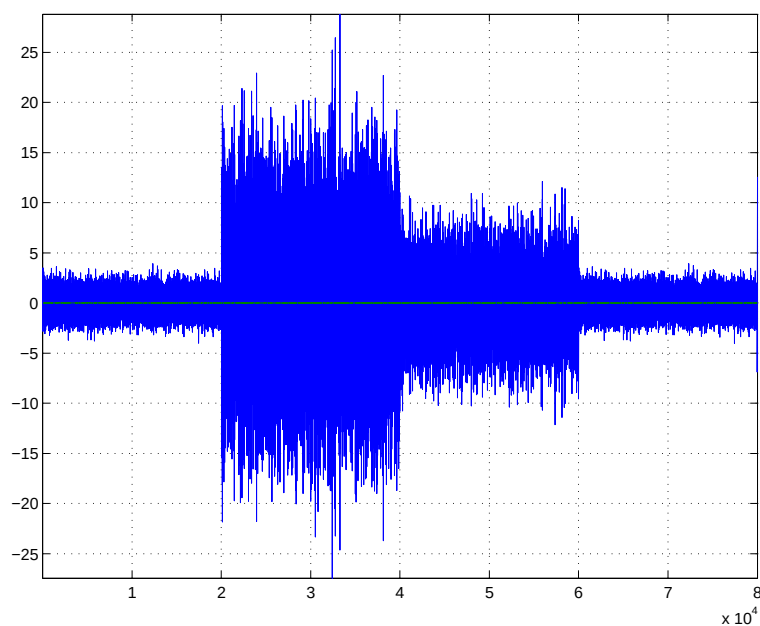


Figuur 2.23: κ_{22} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde AR parameters

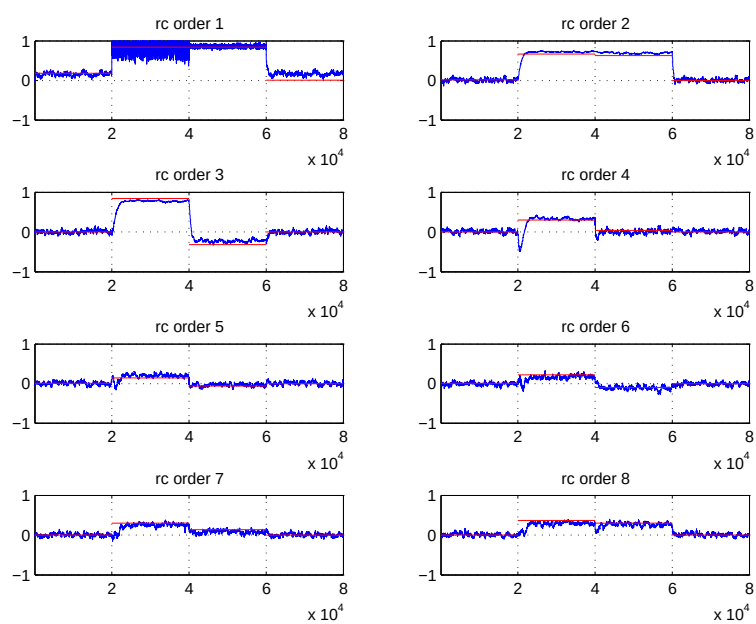


Figuur 2.24: κ_{11} zoals berekend met de roosterpredictor uitgemiddeld over de tijd, vergeleken met de gegenereerde AR parameters

is. Als signalen is er opnieuw gekozen voor de spraaksignalen gebruikt bij de uiteenzetting van het fundamentele probleem, nl. figuren 2.3b en 2.3c. Bemerkt dat hier ook enkel



Figuur 2.25: Gesynthetiseerd AR signaal met gekende parameters voor vier verschillende segmenten

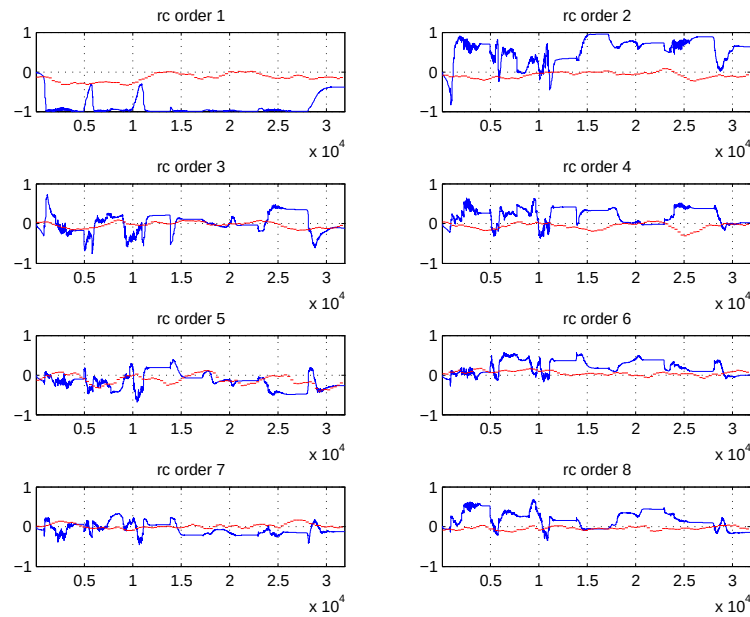


Figuur 2.26: κ_{11} zoals berekend met de roosterpredictor over de tijd, vergeleken met de gegenereerde tijdsafhankelijke AR parameters

zal worden gekeken naar een enkel kanaal, d.i. κ_{11} . Het tweede kanaal is opnieuw on-

gecorrleerde witte normaal verdeelde ruis, met een lage amplitude. In eerste instantie is het immers belangrijk om er zeker van te zijn dat het recursief schatten van de LPC-parameters een gelijkaardig resultaat geeft als de klassieke, venstergebaseerde methode. Aangezien de AR parameters niet a priori gekend zijn, wordt dan ook deze laatste venstergebaseerde methode gebruikt om een vergelijking mee te maken. Concreet wordt een predictieorde van $M = 8$ gebruikt, het venster waarin stationariteit wordt verondersteld is 20ms.

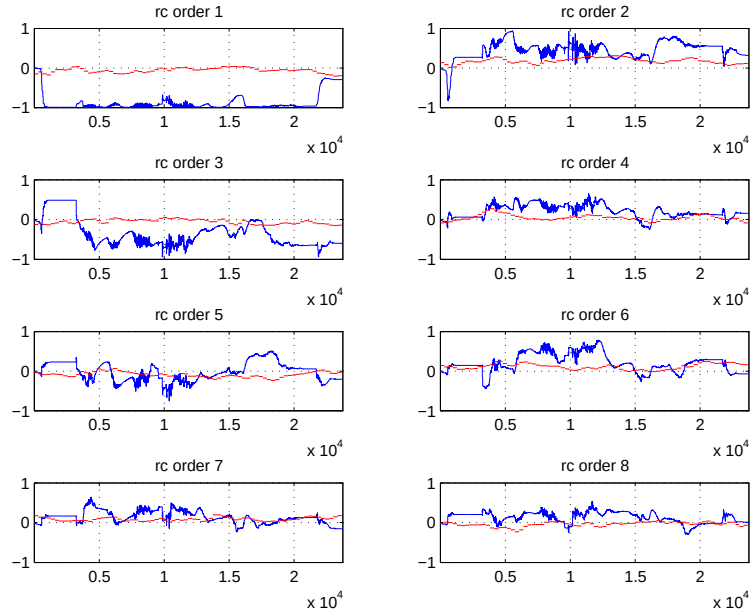
De schatting van reflectiecoëfficiënten van de verschillende ordes over de tijd is te zien in figuren 2.27 en 2.28. De blauwe grafiek geeft de (recursief berekende) schatting van de AR parameters aan, de rode grafiek deze met de venstergebaseerde methode. Bij het



Figuur 2.27: κ_{11} zoals berekend met de roosterpredictor over de tijd voor spraaksignaal man, vergeleken venstergebaseerde LPC-methode

gesynthetiseerde AR signaal met een variabel verloop van de stochastische eigenschappen is opgemerkt dat een grotere stapgrootte vereist is zodat de rooster predictor het veranderen van de AR parameters kan volgen. Een gelijkaardige bedenking kan gemaakt worden voor een spraaksignaal, met dat verschil dat de vensters van (veronderstelde) stationariteit veel korter zijn en elkaar veel sneller opvolgen. Voor het uitvoeren van de simulaties is de stapgrootte μ_r (en bijhorend ook de parameter ϵ) dan ook van een veel grotere grootteorde dan voor de andere, geïdealiseerde gevallen. Concreet is voor de grafieken in voorgaande figuren een stapgrootte $\mu_r = 0.70$ gebruikt. Desondanks deze grote stapgrootte blijkt dat de roosterpredictor vaak veel afwijkt van de venstergebaseerde schatting van de LPC-waarden.

AUTOCORRELATIEMATRIX ACHTERWAARTSE PREDICTIEFOUTEN In [28] wordt aangegeven dat in het geval dat de berekende reflectiecoëfficiënten convergeren naar deze van het



Figuur 2.28: κ_{11} zoals berekend met de roosterpredictor over de tijd voor spraaksignaal vrouw, vergeleken venstergebaseerde LPC-methode

originele signaal, in de aanname dat het signaal stationair is, er een volledige orthogonalisatie van de achterwaartse predictiefouten wordt bekomen, d.i. de achterwaartse predictiefouten staan orthogonaal ten opzichte van elkaar over de verschillende ordes. In mathematische termen betekent dit dat de overeenkomstige autocorrelatiematrix

$$\mathbf{R}_{bb} = E\{\mathbf{b}(n)\mathbf{b}^T(n)\} \quad (2.66)$$

waarbij $\mathbf{b}(n) = [\mathbf{b}_0^T(n), \mathbf{b}_1^T(n), \dots, \mathbf{b}_{L_M-1}^T(n)]^T$ een diagonaalmatrix moet zijn. Dit houdt echter geen rekening met de correlatie die mogelijk bestaat tussen de twee kanalen en die net essentieel is in de context van stereo akoestische echo-onderdrukking. Een betere benadering toont aan dat de achterwaartse predictiefouten een orthogonale set vormen, d.i.

$$E\{\mathbf{b}(n)\mathbf{b}^T(n)\} = \tilde{\mathbf{R}}_{bb}(i)\delta(i-j) \quad (2.67)$$

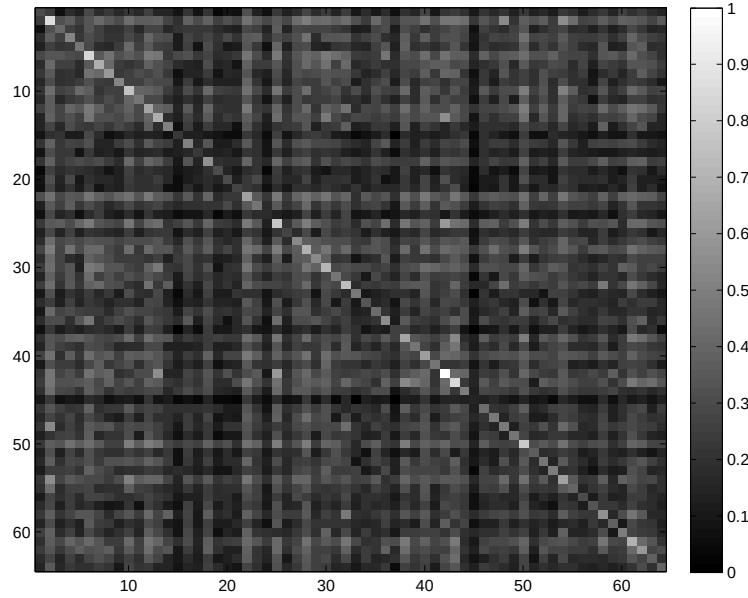
met $\delta(i-j)$ de Kronecker delta, m.a.w. 0 voor $i \neq j$ en 1 voor $i = j$. Hierbij duidt $\tilde{\mathbf{R}}_{bb}(i)$ op

$$\tilde{\mathbf{R}}_{bb}(i) = E\{\mathbf{b}_i(n)\mathbf{b}_i^T(n)\} \quad (2.68)$$

wat ook elementen buiten de diagonaal kan bevatten. $\mathbf{R}_{bb}$ is in het algemeen dan ook een blokdiagonaalmatrix [11].

IMPLEMENTATIE De eerder uitgewerkte implementatie is gebruikt om de structuur van de covariantiematrix $\mathbf{R}_{bb}$ zoals voorgesteld in vergelijking (2.68) te bekijken. Bij wijze van vergelijking zijn dezelfde parameters zoals voorgesteld in [11] gebruikt. Als ingangssignaal wordt normaal verdeelde witte ruis gebruikt, met een tijdsduur van 5s en een bemonsteringssfrequentie f_s van 8kHz. Voor de lengte van de roosterpredictor K is een lengte

32 gekozen. De kamerimpulsresponsies zijn gemodelleerd als een normaal verdeelde sequentie met gemiddelde 0 en variantie die afneemt in functie van de tijd, de lengte L_F is 128. Figuur 2.29 toont de elementen van de matrix $\mathbf{R}_{bb}$ voor een uitmiddelling over 10 verschillende uitvoeringen.



Figuur 2.29: Blokdiagonaal structuur autocorrelatiematrix achterwaartse predictiefouten $\mathbf{R}_{bb}$

Tweekanaals gradient rooster adaptief algoritme

In dit deel wordt het concept van gezamenlijke proces schatting uitgebreid naar twee kanalen. Het idee is hier een algoritme voor een adaptief filter uit te werken dat gebruik maakt van de achterwaartse predictiefouten, aangezien van deze fouten geweten is dat ze orthogonaal zijn. Verder wordt voor iedere iteratie een normalisatie ten opzichte van het signaalvermogen gedaan. Dit komt neer op een tweekanaals versie van een rooster gezamenlijke proces schatting [28, 11].

LINEAIR TRANSVERSAALFILTER Het adaptief algoritme dat hier verder zal worden uitgewerkt is een stap-genormaliseerd LMS- algoritme voor een lineair transversaalfilter. De coëfficiënten $\mathbf{c}(n)$ worden in volgende vorm bijgehouden

$$\mathbf{c}(n) = [c_{1,0}(n), c_{2,0}(n), \dots, c_{1,L_M-1}(n), c_{2,L_M-1}(n)]^T \quad (2.69)$$

Het aanpassen van de coëfficiënten over de tijd gebeurt met de volgende vergelijking [31]

$$\mathbf{c}(n+1) = \mathbf{c}(n) + \mu \mathbf{\Lambda}^{-1}(n) \mathbf{b}(n) e(n) \quad (2.70)$$

waarbij μ de stapgrootte is, $\mathbf{b}(n)$ een $2L_M \times 1$ vector die de achterwaartse predictiefouten bevat, $\mathbf{\Lambda}$ een normalisatiefactor en $e(n)$ een foutsignaal. Dit laatste foutsignaal wordt

berekend als

$$e(n) = d(n) - \mathbf{c}^T(n) \mathbf{b}(n) \quad (2.71)$$

met $d(n)$ het gewenste signaal zoals getoond in figuur 2.2. Bemerk dat $d(n)$ niet volledig overeen komt met de definitie zoals in vergelijking (2.2): alle monsters zitten nu in één grote signaalvector waarbij de monsters voor beide kanalen elkaar afwisselen.

NORMALISATIEFACTOR De normalisatiefactor Λ wordt gebruikt om de aanpassing van de coëfficiënten te normaliseren ten opzichte van een ogenblikkelijke schatting van het signaalvermogen. Eerder is uitgelegd en aangetoond dat de covariantiematrix $\mathbf{R}_{bb}$ van structuur blokdiagonaal is. Bijgevolg zal de matrix Λ een gelijkaardige structuur hebben, d.i. een $2L_M \times 2L_M$ matrix waarbij de elementen rond de diagonaal de matrices Λ_j zijn, voor de rest nul. De verschillende Λ_j , $j = 0 \dots L_M - 1$ zijn van de volgende vorm

$$\Lambda_j = \begin{bmatrix} P_{1,j}(n) & P_{c,j}(n) \\ P_{c,j}(n) & P_{2,j}(n) \end{bmatrix}, \quad j = 0, \dots, L_M - 1 \quad (2.72)$$

De $P_{i,j}$ zijn hierbij schattingen van de energie van de achterwaartse predictiefouten, de $P_{c,j}$ zijn specifiek toegevoegd om te compenseren voor het blokdiagonaal karakter van $\mathbf{R}_{bb}$ [11]. De ogenblikkelijke schatting van het vermogen kan recursief berekend worden: voor de $P_{i,j}$

$$P_{i,j}(n) = \beta P_{i,j}(n-1) + 0.5(1-\beta)[b_{i,j}^2(n)], \quad i = 1, 2; j = 0, \dots, L_M \quad (2.73)$$

en voor de $P_{c,j}$

$$P_{c,j}(n) = \beta P_{c,j}(n-1) + 0.5(1-\beta)[b_{1,j}(n)b_{2,j}(n)], \quad j = 0, \dots, L_M \quad (2.74)$$

GEMIDDELD GEDRAG TRANSVERSAALCOËFFICIËNTEN Om het gemiddeld gedrag van de lineaire transversaalcoëfficiënten te analyseren [11], wordt eerst een foutvector $\mathbf{v}(n)$ ingevoerd

$$\mathbf{v}(n) = \mathbf{c} - \mathbf{c}_0 \quad (2.75)$$

De vector $\mathbf{c}_0$ stelt de optimale coëfficiënten voor van het lineair transversaalfilter. Volgens Wiener filter theorie kunnen deze berekend worden als [4]

$$\mathbf{c}_0 = \mathbf{R}_{bb}^{-1} \mathbf{p}_{db} \quad (2.76)$$

met $\mathbf{R}_{bb}$ zoals in vergelijking (2.66) en $\mathbf{p}_{db}$ de kruiscorrelatie tussen het gewenste signaal $d(n)$ en de achterwaartse predictiefouten $\mathbf{b}(n)$

$$\mathbf{p}_{db} = E\{\mathbf{d}(n)\mathbf{b}(n)\} \quad (2.77)$$

De foutvector $e(n)$ kan dan met behulp van de $\mathbf{v}(n)$ herschreven worden als

$$\begin{aligned} e(n) &= d(n) - \mathbf{b}^T(n) \mathbf{c}(n) \\ &= e_0(n) - \mathbf{b}^T(n) \mathbf{v}(n) \end{aligned} \quad (2.78)$$

waar e_0 overeen stemt met de fout die bereikt wordt voor de optimale transversaalcoëfficiënten. Volgens het principe van orthogonaliteit geldt [4]

$$E\{e_0(n)\mathbf{b}(n)\} = 0 \quad (2.79)$$

Betreffende het gemiddeld gedrag van de coëfficiënten kan nu, op basis van vergelijkingen (2.70) en (2.78) het volgende berekend worden

$$E\{\mathbf{v}(n+1)\} = (\mathbf{I} - \mu E\{\mathbf{\Lambda}^{-1}(n)\mathbf{b}(n)\mathbf{b}^T(n)\}) E\{\mathbf{v}(n)\} \quad (2.80)$$

Uit vergelijking (2.80) kan het convergentiegedrag van het adaptief algoritme afgeleid worden. In deel 2.4.6 is reeds vermeld het convergentiegedrag van een stochastisch gradiënt algoritme zoals LMS volgt uit de eigenwaardenverdeling van de autocorrelatiematrix van het ingangssignaal. Meer bepaald zorgt iedere eigenwaarde voor een aparte modus van convergentie in de richting van de overeenstemmende eigenvector. In dit geval komt dit neer op de eigenwaardenverdeling van $E\{\mathbf{\Lambda}^{-1}(n)\mathbf{b}(n)\mathbf{b}^T(n)\}$. Om een goed convergentiegedrag te bekomen zou idealiter

$$E\{\mathbf{\Lambda}^{-1}(n)\mathbf{b}(n)\mathbf{b}^T(n)\} = \mathbf{I} \quad (2.81)$$

wat zou betekenen dat de convergentie voor iedere richting even snel gaat. Een goede keuze voor $\mathbf{\Lambda}$ komt dan overeen met een goede benadering voor $\mathbf{R}_{bb}$. In vergelijking (2.67) is reeds aangetoond dat deze van vorm blokdiagonaal is, in [11] wordt dit nogmaals aangetoond door een expliciete vergelijking uit te voeren tussen een $\mathbf{R}_{bb}$ die slechts een diagonaalmatrix is en een die een blokdiagonaalmatrix is qua opbouw.

LMS/Newton algoritme gebaseerd op AR modellering

Het lineair transversaalfilter zoals eerder voorgesteld heeft een intrinsiek probleem gerelateerd met de manier waarop de aanpassing van de coëfficiënten gebeurt. Zodra de AR parameters van het ingangssignaal wijzigen, e.g. bij de overgang tussen verschillende fonemen, worden de reflectiecoëfficiënten en de coëfficiënten van het lineair transversaalfilter tegelijk aangepast. Dit heeft tot gevolg dat het gemiddelde foutsignaal zal toenemen [11]. Een ander probleem is dat een rechtstreekse implementatie computationeel niet interessant is om uit te voeren voor grote filterlengtes. Zo is er nood aan een matrixinversie van een matrix met dimensies ter grootte van de filterlengte L_M voor iedere iteratie, wat in de praktijk niet doenbaar is. In dit deel wordt dan ook een alternatief, een LMS/Newton algoritme voorgesteld [11]. Veel van de eerder vermelde technieken komen hier in terug, hetzij in aangepaste vorm.

De decorrelatie van het signaal die nodig is om een goede uitlijning te bekomen wordt opnieuw gedaan met behulp van lineaire predictie, d.i. er wordt gebruik gemaakt van de orthogonaliteit van de achterwaartse predictiefouten. Aangezien het algoritme zal gebruikt worden voor het verwerken van spraak wordt er ook gebruik gemaakt van de link die er bestaat tussen lineaire predictie en de AR parameters.

Het adaptief algoritme is gebaseerd op een variant van het LMS-algoritme: het LMS/Newton algoritme, reeds aangehaald in deel 2.4.6 bij de bespreking van orthogonalisatiemethodes. In het origineel LMS/Newton algoritme uit vergelijking (2.58) wordt

om een normalisatie van de stapgrootte door te voeren gebruik gemaakt van $\mathbf{R}_{xx}$, wat een schatting is van de covariantiematrix van de ingangssignalen: $\mathbf{R}_{xx} = E\{\mathbf{x}(n)\mathbf{x}(n)^T\}$. Doordat er gebruik gemaakt wordt van AR modellering, zal het uiteindelijk algoritme gebaseerd zijn op $\mathbf{R}_{bb}$, d.i. de covariantiematrix van de achterwaartse predictiefouten. Aangezien de orde voor de AR modellering M in de praktijk veel kleiner is dan de filterlengte L_M , zal de dimensie van de gebruikte covariantiematrix ook veel lager liggen, wat een belangrijk computationeel voordeel is.

AANPASSING ADAPTIEVE COËFFICIËNTEN Om een uitdrukking te vinden om de adaptieve coëfficiënten aan te passen in functie van de achterwaartse predictiefouten, beschouw volgende factorisatie

$$\mathbf{b}(n) = \mathbf{L}\mathbf{x}(n) \quad (2.82)$$

Hierbij is $\mathbf{x}(n)$ het signaal dat van het verre eind komt, specifiek worden hier de laatste L_M monsters bedoeld voor beide kanalen

$$\mathbf{x}(n) = [x_1(n), x_2(n), x_1(n-1), x_2(n-1), \dots, x_1(n-L_M+1), x_2(n-L_M+1)]^T \quad (2.83)$$

De $2L_M \times 2L_M$ transformatiematrix $\mathbf{L}$ is als volgt opgebouwd

$$\mathbf{L} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} \\ -\mathbf{G}_{1,1} & \mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} \\ -\mathbf{G}_{2,1} & -\mathbf{G}_{2,2} & \mathbf{I} & \dots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -\mathbf{G}_{L_M-1,1} & -\mathbf{G}_{L_M-1,2} & -\mathbf{G}_{L_M-1,3} & \dots & -\mathbf{G}_{L_M-1,L_M-1} & \mathbf{I} \end{bmatrix} \quad (2.84)$$

Gelijkaardig aan de transformatiematrix in [4], is hier in blokdiagonaalvorm $\mathbf{I}$ een 2×2 eenheidsmatrix, $\mathbf{0}$ een 2×2 nulmatrix en iedere $\mathbf{G}_{m,j}$ een 2×2 matrix van achterwaartse predictieveeltermen. Gegeven de ontbinding van vergelijking (2.82), kan de covariantiematrix $\mathbf{R}_{xx}$ herschreven worden in functie van de covariantiematrix van de achterwaartse predictiefouten

$$\mathbf{R}_{xx}^{-1} = \mathbf{L}^T \mathbf{R}_{bb}^{-1} \mathbf{L} \quad (2.85)$$

De LMS/Newton vergelijking (2.58) waarin de coëfficiënten voor het adaptief filter worden aangepast kan hier meer expliciet uitgeschreven worden als

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \mathbf{R}_{xx}^{-1} \mathbf{x}(n) e(n) \quad (2.86)$$

waarbij de vorm waarin deze bijgehouden worden vergelijkbaar is met deze van $\mathbf{x}(n)$ in vergelijking (2.83)

$$\mathbf{h}(n) = [h_{1,0}(n), h_{2,0}(n), h_{1,1}(n), h_{2,1}(n), \dots, h_{1,L_M-1}(n), h_{2,L_M-1}(n)]^T \quad (2.87)$$

Een belangrijk verschil met de vergelijking voor het lineair combinerend filter (2.70) is dat verstoringen op de reflectiecoëfficiënten enkel direct merkbaar zijn in de $\mathbf{R}_{xx}^{-1}$; de andere factoren $\mathbf{x}(n)$ en $e(n)$ zijn hier niet rechtstreeks onderhevig aan. In de veronderstelling dat μ klein is, blijft het LMS/Newton algoritme robuust onder veranderingen in het stationair proces waarvan de parameters worden berekend met behulp van lineaire predictie [11].

In analogie met de notatie gebruikt in [11] wordt de factor adaptieve stapgrootte gevat in vector $\mathbf{u}(n)$

$$\mathbf{u}(n) = \mathbf{L}^T \mathbf{R}_{bb}^{-1} \mathbf{b}(n) \quad (2.88)$$

zodanig dat de aanpassing van de adaptieve filtercoëfficiënten uit vergelijking (2.86) korter kan geschreven worden als

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \mathbf{u}(n) e(n) \quad (2.89)$$

Bemerk dat in vergelijking (2.88) $\mathbf{R}_{bb}$ gebruikt wordt in plaats van $\mathbf{R}_{xx}$ uit de originele LMS/Newton vergelijking. Aangezien $\mathbf{R}_{bb}$ van een veel lagere dimensie is betekent dit een groot computationeel voordeel, zoals eerder al aangegeven.

INTEGRATIE MET ROOSTERPREDICTOR De essentie van dit algoritme komt neer op het bepalen van de adaptieve stapgrootte $\mathbf{u}(n)$ uit vergelijking (2.88) met behulp van de parameters $\mathbf{b}(n)$ afkomstig van een roosterpredictor, uit deel 2.5.2. De lengte, of de orde van de roosterpredictor M kan variëren van vrij kleine waarden tot de lengte van het adaptief filter L_M : $M \ll L_M$. De achterwaartse predictiefouten voor beide kanalen worden opnieuw bijgehouden in een enkele kolomvector, dimensie $2L_M \times 1$

$$\begin{aligned} \mathbf{b}(n) = & [b_{1,0}(n), b_{2,0}(n), b_{1,1}(n), b_{2,1}(n), \dots, \\ & b_{1,M}(n), b_{2,M}(n), b_{1,M}(n-1), b_{2,M}(n-1), \dots, \\ & b_{1,M}(n-L_M+M+1), b_{2,M}(n-L_M+M+1)]^T \end{aligned} \quad (2.90)$$

Gezien de structuur van $\mathbf{b}(n)$ in vergelijking (2.90) moeten op ieder tijdstip slechts de eerste $M+1$ waarden van $\mathbf{b}(n)$ aangepast worden. De overige elementen zijn vertraagde versies van $\mathbf{b}_M(n) = [b_{1,M}(n), b_{2,M}(n)]^T$. Voor de vermenigvuldiging van $\mathbf{b}(n)$ met $\mathbf{R}_{bb}^{-1}$ zijn de vermogens van de $\mathbf{b}_i(n)$, $i = 0 \dots M$ nodig. Deze kunnen bepaald worden met behulp van de vergelijkingen die voorgesteld zijn voor het lineaire transversaalfilter eerder in deel 2.5.2: de 2×2 normalisatiematrices $\mathbf{\Lambda}_j(n)$ worden hier berekend zoals in vergelijking (2.72). Bemerk dat bij de berekening van $\mathbf{u}(n)$ een matrixinversie verondersteld wordt. In dit geval is dit geen al te groot probleem aangezien het aantal matrixinversies hier beperkt is door de lage orde M van de roosterpredictor.

Een laatste stap in de berekening van $\mathbf{u}(n)$ is de terugtransformatie met behulp van de matrix $\mathbf{L}^T$. De AR modellering in beschouwing genomen, wordt deze matrix van de volgende vorm

$$\mathbf{L} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ -\mathbf{G}_{1,1} & \mathbf{I} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ -\mathbf{G}_{M,1} & -\mathbf{G}_{M,2} & \dots & \mathbf{I} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & -\mathbf{G}_{M,1} & \dots & -\mathbf{G}_{M,M} & \mathbf{I} & \dots & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \mathbf{0} & \dots & -\mathbf{G}_{M,1} & -\mathbf{G}_{M,2} & \dots & \mathbf{I} \end{bmatrix} \quad (2.91)$$

In de praktijk hoeft de matrixvermenigvuldiging $\mathbf{L}^T$ met $\mathbf{R}_{bb}^{-1} \mathbf{b}(n)$ niet volledig te worden uitgevoerd: een groot gedeelte van elementen van $\mathbf{u}(n)$ zijn vertraagde versies van zichzelf.

Enkel de eerste $2(M+1)$ en de laatste $2M$ elementen van $\mathbf{u}(n)$ moeten aangepast worden, de overige elementen zijn herhalingen van monsters $2M+1$ en $2M+2$. De blokmatrices $G_{m,j}$ van achterwaartse predictieveltermen kunnen met behulp van een tweekanaals Levinson-Durbin algoritme berekend worden uit de achterwaartse predictiefouten $\mathbf{b}(n)$. In hoofdstuk 3 wordt hier overigens verder op teruggekomen.

ALGORITME Het uiteindelijke algoritme zoals voorgesteld in [11] kan worden samengevat in de volgende stappen

1. Voer de roosterpredictor van orde M uit op het ingangssignaal om de achterwaartse predictiefouten $\mathbf{b}(n)$ en de reflectiecoëfficiënten $\boldsymbol{\kappa}_m(n)$ te bekomen. Dit komt neer op vergelijkingen
 - voor de roosterpredictor: (2.60), (2.61), (2.62), (2.63), (2.64) en (2.65)
 - voor de normalisatie ten opzichte van het ogenblikkelijk vermogen: (2.72), (2.73) en (2.74)
2. Om de matrices $G_{m,j}$ te berekenen, wordt gebruik gemaakt van het tweekanaals Levinson-Durbin algoritme. De verbanden die in deze stap gebruikt worden, worden hier expliciet opnieuw uiteengezet aangezien deze niet zo evident zijn als voor het overeenkomstig eenkanaals Levinson-Durbin algoritme.

$$\begin{aligned}
 \mathbf{A}_{1,1} &= \boldsymbol{\kappa}_1^T(n) \\
 \mathbf{G}_{1,1} &= \boldsymbol{\kappa}_1(n) \\
 \text{for } m &= 1 : M-1 \\
 \mathbf{A}_{m+1,j}(n) &= \mathbf{A}_{m,j}(n) - \boldsymbol{\kappa}_{m+1}^T(n) \mathbf{G}_{m,j}(n) \quad j = 1, \dots, m \\
 \mathbf{A}_{m+1,m+1}(n) &= \boldsymbol{\kappa}_{m+1}^T(n) \\
 \mathbf{G}_{m+1,j+1}(n) &= \mathbf{G}_{m,j}(n) - \boldsymbol{\kappa}_{m+1}(n) \mathbf{A}_{m,j}(n) \quad j = 1, \dots, m \\
 \mathbf{G}_{m+1,1}(n) &= \boldsymbol{\kappa}_{m+1}(n)
 \end{aligned} \tag{2.92}$$

Vervolgens worden de verschillende delen van de adaptieve stapgroottevector $\mathbf{u}(n)$ berekend.

3. Het middelste gedeelte van de vector $\mathbf{u}(n)$: deze zijn allen vertraagde versies van monsters $2M+1$ en $2M+2$. Qua notatie betekent $\mathbf{u}_j(n) = [u_{1,j}(n), u_{2,j}(n)]^T$, d.i. een vereenvoudigde notatie om om te gaan met het feit dat de kolomvector $\mathbf{u}(n)$ afwisselend monsters bevat van twee kanalen

$$\begin{aligned}
 u_{1,j} &= u_{1,j-1}(n-1) \quad j = M+1, M+2, \dots, L_M - M - 1 \\
 u_{2,j} &= u_{2,j-1}(n-1) \quad j = M+1, M+2, \dots, L_M - M - 1
 \end{aligned} \tag{2.93}$$

4. Het kopgedeelte van $\mathbf{u}(n)$: de eerste $2(M+1)$ elementen. Een aparte notatie cfr. vergelijking (2.88) wordt ingevoerd

$$[u_{1,0}(n), u_{2,0}(n), u_{1,1}(n), u_{2,1}(n), \dots, u_{1,M}(n), u_{2,M}(n)]^T = \mathbf{L}_{tl}^T \mathbf{b}_h(n) \tag{2.94}$$

Hierbij is $\mathbf{L}_{tl}$ de blokmatrix in de linkerbovenhoek van de transformatiematrix uit vergelijking (2.91) van dimensie $2(2M+1) \times 2(M+1)$. De vector $\mathbf{b}_h(n)$ is nog eens uit

twee verschillende delen $\mathbf{b}_{h,1}(n)$, $\mathbf{b}_{h,2}(n)$ opgebouwd, $\mathbf{b}_h(n) = [\mathbf{b}_{h,1}^T(n), \mathbf{b}_{h,2}^T(n)]^T$

$$\mathbf{b}_{h,1}(n) = [[\Lambda_0^{-1}(n)\mathbf{b}_0(n)]^T, [\Lambda_1^{-1}(n)\mathbf{b}_1(n)]^T, \dots, [\Lambda_M^{-1}(n)\mathbf{b}_M(n)]^T]^T \quad (2.95)$$

$$\mathbf{b}_{h,2}(n) = [[\Lambda_M^{-1}(n)\mathbf{b}_M(n-1)]^T, \dots, [\Lambda_M^{-1}(n)\mathbf{b}_M(n-M)]^T]^T \quad (2.96)$$

waarbij de Λ_j de vermogen normalisatiematrixes zijn zoals gedefinieerd in vergelijking (2.72). De notatie $\mathbf{b}_j(n)$ is opnieuw een vereenvoudigde notatie voor de 2×1 vectoren $[b_{1,j}(n), b_{2,j}(n)]^T$.

5. Het staartgedeelte van $\mathbf{u}(n)$: de laatste $2M$ elementen. Een gelijkaardige notatie cfr. vergelijking (2.94) wordt hier gebruikt

$$[u_{1,L_M-M}(n), u_{2,L_M-M}(n), \dots, u_{1,L_M-1}(n), u_{2,L_M-1}(n)]^T = \mathbf{L}_{br}^T \mathbf{b}_t(n) \quad (2.97)$$

Hierbij is $\mathbf{L}_{br}$ de blokmatrix in de rechterbenedenhoek van de transformatiematrix uit vergelijking (2.91) van dimensie $2M \times 2M$. De vector $\mathbf{b}_t(n)$ wordt als volgt opgebouwd

$$\begin{aligned} \mathbf{b}_t(n) = & [[\Lambda_M^{-1}(n-L_M+M+1)\mathbf{b}_M(n-L_M+2M)]^T, \\ & [\Lambda_M^{-1}(n-L_M+M+1)\mathbf{b}_M(n-L_M+2M-1)]^T, \dots, \\ & [\Lambda_M^{-1}(n-L_M+M+1)\mathbf{b}_M(n-L_M+M+1)]^T]^T \end{aligned} \quad (2.98)$$

Op deze manier is de volledige vector $\mathbf{u}(n)$ gekend. Bij wijze van illustratie, een overzicht van de volledige vector in vergelijking (2.99). De nota's aan de linkerkant van de pijlen geven begin en eindindex aan, vanaf index 1 gebaseerd (cfr. indexering in MATLAB), de rechterkant geeft de lengte van het desbetreffende deel aan.

$$\begin{aligned} & \begin{bmatrix} u_{1,0} \\ u_{2,0} \end{bmatrix} \\ & \begin{matrix} 1 \rightarrow 2(M+1) \end{matrix} \downarrow \begin{matrix} 2(M+1) \end{matrix} \\ & \begin{bmatrix} u_{1,M} \\ u_{2,M} \end{bmatrix} \\ & \begin{bmatrix} u_{1,M+1} \\ u_{2,M+1} \end{bmatrix} \\ & \begin{matrix} 2M+3 \rightarrow 2(L_M-M+1) \end{matrix} \downarrow \begin{matrix} 2(L_M-2M-1) \end{matrix} \\ & \begin{bmatrix} u_{1,L_M-M-1} \\ u_{2,L_M-M-1} \end{bmatrix} \\ & \begin{bmatrix} u_{1,L_M-M} \\ u_{2,L_M-M} \end{bmatrix} \\ & \begin{matrix} 2(L_M-M)+1 \rightarrow 2L_M \end{matrix} \downarrow \begin{matrix} 2M \end{matrix} \\ & \begin{bmatrix} u_{1,L_M-1} \\ u_{2,L_M-1} \end{bmatrix} \end{aligned} \quad (2.99)$$

6. Tot slot wordt de uitgang van het adaptief filter $y(n)$ berekend als

$$y(n) = \hat{\mathbf{h}}^T(n) \mathbf{x}(n) \quad (2.100)$$

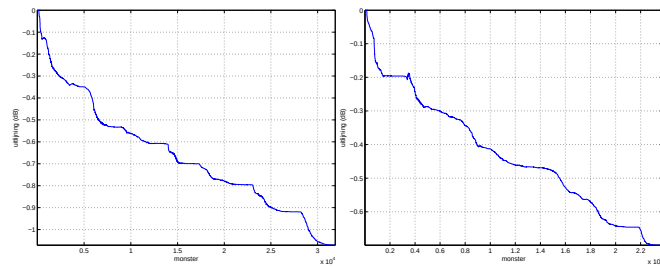
het foutsignaal als

$$e(n) = d(n) - y(n) \quad (2.101)$$

Aangezien de volledige vector $\mathbf{u}(n)$ gekend is, kunnen de adaptieve filtercoëfficiënten nu bijgesteld worden via vergelijking (2.89).

SIMULATIES Hier volgen een aantal simulaties voor het uiteindelijk algoritme zoals voorgesteld door [11]. In vergelijking met de eerdere simulaties voor e.g. het illustreren van het fundamentele probleem in deel 2.3.6 zijn hier wel een aantal verschillen. Zo worden hier niet langer dezelfde kamerimpulsresponsies genomen en zullen de parameters m.b.t. de stapgrootte ook verschillen aangezien dit algoritme geen rechtstreekse toepassing meer is van het originele NLMS-algoritme. Praktisch worden voor de kamerimpulsresponsies volgende data gebruikt. Voor het verre eind worden dezelfde Gaussiaanse sequenties gebruikt met afnemende variantie zoals eerder beschreven in deel 2.3.6, met een lengte van 2048 monsters. Voor het dichte eind daarentegen worden echte kamerimpulsresponsies genomen, opgenomen aan 16kHz met een lengte van 2001 monsters. Als ingangssignaal wordt naast de mannen- en vrouwenstem ook (deel van) een audiosignaal gebruikt [32], allen herbemonsterd tot een bemonsteringsfrequentie van 16kHz.

Aangezien de opstelling die hier gebruikt is verschilt van deze eerder besproken, worden hier eerst opnieuw de resultaten in termen van de uitlijning gegeven voor het toepassen van het standaard NLMS-algoritme; zie figuur 2.30. Bemerk dat voor deze kamerimpulsresponsies de uitlijning hoger ligt dan voor deze besproken in figuur 2.7. Vervolgens toont figuur 2.31 de uitlijning na toepassing van het Newton/LMS-algoritme

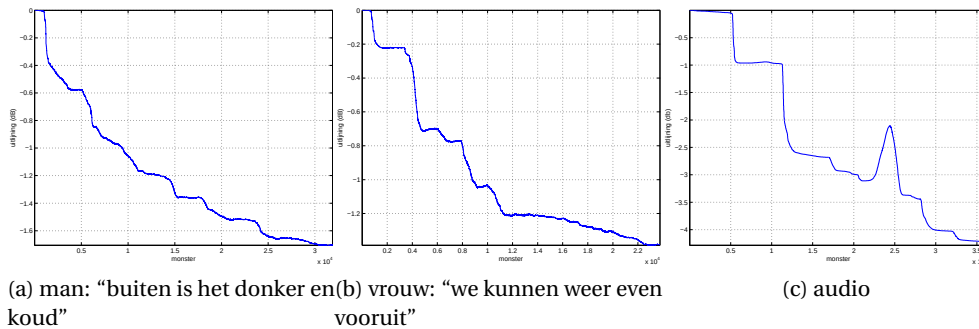


(a) man: “buiten is het donker en koud”
(b) vrouw: “we kunnen weer even vooruit”

Figuur 2.30: Simulatie standaard NLMS-algoritme

gebaseerd op gezamenlijke proces schatting. Bij vergelijking van deze figuur met deze zonder de toepassing van enige decorrelatie, blijkt dat het algoritme in staat is om voldoende decorrelatie te verkrijgen zodanig dat uitlijning sneller bereikt wordt. Figuur 2.31c toont daarbij de uitlijning voor een audiosignaal [32], waarvoor uiteindelijk een uitlijning van -4dB behaald wordt. Bemerk dat om deze resultaten te bereiken voor alle signalen een zeer kleine stapgrootte μ_r voor de roosterpredictor gebruikt is, in de orde van 10^{-6} .

Dit betekent dat, hoewel de roosterpredictor om de uitlijning te bereiken nog steeds een decorrelerend effect heeft, de resultaten ervan niet meer overeen komen met de originele interpretatie van de LPC's. Aangezien dit net een voorwaarde is om een integratie met een codeerschema op basis van lineaire predictie mogelijk te maken, zal in hoofdstuk 4 blijken dat dit niet meer zo voor de hand ligt. Desalniettemin zijn goede resultaten mogelijk met dit algoritme.



Figuur 2.31: Simulatie gezamenlijke proces schatting – Newton/LMS-uitlijning

2.6 Samenvatting

In deel 2.2 is het probleem van echo-onderdrukking opnieuw uitgelegd geweest voor het eenvoudige, monofone geval. Vervolgens is in deel 2.3 dit probleem beschouwd voor het stereofone geval. Na een korte bespreking van de opstelling in deel 2.3.1, is aangetoond dat een uitbreiding naar het stereofone geval niet zo evident is door de correlatie die bestaat tussen de twee kanalen. Uit de intuïtieve benadering kon besloten worden dat het bepalen van de echopaden niet meer evident is doordat vergelijking (2.15) niet meer éénduidig deze echopaden bepaalt. In de meer mathematische benadering die er op volgde is het effect van de lengtes van de verschillende kamerimpulsresponsies in rekening gebracht en samengevat in deel 2.3.4. Een belangrijke conclusie hier is dat de meest praktische oplossing voor het SAEO-probleem er in zal bestaan om een extra decorrelerende stap te voorzien, vooraleer een adaptief filter algoritme toe te passen. Om dit deel af te sluiten, zijn in deel 2.3.6 een aantal simulaties gegeven om een wat praktischer idee te krijgen.

Na de beschrijving van het SAEO-probleem, is in de volgende delen gepoogd om een opsomming te geven van een aantal mogelijke oplossingen. In deel 2.4 zijn een aantal algemene oplossingen aangereikt, allemaal met als doel om de twee kanalen te decorreleren. Voor sommige methodes, zoals het toevoegen van ongecorreleerde ruis aan één van de kanalen en het toepassen van een niet-lineaire operatie op één van de kanalen, zijn opnieuw een aantal simulaties getoond, die het effect van decorrelatie op de uitlijning aantonen. Van andere methodes, zoals het gebruik van decorrelatiefilters en verschoven kamfilters zijn enkel hun bestaan vermeld. Hoewel niet concreet toegepast, is in deze context de methode gebaseerd op orthogonale transformaties uit deel 2.4.6 nog

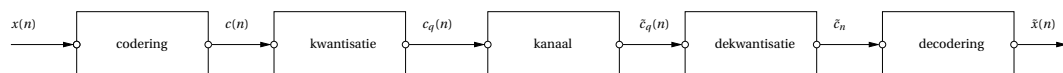
de moeite waard te vermelden, aangezien deze o.a. een beschrijving van het LMS/Newton bevat, belangrijk voor de beschrijving van één van de algoritmes gebaseerd op lineaire predictie.

In het laatste deel is overgegaan tot een andere categorie van oplossingen, nl. deze gebaseerd op lineaire predictie. De voornaamste redenen om een oplossing op basis van lineaire predictie te gebruiken is enerzijds de decorrelerende eigenschap van een lineair predictief analysefilter, anderzijds de mogelijke integratie met een efficiënt coderingsschema zoals beschreven in hoofdstuk 3. Specifiek worden er twee algoritmes voorgesteld. Het eerste algoritme in deel 2.5.1 bestaat uit een directe toepassing van het NLMS-algoritme op de predictiefouten van een lineaire predictie. Enkel het principe wordt hier uitgelegd, aangezien lineaire predictie voor meerdere kanalen verder wordt uitgelegd in hoofdstuk 3 en een integratie met een echo-onderdrukkingsalgoritme volgt in hoofdstuk 4. Het tweede algoritme in deel 2.5.2 maakt ook gebruik van de predictiefouten van een lineaire predictie, hetzij op een andere manier. Allereerst verloopt de schatting van de lineaire predictieparameters op een recursieve manier in plaats van blokgebaseerd, zoals beschreven in het deel van de roosterpredictor, deel 2.5.2. Uit simulaties bleek dat de roosterpredictor in staat is om in bepaalde omstandigheden de AR parameters van het signaal te extraheren. In deel 2.5.2 is, in analogie met een monofone gezamenlijke proces schatting een gelijkaardig algoritme beschreven. In deel 2.5.2 worden deze concepten geïntegreerd in een LMS/Newton algoritme, d.i. een LMS-algoritme met een stapgrootte die genormaliseerd wordt ten opzichte van de autocorrelatiematrix van het ingangssignaal. Deze normalisatieprocedure wordt vereenvoudigd door gebruik te maken van de ontbinding van het signaal in functie van de predictiefouten. Simulaties om de werking hiervan aan te tonen besluiten dit deel. Een belangrijk resultaat uit deze simulaties, is dat dit laatste algoritme het beste werkt wanneer de stapgrootte voor het recursief schatten van de AR parameters van het signaal zeer klein is. Dit geeft aan dat het algoritme niet langer gebruik maakt van een authentieke schatting van de AR parameters, maar op basis van een ander mechanisme een decorrelatie uitvoert. In hoofdstuk 4 wordt hier op terug gekomen bij de integratie van beide delen.

Stereo codering

3.1 Inleiding

Tijdens een tele- of videoconferentie wordt, naast het akoestisch onderdrukken van de gegenereerde echo in elke deelnemende kamer, uiteraard ook data doorgestuurd tussen de twee kamers. Het basisschema van zo'n transmissie wordt in figuur 3.1 weergegeven. De tildes wijzen op een afwijking van het originele signaal, aangezien bij doorsturen over het kanaal de aanwezigheid van ruis in rekening moet worden gebracht. Vandaar dat – indien geen extra foutcorrigerende codes worden voorzien – het ontvangen signaal slechts een benadering van het originele zal zijn. Een belangrijke factor bij dergelijke



Figuur 3.1: Schema van een algemene transmissie met codering en kwantisatie.

transmissie is de soort codering die wordt gebruikt voor datacompressie. Zo bestaat er een onderscheid tussen verliesloze en benaderende codering, waarbij deze laatste een signaal met fouten codeert zodat geen foutloze reconstructie mogelijk is. Het grote praktische verschil hiertussen bevindt zich in de te behalen compressieverhouding, nl. hoeveel bit wordt gebruikt na codering t.o.v. voor codering voor dezelfde data. Benaderende codering behaalt over het algemeen een grotere compressiefactor dan verliesloze codering, maar verliest tegelijk in geluidskwaliteit ten gevolge van een slechtere nauwkeurigheid.

Beiden van de hierboven vermelde coderingstypes kunnen gerealiseerd worden en dit door verschillende coderingsschema's. Daarom worden in sectie 3.2 enkele van deze schema's, bruikbaar voor audio- en spraaksignalen, vergeleken met de uiteindelijk geïmplementeerde stereo codering gebaseerd op lineaire predictie. Deze laatste wordt besproken in sectie 3.3, waarin ook de link gelegd wordt tussen het bron-filtermodel als model voor spraak en lineaire predictieve codering. Hiermee is ook meteen de voornaamste reden gegeven waarom voor lineaire predictieve codering als codering van de verstuurde spraaksignalen is gekozen. Ten slotte bespreekt sectie 3.4 de uitbreiding van deze coderingsvorm van monofone spraaksignalen naar stereofone meer algemene audiosignalen en de problemen die hiermee gepaard gaan.

3.2 Coderingsvormen

3.2.1 Som-verschilcodering

Een eerste mogelijkheid is som-verschilcodering [33]. De naam van dit schema verklapt al grotendeels de werking ervan. Zo wordt van de signalen in linker- en rechterkanaal de helft van zowel de som als het verschil genomen. Deze bewerkingen worden voor de volledigheid hieronder weergegeven.

$$M(n) = \frac{1}{2}(L(n) + R(n))$$

$$S(n) = \frac{1}{2}(L(n) - R(n))$$

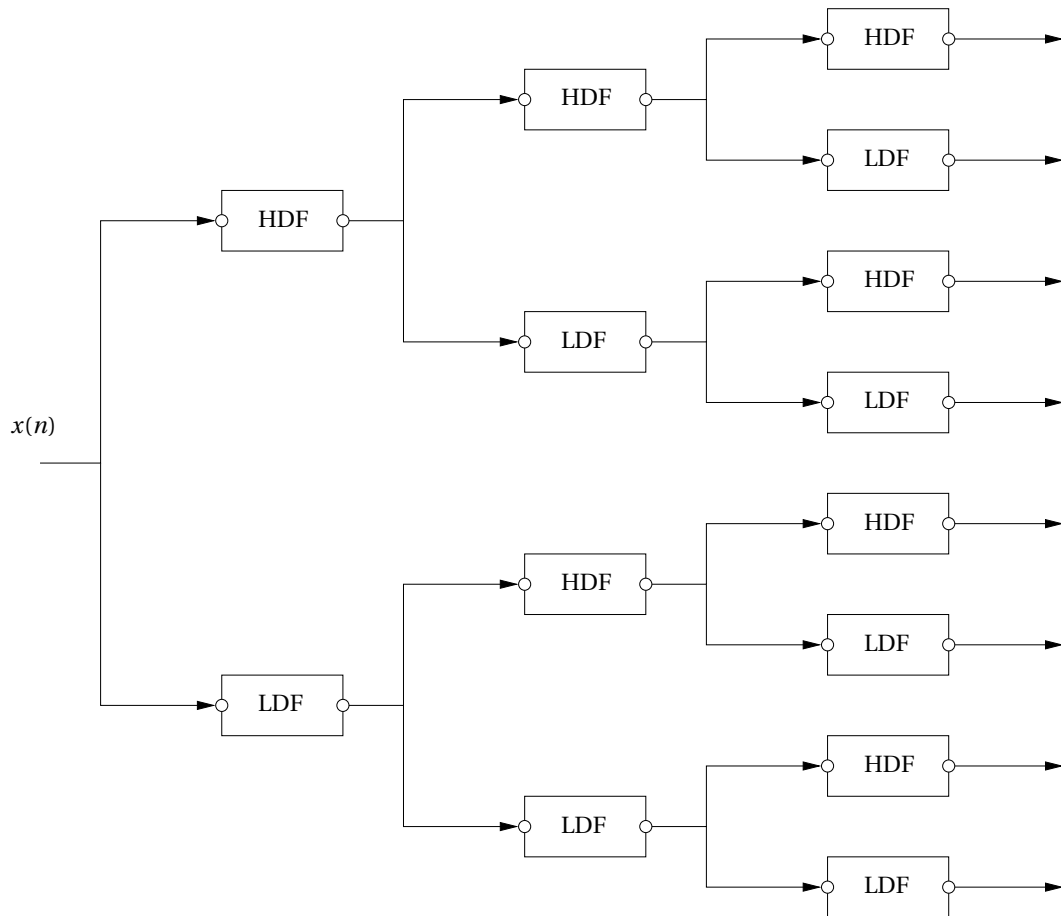
De bekomen resultaten worden verdeeld in tijds- en frequentiedomein en vervangen door de oorspronkelijke signaalwaarden L en R indien welbepaalde grenzen van deze laatste twee minder dan 2 dB van elkaar verschillen. Zo garandeert men dat voor sterk gecorreleerde ingangssignalen het berekende verschilsignaal S weinig energie bevat. Hiervan kan handig gebruik worden gemaakt door weinig (of geen) bits te voorzien voor verdere codering van S . Op deze manier wordt een compressieverhouding bereikt tussen 0% – gebruik van signalen L en R – en 50%. In vergelijking met de bitreductie van lineaire predictie is deze techniek dan ook minder geschikt voor praktische toepassingen [34].

3.2.2 Subbandcodering

Som-verschilcodering is slechts één van de vele coderingen die (al dan niet gedeeltelijk) werken in het frequentiedomein. Subbandcodering, een ander coderingsschema dat over het algemeen meer toegepast wordt voor audiosignalen dan voor spraak, kan enigszins toch voor spraak worden gebruikt [35]. Het basisprincipe beslaat een opdeling van de ingangssignalen in frequentiebanden, waarna een bepaald soort codering wordt toegepast in elke band afzonderlijk. De splitsing in banden gebeurt het vaakst via *QMF*-filters. Deze hebben de eigenschap dat de impulsresponsies van het laagdoorlaatfilter (LDF) en het overeenkomstig hoogdoorlaatfilter (HDF) gegeven worden door respectievelijk $\{h_n\}$ en $\{(-1)^n h_{N-1-n}\}$, met N de filterlengte.

De twee filters zelf worden in meerdere opeenvolgende niveaus in een boomstructuur (zie figuur 3.2) geplaatst, zó dat het gesplitste signaal telkens in twee gelijke delen wordt verdeeld in het frequentiedomein. Dit gebeurt al dan niet met overlappende overdrachtsfuncties van de verschillende filters. Uiteindelijk wordt het oorspronkelijke signaal zo in n frequentiebanden van gelijke grootte opgedeeld, met n een macht van 2 [35].

Het grote nadeel van deze codering en tegelijk één van de redenen waarom hier geopteerd wordt voor het gebruik van lineaire predictieve codering als alternatief is de nood aan een extra vertraging tijdens de uitvoering. Deze vertraging vindt zijn oorsprong in de transformatie naar het frequentiedomein, aangezien dit voor blokken van monsters moet worden uitgevoerd zodat een extra geheugen aanwezig moet zijn aan de ingang. Als gevolg hiervan is subbandcodering dan ook minder geschikt voor real-time toepassingen.



Figuur 3.2: Boomstructuur bekomen door herhaaldelijk gebruik van een LDF en HDF.

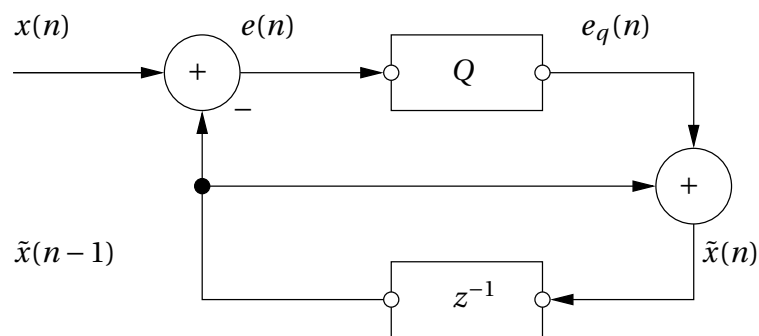
3.2.3 Woordenboekcodering

Een volgende mogelijkheid, het gebruik van woordenboektechnieken, bevat zoals de naam al duidelijk maakt een statisch of dynamisch woordenboek. De verschillende termen in zo'n woordenboek komen overeen met mogelijk door te sturen signalen of een samenstelling van deze signalen en worden elk afzonderlijk gecodeerd met een unieke bitvoorstelling. Zoals eerder vermeld gebeurt de implementatie statisch of dynamisch. Het eerste is van toepassing wanneer alle signalen op voorhand gekend zijn en is het meest efficiënt indien het totaal aantal signalen beperkt is. Men verkiest daarentegen een dynamische aanpak als er geen a priori kennis is van de te verzenden monsters, zodat het woordenboek "on the fly" moet worden opgesteld. De bekendste en meest geïmplementeerde versie van een adaptieve woordenboektechniek, o.a. voor beeldcompressie in het GIF-formaat [36], is het Lempel-Ziv-Welch algoritme [37], waarbij nieuwe woordenboekelementen worden toegevoegd op basis van de vorige verzonden symbolen. Voor een meer gedetailleerde beschrijving van dit algoritme en andere alternatieven wordt de lezer naar de literatuur verwezen [35, p. 127-132],[35, hoofdstuk 5].

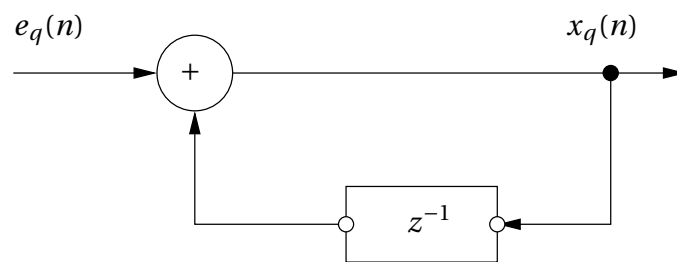
Hoewel woordenboekmethoden voor bepaalde specifieke toepassingen heel efficiënt blijken te zijn, hebben ze voor codering van audio- en spraaksignalen weinig nut. De reden hiervoor is eenvoudig: aangezien de aan te nemen amplitudes van hetingangssignaal te talrijk zijn (zelfs na kwantisatie), zou elk woordenboek te groot zijn en te veel geheugen innemen aan de kant van zowel zender als ontvanger.

3.2.4 Differentiële codering

Differentiële codering, het laatste coderingsschema dat hier wordt besproken, kan zowel benaderend als verliesloos worden aangewend. De algemene werking van de codering langs zendkant wordt getoond op figuur 3.3, terwijl de ontvanger de operatie in figuur 3.4 uitvoert [35]. Het gekwantiseerde foutsignaal $e_q(n)$ wordt niet alleen doorgestuurd, maar ook teruggekoppeld en gesommeerd bij de vorige benaderde signaalwaarde $\tilde{x}(n)$ om verdere propagatie van de fout te verhinderen.



Figuur 3.3: Algemeen schema van differentiële codering met vertragingselement.



Figuur 3.4: Algemeen schema van differentiële decoding met vertragingselement.

Oorspronkelijk wordt slechts het verschil tussen de huidige en de vorige signaalwaarde gekwantiseerd en doorgestuurd. Dit is vooral efficiënt wanneer de te verzenden boodschap weinig variatie bevat en dus voornamelijk laagfrequent is. In andere gevallen is deze manier van coderen minder aantrekkelijk en wordt er een verbetering aan het basisalgoritme toegevoegd. Zo wordt het vertragingselement in de zender vervangen door een predictieblok, waarin via een bepaald predictiealgoritme de volgende monsterwaarde

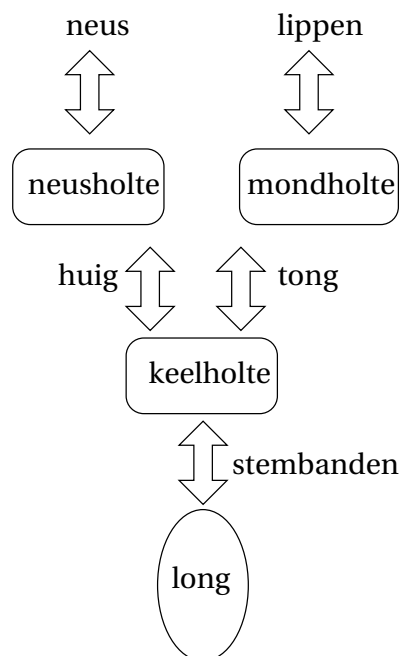
wordt geschat op basis van één of enkele van de vorige waarden. Een goede voorspelling van de volgende signaalwaarde zorgt zo voor een kleinere fout.

Het voordeel van deze methode schuilt, zoals al vermeld, in het beperkte bereik van het resulterende foutsignaal $e(n)$, waardoor het aantal bits nodig voor de codering ervan sterk wordt gereduceerd en zo een grotere compressie wordt verkregen. Indien daarnaast ook benaderende compressie mogelijk is, kan een nog grotere compressieverhouding worden bekomen, wat deze methode zeer nuttig en algemeen bruikbaar maakt.

3.3 Spraakcodering via lineaire predictieve codering

3.3.1 Anatomische achtergrond van spraak

Het geheel van verschillende belangrijke anatomische onderdelen die bijdragen tot de synthese van spraak zijn schematisch voorgesteld in figuur 3.5 hieronder.



Figuur 3.5: Schematische voorstelling van de componenten van het menselijk spraakapparaat.

Bij de generatie van fonemen, het basiselement waaruit geluid en dus ook spraak is opgebouwd, wordt lucht uit de longen door de stembanden gestuurd en via de mond- en neusholte naar buiten geleid. Hierbij zijn twee mogelijke klanken te vormen: stemhebbend en stemloos. Het verschil tussen deze twee bevindt zich in het al dan niet laten trillen van de stembanden tijdens het passeren van de lucht.

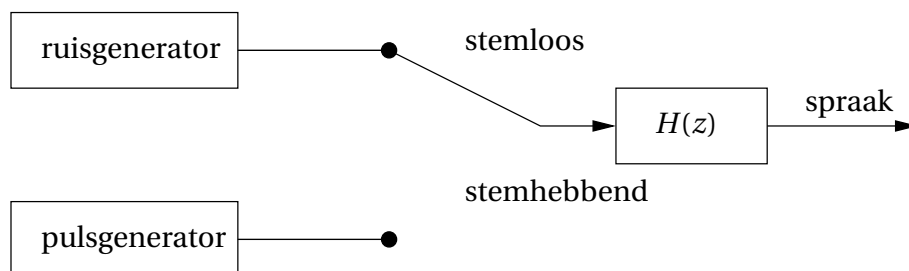
Zo trillen, bij de vorming van stemhebbende klank, de stembanden semi-periodisch en krijgt het signaal een zogenaamde *grondtoon*. Dit is de fundamentele periode van het geproduceerde geluid, die tegelijk overeenkomt met de laagste frequentie. Deze periode is

belangrijk aangezien hieruit kan bepaald worden welke klank (en bv. welke klinker) wordt uitgesproken. Daarnaast hebben stemhebbende klanken over het algemeen een lagere frequentie en hogere energie dan stemloze. Niettegenstaande bevatten stemhebbende fonemen duidelijker afgetekende frequenties (buiten de grondtoon) genaamd *formanten*. De grondtoon gecombineerd met de andere formanten maken de klank rijker en zijn een gevolg van de vorm van de delen in het stemkanaal (met name mond- en/of neusholte en keelholte), of voor muziek: een gevolg van het soort instrument dat de onderzochte klank voortbrengt.

Stemloze signalen daarentegen worden door het stemkanaal gestuurd zonder dat de stembanden hierbij trillen. Fluisteren is hier een voorbeeld van. Vandaar ook dat de grondtoon van het foneem in dit geval van geen belang is. Het onderscheid tussen verschillende fonemen (meestal medeklinkers) wordt deze keer verzorgd door de stand van o.a. tong en lippen. Een s-klank wordt bijvoorbeeld gevormd door de tong tegen het gehemelte te houden, een f-klank door de tanden tegen de lip te zetten [25].

3.3.2 Voorstelling via bron-filtermodel

De voorstelling van stemhebbende en stemloze fonemen wordt uitgevoerd door het zogenaamde *bron-filtermodel* [38]. Hierin wordt een onderscheid gemaakt, zoals de naam al doet vermoeden, tussen de bron van het geluid en het filter waar het wordt doorgestuurd. De voorstelling van het bron-filtermodel wordt afgebeeld in figuur 3.6 en wordt in de volgende twee paragrafen kort toegelicht.



Figuur 3.6: Voorstelling van het bron-filtermodel, hier voor een stemloos foneem.

Bron

Omwillen van het typeverschil in klank (zie 3.3.1) zijn er in het model twee soorten bronnen voorzien: een ruisgenerator voor stemloze fonemen en een pulsgenerator met de grondtoon als fundamentele periode van de pulsen voor stemhebbende fonemen. Tijdens de analyse van de door te zenden spraaksignalen moet dus door de zender vooraf een keuze worden gemaakt tussen stemhebbend of stemloos. Dit gebeurt o.a. door de spraaksignalen eerst te bemonsteren, daarna de data op te splitsen in kleinere blokken en tenslotte elk van deze blokken te onderzoeken op basis van de volgende drie bepalende factoren: de amplitude, het aantal keer dat het signaal de abscis kruist en de omliggende blokken spraakdata, d.i. de context van het beschouwde blok. Voor meer methodes om

een onderscheid te maken tussen stemhebbende en stemloze fonemen wordt verwezen naar [25]. Indien uiteindelijk een blok als stemhebbend wordt gekarakteriseerd, wordt hieruit nog de grondtoon berekend (voor meer informatie omtrent de manier waarop dit gebeurt, wordt de lezer doorverwezen naar [38]).

Filter

Achter de bron wordt een filter met alleen polen (autoregressief of AR filter) geplaatst, dat het equivalent voorstelt van alle delen van het stemkanaal, met name keel-, mond- en neusholte. Het volledige stemkanaal wordt verdeeld in afzonderlijke secties, waarbij het aantal secties een maat is voor het aantal filtercoëfficiënten van het modellerend filter. Hoe hoger de orde van dit filter (en dus hoe meer coëfficiënten worden berekend), hoe beter het model zal overeenkomen met de werkelijkheid, maar hoe complexer tegelijk de berekeningen worden en hoe meer data moet verstuurd worden door de zender per blok stemhebbend of stemloos foneem (zie 3.3.3).

3.3.3 LPC voor mono spraaksignalen

LPC of *lineaire predictieve codering* kan gezien worden als zuiver gebaseerd op het bron-filtermodel, waardoor ze verschilt van alle voorgaande coderingsschema's in 3.2 in de zin dat de doorgestuurde data geen onderdeel van de oorspronkelijke signalen meer is. Bij dergelijke compressie wordt namelijk niet de data zelf doorgestuurd (eventueel in getransformeerde vorm), maar net informatie over deze data. Deze informatie omvat de parameters die volgen uit het bron-filtermodel van 3.3.2 dat uit de originele spraaksignalen wordt gevonden.

Daartegenover kan lineaire predictieve codering ook als vorm van de in 3.2.4 vermelde differentiële codering gezien worden. Vanuit deze visie wordt de waarde van het huidige monster voorspeld door middel van lineaire predictie, nl. door een lineaire combinatie van vorige monsterwaarden. Daarna wordt de fout berekend tussen de huidige waarde en de voorspelling hiervan. Indien mits enige vereenvoudiging het stemkanaal door een equivalent filter $A(z)$ met eindige impulsresponsie wordt gemodelleerd, stelt deze fout in het bron-filtermodel niets anders voor dan het signaal dat tussen stembanden en stemkanaal wordt waargenomen (het residu). De notie dat $A(z)$ een eindige impulsresponsie heeft en dus zuiver transversaal is, is belangrijk aangezien hierdoor wordt aangenomen dat het geproduceerde geluid door ofwel neus- ofwel mondholt propageert en niet door beiden tegelijk. Het laatste geval zou men moeten modelleren door een filter met oneindige impulsresponsie, wat de equivalente voorstelling en de toekomstige berekeningen te complex zou maken.

Filtercoëfficiënten

Het berekenen van de filtercoëfficiënten voor de predictie van het huidige monster gebeurt als volgt. Stel M het aantal gebruikte vorige monsterwaarden, of nog:

$$\tilde{x}(n) = a_1 x(n-1) + a_2 x(n-2) + \dots + a_M x(n-M) = \sum_{i=1}^M a_i x(n-i), \quad (3.1)$$

met $\tilde{x}(n)$ de predictie van de huidige monsterwaarde, $x(n-i)$ de i -de voorgaande monsterwaarde en a_i de daarbij horende filtercoëfficiënt. De overeenkomstige voorwaartse predictiefout $e(n)$ wordt dan

$$e(n) = x(n) - \tilde{x}(n) = x(n) - \sum_{i=1}^M a_i x(n-i). \quad (3.2)$$

De voorwaarde waaronder de berekening van de filtercoëfficiënten gebeurt, houdt in dat het kwadraat van de predictiefout, nl.

$$\sum_n e^2(n) = \sum_n \left(x(n) - \sum_{i=1}^M a_i x(n-i) \right)^2, \quad (3.3)$$

wordt geminimaliseerd. Hiervoor biedt afleiden en gelijkstellen aan nul de oplossing:

$$2 \sum_n x(n-k) \left(x(n) - \sum_{i=1}^M a_i x(n-i) \right) = 0 \quad \text{voor } k = 1, 2, 3, \dots, M \quad (3.4)$$

Uitwerken van (3.4) geeft

$$\begin{aligned} a_1 \sum_n x(n-k)x(n-1) + a_2 \sum_n x(n-k)x(n-2) + \dots + a_M \sum_n x(n-k)x(n-M) \\ = \sum_n x(n-k)x(n) \quad \text{voor } k = 1, 2, 3, \dots, M \end{aligned} \quad (3.5)$$

Berekenen van de filtercoëfficiënten uit bovenstaande vergelijkingen is gedefinieerd als de covariantiemethode. Hierbij dient opgemerkt te worden dat in praktijk bovenstaande sommen genomen worden over een venster monsterwaarden met een eindige grootte. Indien in dit geval blokken ingangsmoesters met een voldoende klein venster worden gehanteerd, kan het ingangssignaal x stationair genomen worden, zodat (3.5) zich herleidt tot

$$\begin{bmatrix} r(0) & r(1) & \dots & r(M-2) & r(M-1) \\ r(1) & r(0) & \dots & r(M-3) & r(M-2) \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ r(M-2) & r(M-3) & \dots & r(0) & r(1) \\ r(M-1) & r(M-2) & \dots & r(1) & r(0) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{M-1} \\ a_M \end{bmatrix} = \begin{bmatrix} r(1) \\ r(2) \\ \vdots \\ r(M-1) \\ r(M) \end{bmatrix} \quad (3.6)$$

met

$$r(k) = \sum_{n=0}^{N-1-k} x(n)x(n+k). \quad (3.7)$$

Bovenstaande termen zijn niets anders dan de autocorrelaties van het ingangssignaal. Om die reden noemt men deze vorm dan ook de autocorrelatiemethode, in tegenstelling tot de covariantiemethode in (3.5). Omwille van de speciale Toeplitz structuur van het stelsel in (3.6) kan de oplossing efficiënt worden gevonden via het recursief Levinson-Durbin [30] algoritme (zie tabel (3.1)). Hierbij is E_i de som van de kwadraten van de predictiefouten overeenstemmend met de i -de predictieorde, a_{ji} is de j -de predictiecoëfficiënt van de i -de orde predictie en de κ 's zijn gedefinieerd als reflectiecoëfficiënten.

Eens hieruit de predictiecoëfficiënten bekend zijn, volgt uit vergelijking (3.2) de predictiefout. Door de benadering $\hat{x}$ van het originele signaal x af te trekken, verdwijnt een groot deel van de correlatie tussen opeenvolgende monsterwaarden en verkleint ook het bereik van het overgebleven signaal. Als gevolg daarvan kan het aantal bit nodig voor codering aanzienlijk worden verminderd.

Levinson-Durbin

Initialisatie:

$$E_0 = r(0)$$

$$a_{11} = \kappa_1 = r(1)/E_0$$

$$E_1 = E_0(1 - \kappa_1^2)$$

$$m = 2$$

Itereer over:

- $q_m = r(m) - \sum_{i=1}^{m-1} a_{i(m-1)} r(m-i)$
- $\kappa_m = \frac{q_m}{E_{m-1}}$
- $a_{mm} = \kappa_m$
- $a_{im} = a_{i(m-1)} - \kappa_m a_{(m-i)(m-1)} \quad \text{voor } i = 1, \dots, m-1$
- $E_m = E_{m-1}(1 - \kappa_m^2)$
- Als $m < M$, doe dan $m = m + 1$. Als $m = M$, stop.

Tabel 3.1: Recursief Levinson-Durbin algoritme voor monofone spraak.

Het resulterend coderingsschema is bijgevolg opgebouwd uit een analysefilter met overdrachtsfunctie

$$A(z) = 1 - \sum_{i=1}^M a_i z^{-i} \quad (3.8)$$

Hiermee wordt de ingang x gefilterd. De uitgang van dit filter wordt eventueel gekwantiseerd en nadien doorgestuurd over het kanaal. De ontvanger voert zo nodig een dekwantisatie uit en filtert het resultaat via een synthesefilter met overdrachtsfunctie $\frac{1}{A(z)}$, met $A(z)$ gedefinieerd in (3.8).

Roosterimplementatie

De voorstelling van differentiële codering (met lineaire predictie) kan ook door een equivalent roosterschema worden vervangen. Om tot dit schema te komen, wordt begonnen van twee soorten fouten: de voorwaartse predictiefout $e(n)$, vanaf hier genoteerd als $f(n)$ en de achterwaartse predictiefout $b(n)$, beiden van orde m . Indien, naar analogie met de eerste, de achterwaartse predictiefout het verschil is tussen $x(n-m)$ en de predictie hiervan door een lineaire combinatie van $x(n), x(n-1), \dots, x(n-m+1)$, zijn beide fouten

als volgt bepaald:

$$\begin{aligned} f_m(n) &= x(n) - \sum_{i=1}^m a_{im} x(n-i) \\ b_m(n) &= x(n-m) - \sum_{i=1}^m a_{im} x(n-m+i) \end{aligned} \quad (3.9)$$

Daarnaast zijn de predictiefouten van orde $m-1$ gegeven door

$$\begin{aligned} f_{m-1}(n) &= x(n) - \sum_{i=1}^{m-1} a_{i(m-1)} x(n-i) \\ b_{m-1}(n) &= x(n-m+1) - \sum_{i=1}^{m-1} a_{i(m-1)} x(n-m+1+i) \end{aligned} \quad (3.10)$$

Vervolgens kan, uit de recursieve formule voor de predictiecoëfficiënten

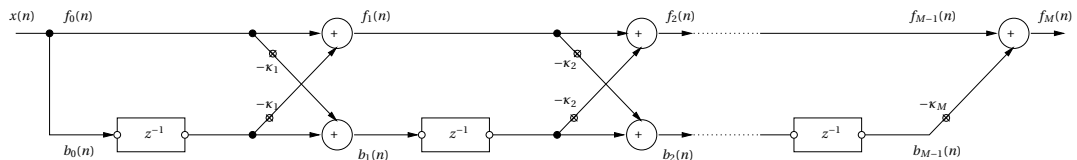
$$\begin{aligned} a_{im} &= a_{i(m-1)} - \kappa_m a_{(m-i)(m-1)} \quad \text{voor } i = 1, 2, \dots, m-1 \\ a_{mm} &= \kappa_m, \end{aligned} \quad (3.11)$$

het bestaan van volgende vergelijkingen aangetoond worden:

$$f_m(n) = f_{m-1}(n) - \kappa_m b_{m-1}(n-1) \quad (3.12)$$

$$b_m(n) = b_{m-1}(n-1) - \kappa_m f_{m-1}(n) \quad (3.13)$$

voor $m = 1, 2, \dots, M$. Het bewijs hiervoor is terug te vinden in [4]. Het roosterschema dat vergelijkingen (3.12) en (3.13) implementeert en tegelijk het stemkanaal als aaneenschakeling van orde M verschillende basiscellen modelleert wordt weergegeven in figuur 3.7. Er dient wel opgemerkt te worden dat dit het equivalent rooster is dat de analyse van het



Figuur 3.7: Analysefilter uit vergelijking (3.8) als equivalent voor het stemkanaal: implementatie als rooster.

ingangssignaal uitvoert. De ingang komt daarom overeen met het te ontleden spraaksignaal, dat na een volledige doorgang herleid wordt tot de voorwaartse predictiefout verkregen na lineaire predictie met orde M . De initiële ingangswaarden zijn

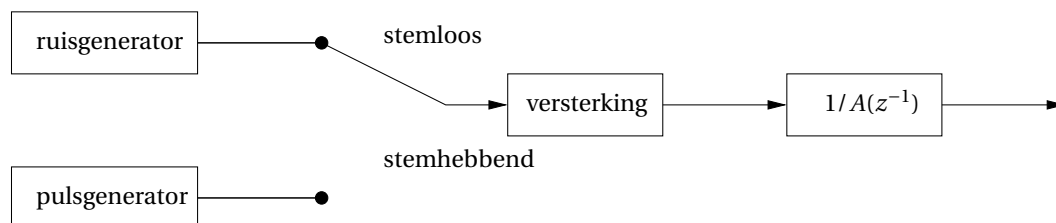
$$f_0(n) = b_0(n) = x(n) \quad (3.14)$$

en de uiteindelijke uitgangswaarde

$$f(n) = f_M(n). \quad (3.15)$$

Transmissie en synthese

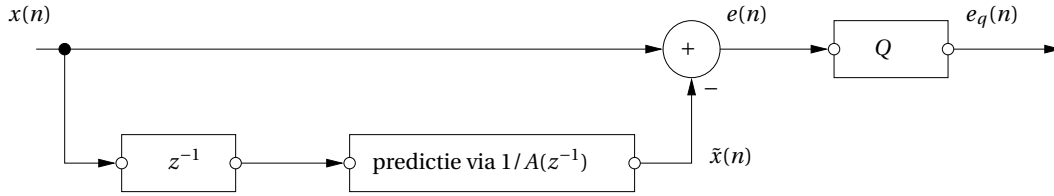
In 3.3.3 werd al een mogelijke codering met lineaire predictie kort ingeleid. Het gaat hier over de realisatie waarbij geen enkel datasymbool wordt verzonden, alleen de parameters die deze data omschrijven. De parameters die hier bedoeld worden zijn: de grondtoon, de versterkingsfactor en de berekende filtercoëfficiënten. Zie deel 3.3.2 voor de betekenis van deze termen. De versterkingsfactor, hiervoor nog niet vermeld, is niets anders dan dat: een versterking van het bronsignaal om de gesynthetiseerde spraak op het juiste amplitudenniveau te krijgen. De waarde ervan wordt bepaald door de vierkantswortel te nemen van het gemiddelde van de som van de kwadraten van de signaalwaarden binnen één venster (of nog, de *rms*-waarde van een venster signaalwaarden). De manier waarop fonemen met dergelijke codering worden gesynthetiseerd is te zien in figuur 3.8. Meer specifiek wordt er voor de standaard FS-1015 [39] 1 bit doorgestuurd om aan te



Figuur 3.8: Synthesefilter gebaseerd op volledig bron-filtermodel.

geven of het te vormen spraakfoneem stemhebbend of stemloos is en dus al dan niet een grondtoon heeft. Indien stemhebbend, wordt er 6 bit gebruikt voor de grondtoon, terwijl een 10de orde filter wordt voorzien. Als het foneem stemloos is, is slechts een 4de orde filter nodig. Het totaal aantal waarden dat nog moet verstuurd worden is dus gelijk aan respectievelijk 11 en 5 (de orde van het filter en de versterkingsfactor).

Een andere optie houdt in dat niet de volledige data wordt gekarakteriseerd door parameters, maar slechts tot op de residu's na. Zo wordt het stemkanaal nog voorgesteld door zijn filtercoëfficiënten, maar alles wat zich ervoor bevindt (nl. het residu: het bronsignaal en de versterkingsfactor) wordt op zich verstuurd. Een eenvoudige vergelijking met formule (3.2) toont hier meteen aan wat al in de inleiding verondersteld werd, nl. dat de overblijvende residu's dezelfde zijn als de predictiefouten bij differentiële codering. Bij dit type codering is er nog steeds een (weliswaar kleinere) reductie van het aantal nodige transmissiebits en is de reconstructie van het spraaksignaal tegelijk van betere kwaliteit. Als de transmissie verliesloos gebeurt, is er zelfs perfecte reconstructie mogelijk. Het bijhorende coderingsschema wordt weergegeven in figuur 3.9. Uit het principe van de Wilkinson-veelterm ten slotte [40] is geweten dat kleine perturbaties op de coëfficiënten van een veelterm grote verschillen in de posities van de nulpunten kunnen teweegbrengen. Aangezien bij het verzenden van de filtercoëfficiënten van het stemkanaal kleine perturbaties bijna niet weg te denken zijn in een niet-ideaal transmissiekanaal, zullen de polen van het equivalent filter sterk verschuiven als gevolg. De eis voor een juiste reconstructie en een correcte werking van het coderingsschema houdt in dat de polen van het filter stabiel moeten zijn (binnen de eenheidscirkel moeten liggen), ook na doorgang



Figuur 3.9: Coderingsschema van differentiële codering met lineaire predictie gebaseerd op bron-filtermodel.

door het transmissiekanaal. Omwille van die reden en omdat het filter ook gevoelig is voor perturbaties op reflectiecoëfficiënten met absolute waarde dicht bij 1, worden de predictiecoëfficiënten vervangen door de overeenstemmende reflectiecoëfficiënten en worden deze laatsten getransformeerd tot bv. *LAR*'s, de zogenaamde *log-area ratios* of log-oppervlakte verhoudingen [30]. De algemene vorm wordt gegeven door (3.16)

$$LAR_m = \log \left(\frac{1 - \kappa_m}{1 + \kappa_m} \right). \quad (3.16)$$

De naam *LAR* vindt zijn oorsprong in het feit dat $1 - \kappa_m$ en $1 + \kappa_m$ de oppervlakten A_m en A_{m-1} voorstellen, met A_i de oppervlakte van sectie i in het stemkanaal (cfr. hoofdstuk 3.3.2, deel over filter). Deze transformatie uitvoeren vóór een lineaire kwantisatie van de resultaten leidt in het geheel tot een niet-lineaire kwantisatie, waardoor perturbaties op κ 's met grote absolute waarden (dicht bij 1) minder invloed hebben op de kwaliteit van de reconstructie en er zo dus een grotere robuustheid wordt bekomen.

3.4 Stereo lineaire predictie

Zoals in voorgaand deel duidelijk gemaakt baseert lineaire predictie zich op het lineair verband dat er bestaat tussen opeenvolgende monsterwaarden, hetgeen ook terugkomt in de modellering van spraak via het bron-filtermodel (zie hoofdstuk 3.3.2). Er kan dus ingezien worden dat lineaire predictie de best bekende methode is om correlatie uit een spraaksignaal te halen, juist omwille van de sterke gelijkenis tussen de generatie van spraak door de stembanden en het stemkanaal en deze vorm van predictie.

Voor mono spraak was dit positief in de zin dat als er veel correlatie tussen verschillende bemonsterde waarden kan verwijderd worden, het resulterende foutsignaal heel klein kan worden gemaakt. Dit garandeert een sterke reductie in het aantal gebruikte transmissiebits en vergroot zo de efficiëntie van het coderingsschema via een grotere compressie van het originele spraaksignaal.

In het geval van stereo spraak heeft deze methode nog een bijkomend voordeel: door ook het verminderen van de kruiscorrelaties tussen de twee stereo kanalen mogelijk te maken (en zo dus een inter- en intrakanaaldecorrelatie uit te voeren) kan de codering evenzeer gebruikt worden als een decorrelatiefilter voor akoestische echo-onderdrukking van stereo signalen (zie hoofdstuk 2). Vervolgens leidt het feit dat voor deze beide onderdelen van spraakverwerking hetzelfde coderingsschema kan toegepast worden tot het

idee van integratie. Dit principe en de resultaten ervan worden uitvoerig besproken in hoofdstuk 4.

Het huidige hoofdstuk daarentegen handelt over lineaire predictieve codering zoals besproken in deel 3.3, alleen hier toegepast op het geval met twee verschillende kanalen. Het principe van monofone LPC wordt hier dus uitgebreid, zó dat de methode niet alleen voor spraak, maar ook meer algemeen voor audiosignalen kan toegepast worden. Eerst en vooral worden de nodige theoretische afleidingen i.v.m. stereofone LPC besproken, zoals ze ook zijn uitgezet in [41] en dezelfde structuur zoals in sectie 3.3 wordt hierbij gehandhaafd. Daarna wordt aandacht besteed aan enkele resultaten die bekomen zijn door onze eigen implementatie en uitvoering. Ten slotte worden dan de belangrijke coderingsparameters gevarieerd en de invloed hiervan op de coderingsperformantie in kaart gebracht.

3.4.1 Basis: uitbreiding van mono lineaire predictie

Vermits, zoals hierboven vermeld, er gestreefd wordt naar een volledige eliminatie van zowel auto- (binnen hetzelfde kanaal) als kruiscorrelaties (tussen de twee kanalen), is codering na uitbreiding naar twee kanalen niet volledig analoog aan de monofone situatie [42]. Bij stereo codering is er namelijk sprake van vier verschillende predictieveeltermen, twee voor elk kanaal. Om die reden worden monsters van het eerste en tweede kanaal op de volgende manier geschat:

$$\tilde{x}_1(n) = \sum_{i=1}^{K_a} a_i x_1(n-i) + \sum_{i=1}^{K_b} b_i x_2(n-i) \quad (3.17)$$

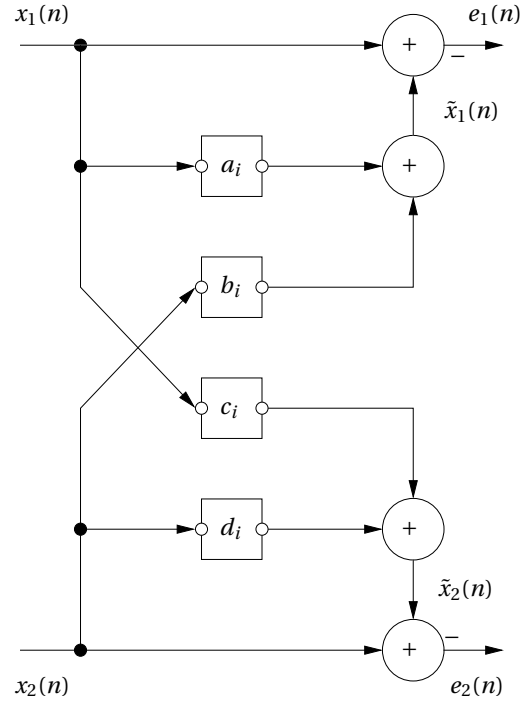
$$\tilde{x}_2(n) = \sum_{i=0}^{K_c} c_i x_1(n-i) + \sum_{i=1}^{K_d} d_i x_2(n-i). \quad (3.18)$$

Hierbij zijn K_a, K_c en K_b, K_d de predictieordes voor respectievelijk de auto- en kruiscorrelatieveeltermen van beide kanalen. Ze kunnen verschillend genomen worden om bv. één kanaal meer invloed op het uiteindelijke resultaat te geven. Een nadeel hierbij is dat moet rekening gehouden worden met de stabiliteit van de veeltermen en de bijhorende reflectiecoëfficiënten. Wanneer deze in absolute waarde groter zijn dan 1, zal de reconstructie bij decoding aan de ontvangzijde niet meer stabiel kunnen gebeuren en is ze bijgevolg niet meer correct. Een wat diepergaande bespreking van de stabiliteit van de predictieveeltermen en de optimale keuze voor de verschillende predictieordes volgt later in dit hoofdstuk.

Daarnaast dient opgemerkt te worden dat sommige bronnen werken met een asymmetrische en anderen met een symmetrische predictie. Het verschil zit in het gebruik van het eerste monster als bijkomende term voor de schatting van het tweede kanaal in (3.18). In [41] wordt geopteerd voor de symmetrische aanpak, aangezien in dat geval het stelsel vergelijkingen waaruit de optimale oplossing van de predictiecoëfficiënten volgt een blok-Toeplitz structuur krijgt. Zo kan het efficiënte, recursieve Levinson-Durbin algoritme ook toegepast worden voor predictie van stereo audio en spraak. In het proefschrift van Biswas wordt daarnaast nog een bewijs gegeven voor de stabiliteit van de berekeningen volgens de Levinson-Durbin methode die gepaard gaat met de keuze van dergelijk symmetrisch

schema. Dit bewijs wordt hier niet herhaald, maar de geïnteresseerde lezer kan zich hiervoor steeds wenden tot [41, deel 2.3.5].

Omwille van de eenvoud en de aangetoonde stabiliteit van een symmetrisch coderingsschema met gelijke predictieorden voor de verschillende veeltermen is hier gekozen voor de verdere ontwikkeling van dit schema in tegenstelling tot één met ongelijke predictieorden. De algemene voorstelling van het analysefilter van zo'n systeem wordt getoond in figuur 3.10. Vooraleer over te gaan naar de eigenlijke berekening van de coëfficiënten,



Figuur 3.10: Analysefilter van een systeem met stereo lineaire predictie.

eerst nog enkele definities ter verduidelijking. Het tweekanaals ingangssignaal wordt, net zoals in sectie 3.3 over mono spraak, voorgesteld door

$$\mathbf{x}(n) = \begin{bmatrix} x_1(n) & x_2(n) \end{bmatrix}. \quad (3.19)$$

De voorspelling van deze ingang op basis van K voorgaande ingangsmonsters wordt hier gedefinieerd als

$$\tilde{\mathbf{x}}(n) = \sum_{i=1}^K \mathbf{x}(n-i) \mathbf{A}_{K,i}, \quad (3.20)$$

met

$$\mathbf{A}_{K,i} = \begin{bmatrix} a_i & c_i \\ b_i & d_i \end{bmatrix} \quad (3.21)$$

De fout $\mathbf{e}(n)$ die wordt verkregen door, analoog aan vergelijking (3.2), het verschil te nemen tussen de ingang en de voorspelling ervan via lineaire predictie, wordt als volgt

gevonden:

$$\mathbf{e}(n) = \mathbf{x}(n) - \tilde{\mathbf{x}}(n) = \mathbf{x}(n) - \sum_{i=1}^K \mathbf{x}(n-i) \mathbf{A}_{K,i}. \quad (3.22)$$

Gelijkaardig als voor het monofone geval wordt qua notatie in de afleiding $\mathbf{e}(n)$ gebruikt om de voorwaartse predictiefouten aan te geven, later aangeduid met $\mathbf{f}(n)$ om het contrast met de achterwaartse predictiefouten $\mathbf{vect}b(n)$ aan te geven. De voorwaarde waarop gesteund wordt om de optimale filtercoëfficiënten te berekenen is net dezelfde als voor het monofone geval, nl. het minimaliseren van de energie in bovenstaande predictiefout. De energie wordt gegeven door het kwadraat van het te beschouwen signaal, nl.

$$\begin{aligned} \sum_n e_1^2(n) &= \sum_n [x_1(n) - \tilde{x}_1(n)]^2 \\ &= \sum_n \left[x_1(n) - \sum_{i=1}^K x_1(n-i) a_i - \sum_{i=1}^K x_2(n-i) b_i \right]^2 \end{aligned} \quad (3.23)$$

$$\begin{aligned} \sum_n e_2^2(n) &= \sum_n [x_2(n) - \tilde{x}_2(n)]^2 \\ &= \sum_n \left[x_2(n) - \sum_{i=1}^K x_1(n-i) c_i - \sum_{i=1}^K x_2(n-i) d_i \right]^2. \end{aligned} \quad (3.24)$$

Minimalisatie wordt opnieuw uitgevoerd door de afgeleide van de vergelijkingen gelijk te stellen aan nul, met name

$$\frac{\partial \sum_n e_1^2(n)}{\partial a_l} = 0, \quad l = 1, 2, \dots, K, \quad (3.25)$$

$$\frac{\partial \sum_n e_1^2(n)}{\partial b_m} = 0, \quad m = 1, 2, \dots, K. \quad (3.26)$$

De verdere uitwerking wordt hieronder uitgevoerd voor de eerste vergelijking. Invullen van (3.23) in (3.25) geeft:

$$\begin{aligned} \frac{\partial \sum_n e_1^2(n)}{\partial a_l} &= \frac{\partial \left\{ \sum_n \left[x_1(n) - \sum_{i=1}^K x_1(n-i) a_i - \sum_{i=1}^K x_2(n-i) b_i \right]^2 \right\}}{\partial a_l} \\ &= \sum_n \left[x_1(n-l) 2 \left(x_1(n) - \sum_{i=1}^K x_1(n-i) a_i - \sum_{i=1}^K x_2(n-i) b_i \right) \right] \\ &= 0 \end{aligned} \quad (3.27)$$

In uitgewerkte vorm wordt dit:

$$\begin{aligned} &\sum_n \left[\sum_{i=1}^K x_1(n-l) x_1(n-i) a_i + \sum_{i=1}^K x_1(n-l) x_2(n-i) b_i \right] \\ &= \sum_n [x_1(n-l) x_1(n)], \end{aligned} \quad (3.28)$$

of bij het wisselen van de sommatietekens

$$\begin{aligned} & \sum_{i=1}^K \left[\sum_n x_1(n-l)x_1(n-i)a_i \right] + \sum_{i=1}^K \left[\sum_n x_1(n-l)x_2(n-i)b_i \right] \\ &= \sum_n [x_1(n-l)x_1(n)]. \end{aligned} \quad (3.29)$$

Herinner uit sectie 3.3 dat de correlatie tussen twee signalen x_p en x_q kan voorgesteld worden door

$$r_{pq}(k) = \sum_n x_p(n)x_q(n-k), \quad (3.30)$$

met p en $q = 1, 2$ en de som die loopt over alle waarden n van het signaal x . Vergelijking (3.30) inbrengen in (3.29) resulteert in

$$\sum_{i=1}^K r_{11}(l-i)a_i + \sum_{i=1}^K r_{12}(l-i)b_i = r_{11}(l) \quad \text{voor } l = 1, 2, \dots, K. \quad (3.31)$$

Analoog aan (3.31) wordt uit (3.26) gevonden dat

$$\sum_{i=1}^K r_{21}(m-i)a_i + \sum_{i=1}^K r_{22}(m-i)b_i = r_{21}(m) \quad \text{voor } m = 1, 2, \dots, K. \quad (3.32)$$

Indien nu dezelfde berekeningen (3.25) - (3.31) uitgevoerd worden voor de predictiefouten op schattingen van waarden $x_2(n)$ van het tweede kanaal, wordt een vergelijkbaar stelsel uitgekomen. De minimalisatievoorwaarden van waaruit dit keer wordt gestart, zijn

$$\frac{\partial \sum_n e_2^2(n)}{\partial c_l} = 0, \quad l = 1, 2, \dots, K, \quad (3.33)$$

$$\frac{\partial \sum_n e_2^2(n)}{\partial d_m} = 0, \quad m = 1, 2, \dots, K.. \quad (3.34)$$

Uiteindelijk geeft dit:

$$\sum_{i=1}^K r_{11}(l-i)c_i + \sum_{i=1}^K r_{12}(l-i)d_i = r_{12}(l) \quad \text{voor } l = 1, 2, \dots, K, \quad (3.35)$$

$$\sum_{i=1}^K r_{21}(m-i)c_i + \sum_{i=1}^K r_{22}(m-i)d_i = r_{22}(m) \quad \text{voor } m = 1, 2, \dots, K. \quad (3.36)$$

Vergelijkingen (3.31), (3.32), (3.35) en (3.36) vormen samen het stelsel van Yule-Walker

vergelijkingen, wat in matrixvorm als volgt kan worden genoteerd:

$$\begin{bmatrix} r_{11}(0) & \dots & r_{11}(1-K) & r_{12}(0) & \dots & r_{12}(1-K) \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ r_{11}(K-1) & \dots & r_{11}(0) & r_{12}(K-1) & \dots & r_{12}(0) \\ r_{21}(0) & \dots & r_{21}(1-K) & r_{22}(0) & \dots & r_{22}(1-K) \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ r_{21}(K-1) & \dots & r_{21}(0) & r_{22}(K-1) & \dots & r_{22}(0) \end{bmatrix} \times \begin{bmatrix} a_1 & c_1 \\ \vdots & \vdots \\ a_K & c_K \\ b_1 & d_1 \\ \vdots & \vdots \\ b_K & d_K \end{bmatrix} = \begin{bmatrix} r_{11}(1) & r_{12}(1) \\ \vdots & \vdots \\ r_{11}(K) & r_{12}(K) \\ r_{21}(1) & r_{22}(1) \\ \vdots & \vdots \\ r_{21}(K) & r_{22}(K) \end{bmatrix} \quad (3.37)$$

Bij het vergelijken van (3.6) met (3.37) zijn duidelijke overeenkomsten waar te nemen. Terwijl (3.6) in vorige sectie recursief werd opgelost via het Levinson-Durbin algoritme, bestaat er voor de berekening van de auto- en kruiscorrelaties in (3.37) een analoge manier. Het principe van de recursieve methode voor monofone spraak blijft behouden, mits de aanpassing dat de voorkomende signalen nu in matrixvorm neergeschreven worden omwille van de twee kanalen en de vier verschillende correlatieveeltermen. Om het zogenaamde blok-Levinson algoritme hier te kunnen toepassen, moet de correlatiematrix net zoals in sectie 3.3 een Toeplitz structuur hebben. Daarom volgende definitie van C_i :

$$C_i = \begin{bmatrix} r_{11}(i) & r_{12}(i) \\ r_{21}(i) & r_{22}(i) \end{bmatrix}. \quad (3.38)$$

Gebruik makend van de symmetrische eigenschap van de kruiscorrelatie, nl. dat $r_{pq}(k) = r_{qp}(-k)$, kan (3.37) herschreven worden tot

$$\Gamma_{K-1} A_K = P, \quad (3.39)$$

waarbij

$$\Gamma_{K-1} = \begin{bmatrix} C_0 & C_{-1} & C_{-2} & \dots & C_{-K+1} \\ C_1 & C_0 & C_{-1} & \dots & C_{-K+2} \\ C_2 & C_1 & C_0 & \dots & C_{-K+3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ C_{K-1} & C_{K-2} & C_{K-3} & \dots & C_0 \end{bmatrix} \quad (3.40)$$

de coëfficiëntencorrelatiematrix in blok-Toeplitz vorm voorstelt en

$$A_K = [A_{K,1} \ A_{K,2} \ A_{K,3} \ \dots \ A_{K,K}]^T \quad (3.41)$$

en

$$P = [C_1 \ C_2 \ C_3 \ \dots \ C_K]^T \quad (3.42)$$

respectievelijk de matrix met predictieveltermen zoals bepaald in (3.21) en de correlatiematrix van de bekende termen uit het rechterlid. Het stelsel in (3.39) kan nu worden opgelost door het blok-Levinson algoritme dat gegeven is in tabel 3.2. Hierin is $A_j^{(M)}$ de predictiematrix A van orde j en iteratie M en P_j de j -de deelmatrix C_j van P , beiden zoals hierboven bepaald. Matrix C_j zelf is gegeven door vergelijking (3.38). De matrices $E_j^{(M)}$ en $E_j'^{(M)}$ ten slotte zijn respectievelijk de voorwaartse en achterwaartse reflectiematrices van orde j en iteratie M .

Blok-Levinson

Initialisatie:

$$A_1^{(1)} = [C_0]^{-1} P_1$$

$$E_1^{(1)} = [C_0]^{-1} C_{-1}$$

$$E_1'^{(1)} = [C_0]^{-1} C_1$$

Itereer over:

- $E_{M+1}'^{(M+1)} = \left[\sum_{j=1}^M C_{M+1-j} E_{M+1-j}^{(M)} - C_0 \right]^{-1} \left[\sum_{j=1}^M C_{M+1-j} E_j'^{(M)} - C_{M+1} \right]$
- $E_{M+1}^{(M+1)} = \left[\sum_{j=1}^M C_{j-M-1} E_{M+1-j}^{(M)} - C_0 \right]^{-1} \left[\sum_{j=1}^M C_{j-M-1} E_j^{(M)} - C_{-M-1} \right]$
- $E_j^{(M+1)} = E_j^{(M)} - E_{M+1-j}^{(M)} E_{M+1}^{(M+1)}$ voor $j = 1, 2, \dots, M$.
- $E_j'^{(M+1)} = E_j'^{(M)} - E_{M+1-j}^{(M)} E_{M+1}'^{(M+1)}$ voor $j = 1, 2, \dots, M$.
- $A_{M+1}^{(M+1)} = \left[\sum_{j=1}^M C_{M+1-j} E_{M+1-j}^{(M)} - C_0 \right]^{-1} \left[\sum_{j=1}^M C_{M+1-j} A_j^{(M)} - P_{M+1} \right]$
- $A_{M+1-j}^{(M+1)} = A_{M+1-j}^{(M)} - E_j^{(M)} A_{M+1}^{(M+1)}$ voor $j = 1, 2, \dots, M$.
- Zolang de gewenste predictieorde M niet bereikt is, herbegin.

Tabel 3.2: Recursief blok-Levinson algoritme voor stereofone audio en spraak.

Nadat uiteindelijk alle predictiecoëfficiënten gevonden zijn, kan het analysefilter opgebouwd worden door de ingang gegeven in (3.19) te vermenigvuldigen met de matrix van overdrachtsfuncties $H(z)$. Hierbij is

$$H(z) = \begin{bmatrix} 1 - A(z) & -C(z) \\ -B(z) & 1 - D(z) \end{bmatrix} \quad (3.43)$$

waarbij de afzonderlijke veeltermen de vorm hebben van $A(z)$ met

$$A(z) = \sum_{i=1}^K z^{-i} a_i. \quad (3.44)$$

Analoog aan hoofdstuk 3.3.3 kan ook bovenstaande methode toegepast worden in een differentieel coderingsschema gebaseerd op (weliswaar stereo) lineaire predictie. Het

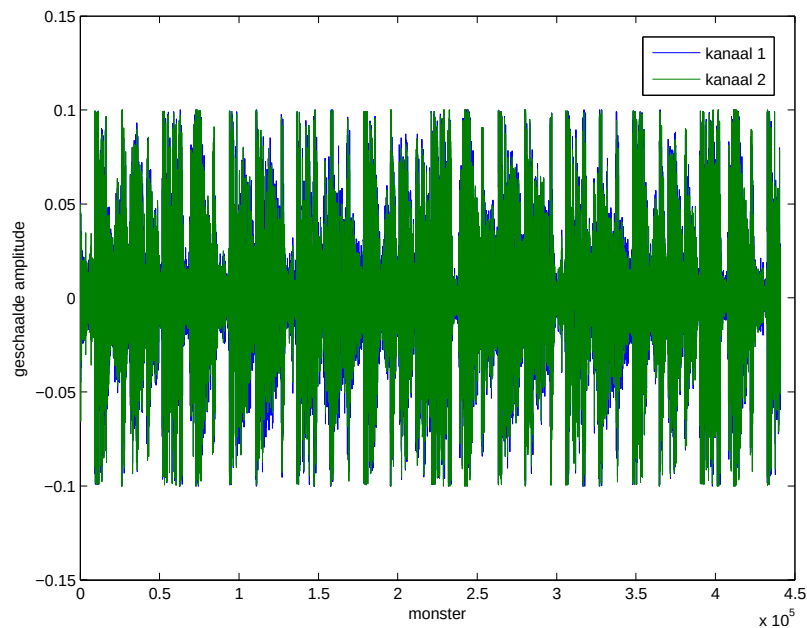
bijhorend analysefilter dat de twee ingangssignalen omzet naar residu's was al gegeven in figuur 3.10. De overblijvende predictiefouten $e_1(n)$ en $e_2(n)$ kunnen, net zoals in vorig hoofdstuk, ofwel op zich gecodeerd en verstuurd worden, ofwel nog verder ontleed worden tot hun basisfactoren. Aangezien in wat volgt geopteerd is voor de implementatie van de eerste mogelijkheid - omdat deze meer flexibiliteit biedt voor de integratie van LPC en SAEO in hoofdstuk 2 - wordt ook hier alleen nog aandacht aan besteedt.

3.4.2 Eigen implementatie

Gebaseerd op voorgaande theorie over stereo lineaire predictie bespreekt dit deel de rekenkundige en grafische resultaten bekomen uit simulaties van een zelf geschreven LP coderingsmethode. De resultaten hier besproken volgen uit tests uitgevoerd op steeds hetzelfde ingangssignaal, nl. [32]. Vergelijkbare simulaties zijn ook op andere spraak- en audiobestanden uitgevoerd en leiden tot soortgelijke conclusies.

In volgende onderdelen wordt de invloed van verschillende factoren op de performantie van het systeem getest. Met performantie worden hier meerdere afzonderlijke begrippen bedoeld. Zo bepaalt eerst en vooral de energie in het residu de mogelijke grootte van de compressiefactor bij codering, aangezien een kleinere energie een kleiner aantal coderingsbits toelaat bij een identieke geluidskwaliteit. Vervolgens moet ook aandacht worden besteed aan de *witheid* van deze residu's om zo een grotere decorrelatie van de ingang te verkrijgen, wat op zijn beurt een positief effect heeft op de convergentiesnelheid van een SAEO-systeem indien stereo LPC hierbij als decorrelatiemethode wordt gebruikt (zie hoofdstuk 4). Tot slot speelt de reconstructiefout tijdens het decoderen in de ontvanger een principiële rol, hoewel deze door het gebruik van het blok-Levinson algoritme (tabel 3.2 in sectie 3.4.1) een minimaal negatief effect oplevert. Om bovenstaande redenen wordt hieronder aan elk van de onderzochte parameters voldoende aandacht besteed, waarin de gevolgen van variatie in deze parameterwaarden duidelijk worden uiteengezet.

Vooraleer over te gaan tot een onderzoek van de implicaties van variaties in de vooraanstaande coderingsparameters, geeft dit inleidend gedeelte een illustratie van de correcte werking van het coderingsschema gebaseerd op de theorie uit het proefschrift van Biswas. Het eerder vernoemde audiobestand is hier als ingang gebruikt en een gedeelte ervan (van 0:20 tot 0:30 seconden, 10 seconden lang) zal worden verwerkt door het stereo analysefilter en wordt in figuur 3.11 grafisch getoond. Het ingangssignaal is geschaald in amplitude zodat de hoogste waarde gelijk is aan 0.1 en deze amplitude is uitgezet t.o.v. de monsters op de horizontale as. Het signaal is bemonsterd aan 44100 Hz, zodat 441000 monsters overeenkomen met exact 10 seconden geluid. Er zijn twee verschillende kleuren waar te nemen, blauw en groen, dewelke de twee kanalen van het stereo signaal voorstellen. Zo komt blauw overeen met het eerste kanaal, het tweede kanaal (groen) is er overheen getekend. De gebruikte predictieorde in deze simulatie is 10; de reden hiervoor zal later duidelijk worden. Het venster waarin telkens een nieuwe predictie wordt berekend is 882 monsterwaarden lang, zodat 20 ms geluid hier net in past, en er is ten slotte 50% overlapping tussen opeenvolgende vensters.



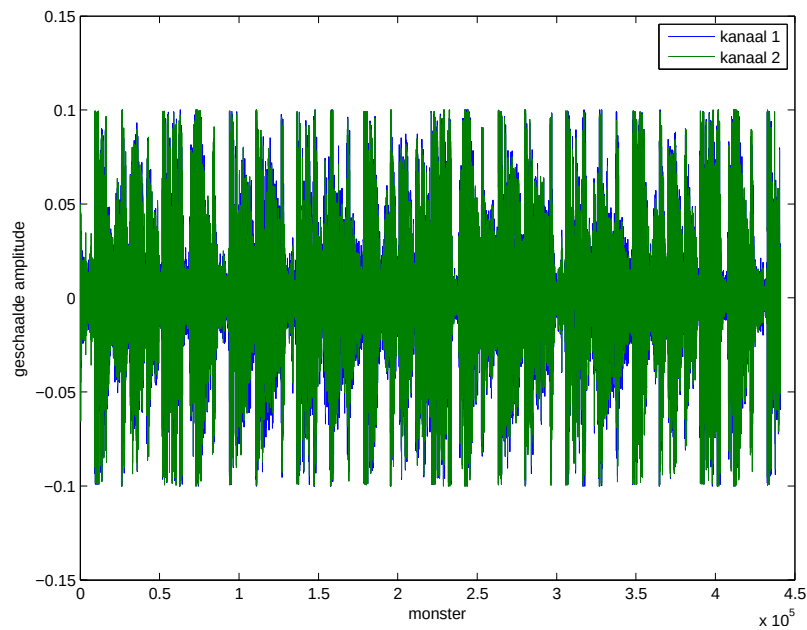
Figuur 3.11: Originele stereofone ingangssignalen met de twee kanalen in verschillende kleur gegeven.

Reconstructiefout

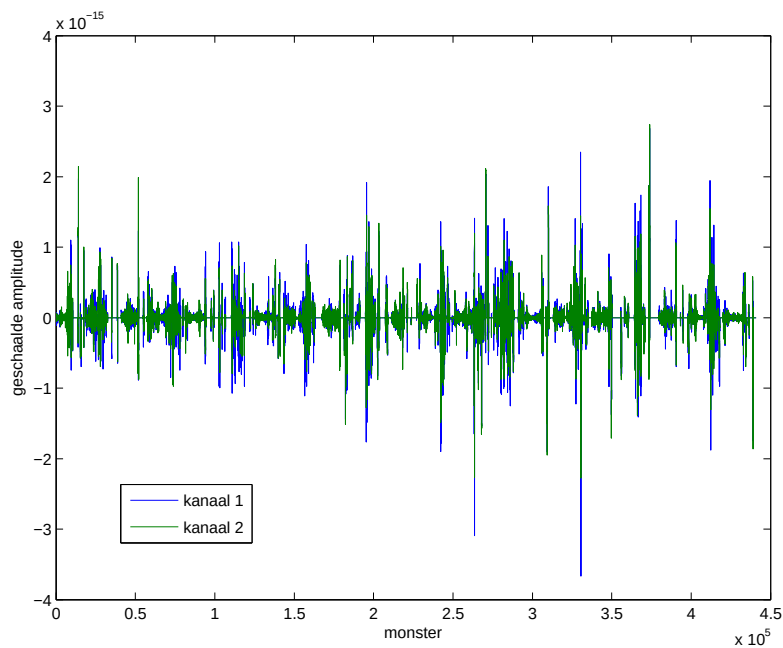
Een volgende grafiek, figuur 3.12, geeft het gereconstrueerde signaal aan de ontvangzijde weer. Op het eerste zicht is er geen merkbaar onderscheid tussen deze en vorige figuur, hetgeen duidt op een juiste werking van het geïmplementeerde coderingsschema. Indien beide signalen nauwkeuriger worden onderzocht, is er niettemin toch een verschil waar te nemen, hoewel uitermate klein (figuur 3.13). De reden hiervoor is eenvoudig: tijdens de berekeningen van analyse en synthese werkt MATLAB met een inwendige nauwkeurigheid van ongeveer 16 cijfers na de komma, zodat een fout van orde 10^{-15} niet te vermijden is. Aangezien dit het gevolg is van een eindige nauwkeurigheid waarin de berekeningen uitgevoerd worden, kan worden aangenomen dat analyse en synthese nagenoeg perfect werken.

Residu

Naast de reconstructiefout is het belangrijkste aspect aan een coderingsschema de waarde van het residu dat doorgestuurd wordt over het transmissiekanaal. Om in het oorspronkelijke opzet van codering te slagen, dient het residu over beide kanalen zo klein mogelijk te zijn, of op z'n minst één of meerdere grootteordes kleiner dan het originele signaal. Om dit aan te tonen, wordt de lezer verwezen naar figuur 3.14 waarin de residu's van beide kanalen wordt voorgesteld. Afgezien van de sporadische uitschieters, bevindt de amplitude van elk residu zich rond de waarde 0.002, wat t.o.v. de oorspronkelijke maximale waarde 0.1 een verbetering van bijna 2 grootteordes betekent.



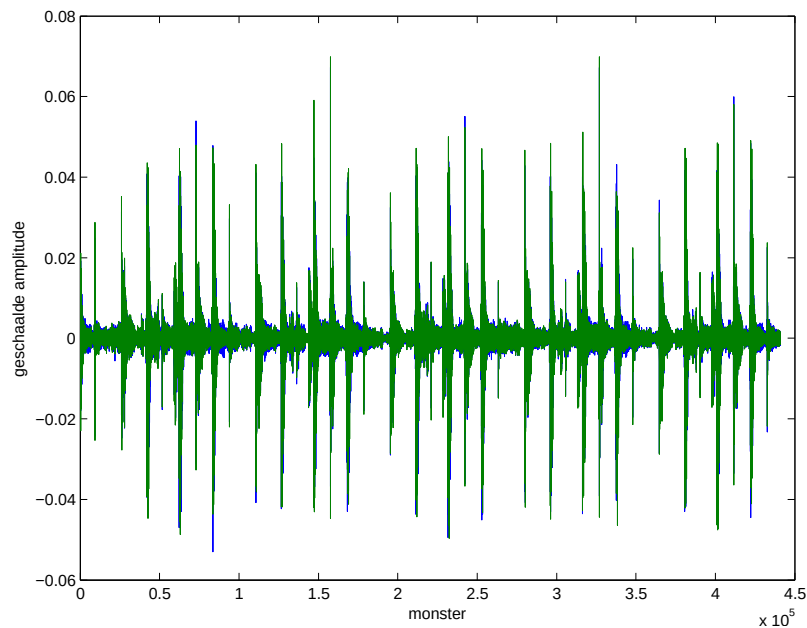
Figuur 3.12: Gereconstrueerde stereofone ingangssignalen.



Figuur 3.13: Reconstructiefout als verschil tussen originele en gereconstrueerde ingangen.

Residu na rotatie

Nu bestaat de mogelijkheid om deze residu's nog verder te verkleinen, nl. via het toevoegen van een extra rotatie na het analysefilter [41]. Deze rotatie zorgt voor een heroriëntering



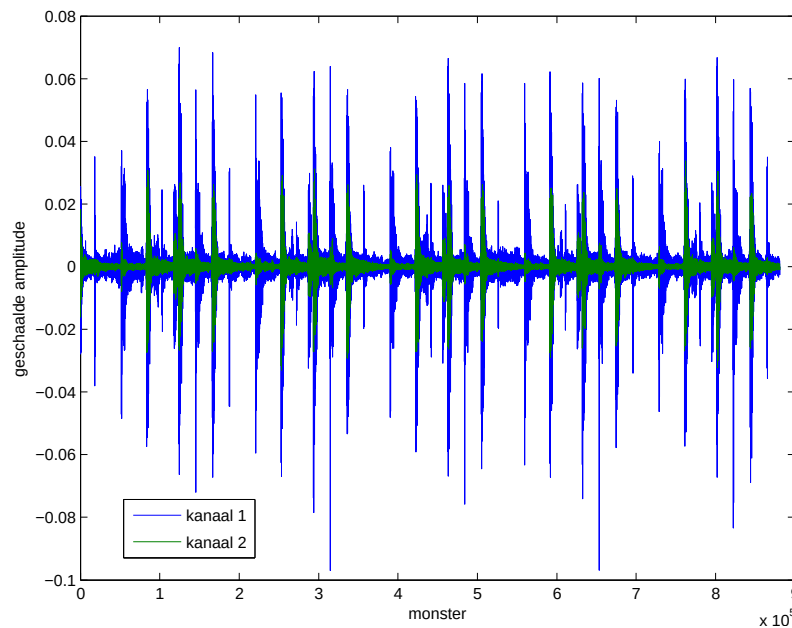
Figuur 3.14: Residuwaarden voor beide kanalen na analyse voor de ingangssignalen getoond in figuur 3.11.

van de assen in het tweedimensionaal assenstelsel waarin $e_1(n)$ en $e_2(n)$ coördinaten voorstellen van de vector $e(n)$. Gebaseerd op het principe van PCA [21] kan zo de waarde van één van de twee coördinaten verkleind en zelfs verwijderd worden, terwijl de andere het grootste deel van de totale energie in $e(n)$ overhoudt. Het doel hiervan is uiteraard hetzelfde als daarvoor: het nodige aantal transmissiebits nog sterker verkleinen om meer bandbreedte uit te sparen. Het toepassen van de rotatie uit de paper van Biswas op de verkregen signalen uit figuur 3.14 heeft de vervormde residu's van figuur 3.15 als resultaat.

Uit een nauwkeurige studie van bovenstaande grafiek kan geoordeeld worden dat het residu van kanaal 1 (blauw) bij momenten iets groter is in waarde dan zijn equivalent waarop geen rotatie is uitgevoerd. Echter, het contrast is klein en belangrijker: het is kleiner dan het waar te nemen verschil tussen het residu van kanaal 2 (groen) met en zonder rotatie. Indien gewenst kan zelfs zo ver gegaan worden dat alleen het eerste residu nog wordt doorgestuurd (eventueel met 1 bit meer nauwkeurigheid bij kwantisatie). De finale geluidskwaliteit is de bepalende factor bij dergelijke keuzes en de noden en specificaties van de toepassing waarin deze coderingsmethode wordt aangewend, zullen uiteindelijk de doorslag geven.

Predictieorde

Na een enkelvoudige uitvoering van het geïmplementeerde algoritme te hebben besproken, dient vervolgens het toedoen van de twee fundamentele parameters op de performantie onderzocht te worden. De predictieorde is de eerste van deze twee factoren. De proefopstelling hier aangewend is de volgende: een reeks van analyses zoals gedaan

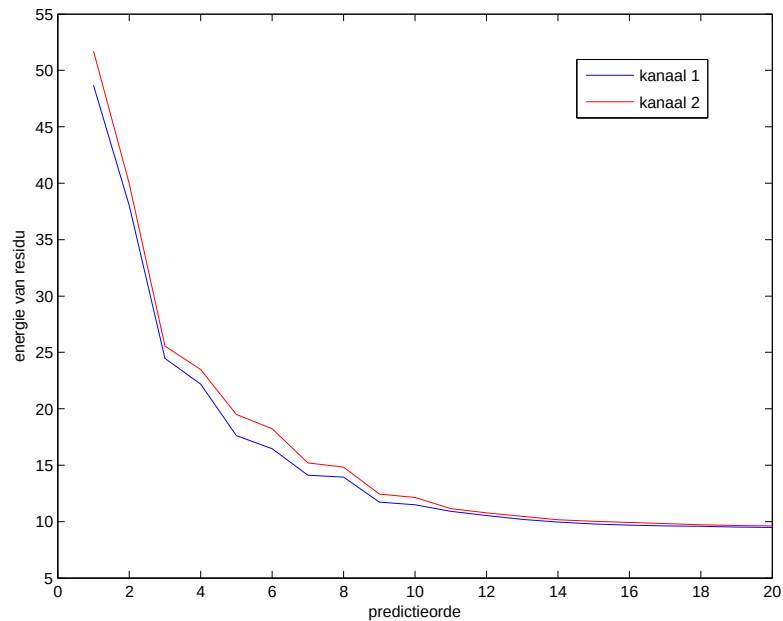


Figuur 3.15: Residuwaarden voor beide kanalen na analyse en rotatie voor deingangssignalen getoond in figuur 3.11.

in 3.4.2 om het residu te verkrijgen wordt uitgevoerd, waarbij de orde van de symmetrische predictieveltermen varieert van 1 tot 20. Om een veelzeggende maat voor de grootte van de residu's te garanderen is hier gekozen voor de energie van het residu van elk kanaal over de volledige 10s van hetingangssignaal. Deze termen zijn gegeven in (3.23) en (3.24). Het doel van het gehele lineaire predictieve coderingsschema was de minimalisatie van deze energie en verwacht wordt dat naarmate de predictie beter wordt, een kleinere energiewaarde zal worden bekomen. De experimentele resultaten van deze veronderstelling zijn getoond in figuur 3.16 en stemmen overeen met de verwachtingen.

Aangezien een grotere predictieorde gelijkloopt met een betere predictie is de residu-energie hiermee omgekeerd evenredig. Voornamelijk voor kleinere predictieorden is het verschil in energie van het residu erg opvallend. Zo geeft een overgang van orde 1 naar 3 een verbetering van ongeveer 50%. Daarnaast dient ook opgemerkt te worden dat de mogelijke verkleining in energie die kan bekomen worden voor predictieorden groter dan 10 zo goed als niet afweegt tegenover de extra complexiteit in de berekeningen. Dit is ook de reden waarom er over het algemeen voor een orde 10 of kleiner wordt gekozen, zoals ook het geval is voor de simulaties in dit hoofdstuk.

Vervolgens en vooral van belang voor de integratie uitgezet in hoofdstuk 4 bezit lineaire predictieve codering ook de eigenschap van decorrelatie van hetingangssignaal. De afhankelijkheid tussen de twee stereo kanalen wordt bij de transformatie naar residu's verminderd en op die manier verbeterd met betrekking tot akoestische echo-onderdrukking. Om de aard van de simultane decorrelatie te onderzoeken, is de coherentie tussen beide kanalen berekend door middel van de MATLAB functie `mscohere()` (zie ook deel 2.3.5) en



Figuur 3.16: Variatie van predictieorde en invloed hiervan op energie van residu's van beide kanalen.

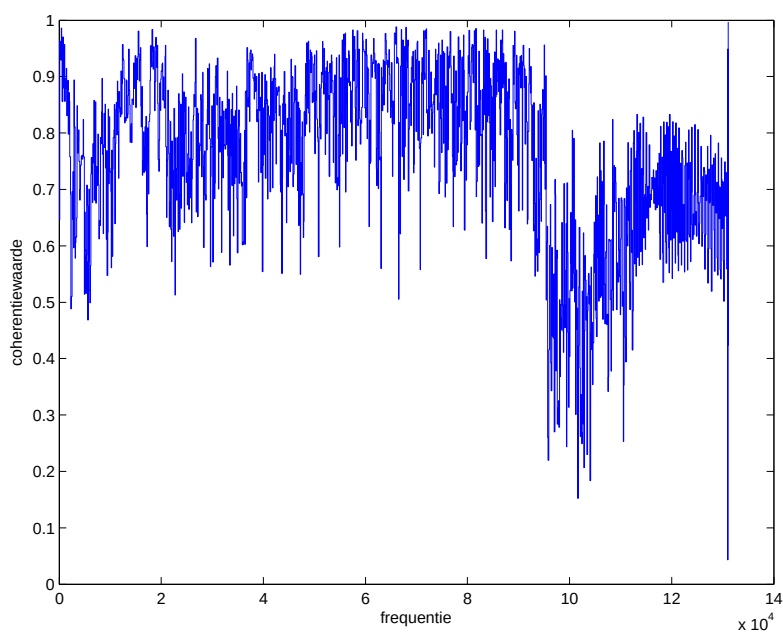
vergeleken voor predictieorden 1 en 20 (afbeeldingen 3.17 en 3.18). Vermits een grotere predictieorde een betere benadering van de ingang genereert (dewelke wordt afgetrokken van de eigenlijke ingang), zou dit moeten resulteren in een sterkere kruisdecorrelatie en kleinere kruiscoherentie.

Bovenvermelde grafieken tonen de bekomen resultaten voor de twee predictieorden, met een uitmiddeling van de coherentiewaarden per 100 monsters om de grafiek duidelijker te kunnen weergeven. Over het algemeen is er een kleine daling waar te nemen in de coherentie voor orde 20 t.o.v. 1, dewelke duidelijker aanwezig is in 3.19.

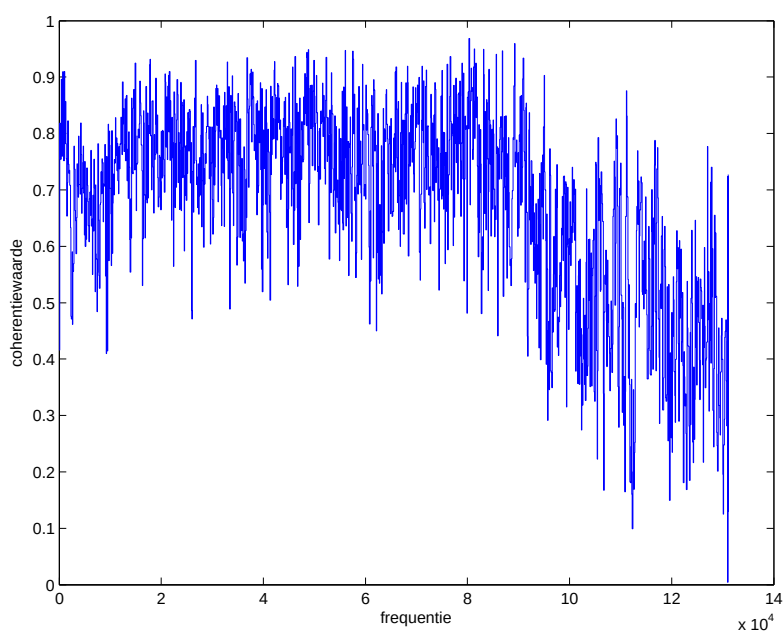
In deze grafiek is de energie van de gemiddelde kwadratische coherentie tussen de residu's van de twee kanalen bepaald. Een grotere predictieorde gaat ook hier gepaard met een kleinere energie en zo een sterkere decorrelatie via lineaire predictieve codering. Het kiezen van een predictieorde van 10 (zoals hierboven aangehaald) resulteert in een voldoende kleine residuenergie t.o.v. de minimale energie wanneer een maximale orde van 20 wordt gehandhaafd. Uit deze simulatieresultaten volgt in hoofdstuk 4 dat stereo LPC een mogelijk alternatief biedt als decorrelatiefilter.

Venstergrootte

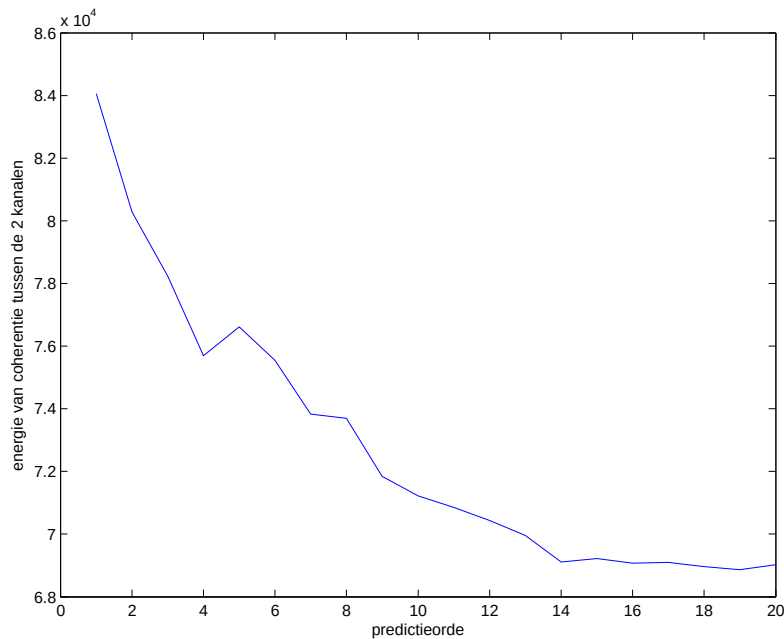
Ten slotte blijft de venstergrootte nog over, hoewel deze zeker niet te verwaarlozen is. In het ideale geval is het venster een exact veelvoud van de fundamentele periode van het foneem dat op dat moment wordt beschouwd. Omwille van de variatie tussen stemhebbende en stemloze fonemen, tussen de periodes van verschillende fonemen en de mogelijke aanwezigheid van een foneemovergang binnen het beschouwde venster is de



Figuur 3.17: Coherentie tussen de twee residu's in het frequentiedomein, voor predictieorde 1.



Figuur 3.18: Coherentie tussen de twee residu's in het frequentiedomein, voor predictieorde 20.



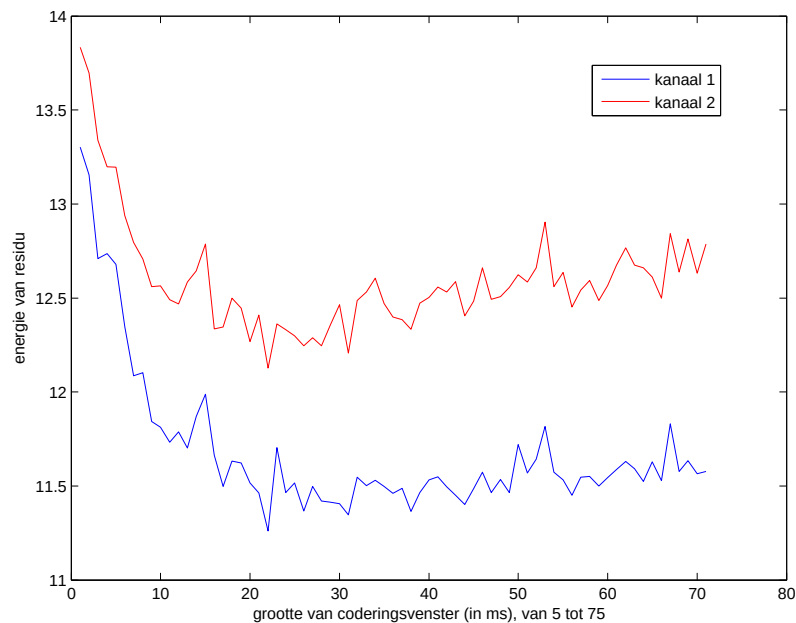
Figuur 3.19: Energie van de coherentie tussen de twee stereo kanalen in het frequentiedomein, voor verschillende predictieorden.

bepaling van de venstergrootte op die manier praktisch gezien van weinig nut. Daarom is voor de verdere testopstellingen in deze tekst gekozen voor een standaardgrootte van 20 ms [25]. Ter illustratie van een mogelijke optimalisatie toegepast op het te verwerken signaal (of soortgelijke signalen) wordt in bijhorende grafiek (figuur 3.20) de energie van de residu's uitgezet tegenover de grootte van het coderingsvenster. Dit laatste varieert van 5 tot 75 ms.

De figuur toont duidelijk aan dat er een venstergrootte kan gevonden worden waarbij de energie van het residu na codering en hiermee tegelijk het nodige aantal transmissiebits kunnen worden geminimaliseerd. Het verloop van de curve geeft een minimum weer om en bij de 25-30 ms voor dit specifieke stereo ingangssignaal. Indien dus op voorhand de aard van het te verzenden signaal gekend is en overeenkomt met de ingang hier getest, zou dergelijke grootte van coderingsvenster de beste optie zijn.

3.5 Samenvatting

Als recapitulatie worden hier de inhoud en gevolgen van dit hoofdstuk nog eens overlopen en benadrukt. Zo gaf deel 3.2 een eerste inleiding tot een aantal veelgebruikte bestaande coderingsmethoden en werden deze bondig vergeleken met de methode uitgewerkt in de rest van het hoofdstuk, nl. lineaire predictieve codering. Een aanleiding voor het gebruik van LPC is aangehaald in deel 3.3.1 en beslaat het verband dat bestaat tussen de modellering van spraaksignalen en het principe waarrond LPC is opgebouwd. In deel 3.3.3 werden de bijhorende berekeningen voor de modelleringsparameters vervolgens toegelicht en



Figuur 3.20: Energie van residu's van de twee stereo kanalen voor verschillende venster-groottes, te beginnen bij 5 ms.

is een uiteenzetting over de praktische implementatie van analyse en synthese neergeschreven. Sectie 3.4 ten slotte behandelde de uitbreiding naar stereo signalen, waarbij de besproken implementatie van stereo LPC ook voor audio werkte - in tegenstelling tot het schema uit deel 3.3.3 dat voor spraak het meest efficiënt werkt. Een algemene beschouwing van het toepassen van (stereo) lineaire predictie specifiek op audio is hier niet overwogen, zie [43] voor meer informatie. Een eigen implementatie en de resultaten hiervan vormden uiteindelijk het laatste onderdeel van dit hoofdstuk. Deze resultaten zullen in het volgende hoofdstuk opnieuw aangehaald worden, wanneer de integratie van codering en echo-onderdrukking wordt onderzocht. Twee factoren uit deel 3.4, nl. predictieorde en venstergrootte hebben hierbij een belangrijke invloed. Het vergroten van deze parameters - of toch voornamelijk de predictieorde - gaf over het algemeen een kleinere residuenergie, wat leidde tot een betere codering. In eerste instantie lijkt het dus zonder extra gegevens voordelig om een zo groot mogelijke (rekening houdend met de extra complexiteit van de berekeningen in de analyse- en synthesesetap) orde te gebruiken.

Integratie

4.1 Inleiding

Het hoofddoel van deze thesis beslaat, naast het onderzoeken van recente technieken inzake stereo akoestische echo-onderdrukking en stereo codering, het opzetten van een geïntegreerd systeem waarin één of beide onderdelen wordt (worden) aangepast om een optimale systeempersormantie te bekomen. In hoofdstukken 2 en 3 zijn de principes van deze twee begrippen grondig bestudeerd en werden eveneens simulatieresultaten van eigen implementaties getoond en besproken. Het huidige hoofdstuk gaat vervolgens verder met de integratie van echo-onderdrukking en codering, beiden gebaseerd op lineaire predictie omwille van redenen vermeld in vorige hoofdstukken. In het algemene geval van codering en SAEO worden de ontvangen signalen door het synthesefilter (dat bij het toepassen van lineaire predictieve codering instaat voor de inverse operatie van het analysefilter) gestuurd en wordt daarna SAEO (met decorrelatie) uitgevoerd. Door een integratie van beide delen worden bepaalde operaties overbodig en kan het volledige systeem ingekort worden en zo efficiënter worden gerealiseerd.

Hierbij zijn de gevonden resultaten en conclusies van de afzonderlijke delen uiteraard nuttig, maar moet ook rekening worden gehouden met de invloed van bepaalde variabelen op de werking van het gehele systeem. Uit de simulatieresultaten van vorig hoofdstuk, en meer specifiek sectie 3.4.2, volgen optimale waarden voor de predictieorde en venstergrootte in het LPC coderingsschema. Zoals in de samenvatting van hoofdstuk 3 besproken, wordt bv. de eerste van deze twee variabelen het best zo groot mogelijk genomen. Hoewel deze conclusie voor het principe van de toegepaste codering het meest positieve resultaat geeft, werden hierbij geen gevolgen voor andere aspecten beschouwd. Zo zal bij een integratie van codering en echo-onderdrukking ook aandacht moeten worden besteed aan de invloed van bovenstaande factoren op de aanwezigheid van convergentie en de bijhorende convergentiesnelheid van het stereo AEO-algoritme.

In volgende secties wordt daarom een eerste theoretisch (maar praktisch onbruikbaar) en een alternatief integratieschema voorgesteld door middel van een theoretische bespreking en het bijhorende verwerkingsproces toegepast op het ingangssignaal om (in volgorde) codering, decorrelatie en echo-onderdrukking uit te voeren. Daarna volgen resultaten voor beide schema's van simulaties met verschillende ingangssignalen en wordt de robuustheid van deze methoden bepaald t.o.v. variërende predictieorde en venstergrootte. Als laatste onderdeel volgt dan een meer praktische implementatie, met

name de werking van het systeem bij toevoeging van ruis op de predictiecoëfficiënten.

4.2 Combinatie LPC en SAEO: decorrelatie via roosterimplementatie

4.2.1 Theorie

In hoofdstuk 2 werd het algoritme in sectie 2.5.2 geïmplementeerd gebruik makend van een recursieve bepaling van de reflectiecoëfficiënten en dit voor elk nieuw tijdstip. Wanneer nu eerst codering wordt toegepast op hetingangssignaal worden daarentegen de reflectiecoëfficiënten slechts opnieuw berekend voor elk nieuw venster van monsterwaarden; dus veel minder snel. Concreet wil dit zeggen dat predictieveeltermen van eenzelfde graad worden gebruikt om een aantal, gelijk aan de venstergrootte, monsterwaarden te modelleren. Een poging tot integratie van codering en SAEO leidt in dit geval tot het gebruik van de constante reflectiecoëfficiënten per venster in de plaats van de oorspronkelijk recursief bepaalde coëfficiënten via de roosterimplementatie besproken in sectie 2.5.2.

Een meer concrete uiteenzetting begint bij een codering op basis van lineaire predictie, waarbij zowel codering als decodering volledig uitgevoerd worden. De ingang $\mathbf{x}(n)$ wordt via het analysefilter in figuur 3.10 omgezet naar de vector van voorwaartse predictiefouten $\mathbf{f}(n)$ volgens volgende vergelijking:

$$\mathbf{f}(n) = \mathbf{x}(n) - \hat{\mathbf{x}}(n) = \mathbf{x}(n) - \sum_{i=1}^K \mathbf{x}(n-i) \mathbf{A}_{K,i}. \quad (3.25)$$

waarbij de matrix $\mathbf{A}_{K,i}$ wordt gegeven door

$$\mathbf{A}_{K,i} = \begin{bmatrix} a_i & c_i \\ b_i & d_i \end{bmatrix}. \quad (3.24)$$

De codering gebeurt voor de rest exact zoals beschreven in hoofdstuk 3, eventueel met bijkomende rotatie en inverse rotatie toegepast op de twee kanalen van predictiefouten. De predictiecoëfficiënten waaruit de doorgestuurde reflectiecoëfficiënten κ_{ij} met $i = 1, 2$ en $j = 1, 2$ worden berekend, volgen uit de vergelijkingen:

$$\sum_{i=1}^K r_{11}(l-i) a_i + \sum_{i=1}^K r_{12}(l-i) b_i = r_{11}(l) \quad \text{voor } l = 1, 2, \dots, K, \quad (3.34)$$

$$\sum_{i=1}^K r_{21}(m-i) a_i + \sum_{i=1}^K r_{22}(m-i) b_i = r_{21}(m) \quad \text{voor } m = 1, 2, \dots, K, \quad (3.35)$$

$$\sum_{i=1}^K r_{11}(l-i) c_i + \sum_{i=1}^K r_{12}(l-i) d_i = r_{12}(l) \quad \text{voor } l = 1, 2, \dots, K, \quad (3.38)$$

$$\sum_{i=1}^K r_{21}(m-i) c_i + \sum_{i=1}^K r_{22}(m-i) d_i = r_{22}(m) \quad \text{voor } m = 1, 2, \dots, K. \quad (3.39)$$

De waarden voor de verschillende κ_{ij} 's worden zonder verdere behandeling in het LMS-algoritme ingevoerd. Tegelijk wordt de filteroperatie van het rooster in figuur 2.18 gedeeltelijk overgeslagen, aangezien alleen nog vergelijkingen (2.60) en (2.61), die hieronder nogmaals worden herhaald, moeten uitgevoerd worden, terwijl vergelijkingen (2.64) en (2.65) wegvallen.

$$\mathbf{b}_m(n) = \mathbf{b}_{m-1}(n-1) + \boldsymbol{\kappa}_m \mathbf{f}_{m-1}(n) \quad (2.59)$$

$$\mathbf{f}_m(n) = \mathbf{f}_{m-1}(n) + \boldsymbol{\kappa}_m^T \mathbf{b}_{m-1}(n-1) \quad (2.60)$$

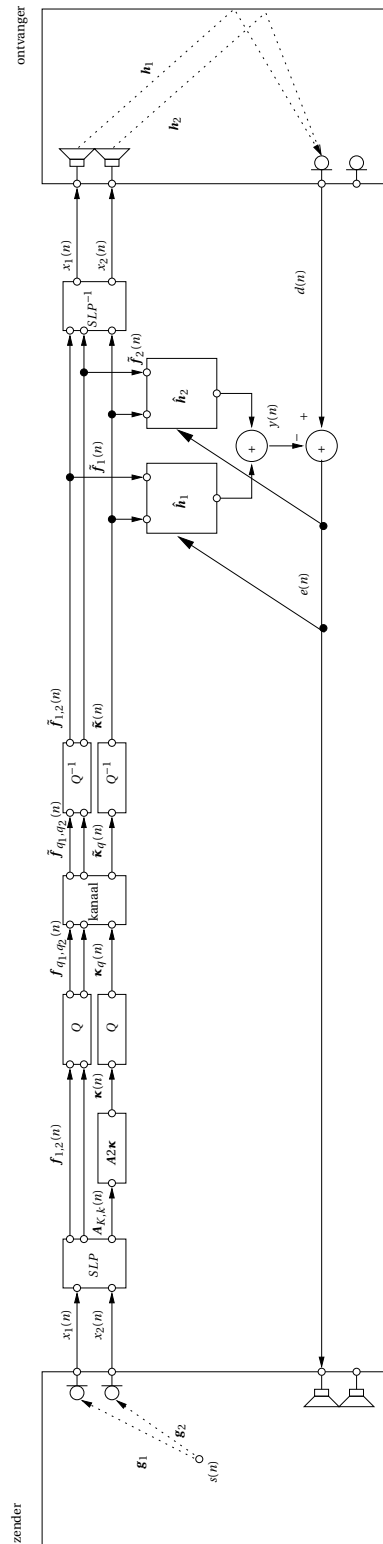
Daarna kan meteen worden overgegaan tot het adaptief aanpassen van de LMS-coëfficiënten via de methode beschreven in deel 2.5.2, waarbij alle operaties vanaf stap 2 en vergelijking (2.92) in volgorde worden toegepast.

Als in de plaats van de voorwaartse de achterwaartse predictiefouten $\mathbf{b}(n)$ zouden gebruikt worden, kan stap 1 van het hierboven vermelde algoritme volledig worden weggelaten. Zowel de waarden voor de κ_{ij} 's als de $\mathbf{b}(n)$'s zijn dan gekend en deze laatste dienen als ingang voor het aangepaste LMS-algoritme uit sectie 2.5.2. Toch kan dit niet op die manier geïmplementeerd worden, aangezien in dat geval een achterwaartse predictie van de ingangssignalen wordt uitgevoerd. Dit betekent dat de reconstructie (decoding) niet kan starten vanaf het eerste monster (in tijd) van het oorspronkelijk signaal $\mathbf{x}(n)$, waardoor een extra vertraging veroorzaakt wordt. Voor een real-time uitvoering is dit allesbehalve efficiënt. Het tegelijk doorsturen van voorwaartse en achterwaartse predictiefouten is ook geen optie omdat zo meer informatie dient verstuurd te worden, wat de compressieverhouding (te) sterk verkleint. Het schema dat voorgaande integratiemethode voorstelt, wordt getoond in figuur 4.1.

Gesteld dat de oorspronkelijk recursief gevonden reflectiecoëfficiënten (snel) convergeren naar de overeenkomstige waarden berekend via lineaire predictieve codering, wordt het algoritme uitgevoerd met een vergelijkbare performantie en convergentiesnelheid van de adaptieve filters.

4.2.2 Praktijk

In realiteit geeft het geïntegreerd schema minder positieve resultaten dan aanvankelijk verwacht. Na een aanzienlijk aantal simulaties met continu variërende waarden voor de verschillende parameters in het algoritme (stapgrootte van het LPC-rooster, β , ϵ en stapgrootte van LMS-algoritme) zijn geen bevredigende parameterwaarden gevonden. De reden hiervoor ligt in de interne werking van het afzonderlijke SAEO-algoritme beschouwd in hoofdstuk 2. De stapgrootte en als gevolg de recursief bepaalde reflectiecoëfficiënten nemen zo'n kleine waarden aan, dat ze helemaal niet overeenkomen met de gevonden reflectiecoëfficiënten uit het lineair predictief model. De SAEO-methode lijkt namelijk alleen te werken bij zeer strikte waarden voor de κ_{ij} 's; waarden die nauwelijks nog iets te maken hebben met de eigenlijke predictieve termen van het ingangssignaal. Het idee van een geïntegreerd schema komt hierdoor dan ook zonder twijfel in het gedrang en blijkt bij een correcte werking van de twee onderdelen niet mogelijk te zijn.



Figuur 4.1: Geïntegreerd schema decorrelatie via roosterimplementatie

4.3 Combinatie LPC en SAEO: decorrelatie via LPC-data

4.3.1 Theorie

Het doorgestuurd spraaksignaal, dat gecodeerd wordt via LPC, wordt ontbonden in reflectiecoëfficiënten, dewelke over het transmissiekanaal worden verzonden. Indien nu voor de decorrelatie ook lineaire predictie wordt aangewend, kunnen deze reflectiecoëfficiënten rechtstreeks in het decorrelatiealgoritme worden ingevoerd zonder hiervoor nog extra berekeningen noodzakelijk zijn. Zo kunnen de onderdelen *decoding* en *decorrelatie* vervangen worden door één gezamenlijke operatie. Hierbij wordt een winst in rekencomplexiteit bekomen, aangezien de decorrelatiestap omwille van de eerdere codering al gedeeltelijk gerealiseerd is. Deze winst zorgt op zijn beurt voor een snellere uitvoering van het SAEO-algoritme, wat positief is met het oog op een real-time implementatie.

Meer concreet werd in hoofdstuk 3.4 een uiteenzetting van stereo lineaire predictie gegeven, waarvan de belangrijkste stappen in deel 4.2.1 nog eens herhaald werden. Dezelfde operaties als hierboven worden voor deze vorm van integratie ook als eerste stap uitgevoerd en resulteren evenzeer in het signaal $\mathbf{f}(n)$ gegeven door vergelijking (3.25). Van dit resulterende signaal is bekend dat het meer gedecorreleerd is dan de ingang, waardoor het analysefilter eveneens dienst kan doen als decorrelatiefilter voor SAEO. Zo kan $\mathbf{f}(n)$ rechtstreeks worden gebruikt als gedecorreleerde ingang van het adaptief echo-onderdrukkingsfilter. Het adaptief algoritme dat hierbij wordt gebruikt is niets anders dan een standaard (N)LMS-algoritme, met als vergelijking

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \frac{\mathbf{x}(n)e(n)}{\mathbf{x}^T(n)\mathbf{x}(n) + \epsilon} \quad (4.1)$$

waarbij $\hat{\mathbf{h}}$ de vector van filtercoëfficiënten voorstelt, μ de stapgrootte, $\mathbf{x}(n)$ hetingangssignaal en $e(n)$ het verschil tussen de LMS-predictie en het eigenlijke opgevangen signaal van de microfoon in de dichte eind kamer (zie hoofdstuk 2):

$$e(n) = d(n) - y(n), \quad (2.4)$$

met $d(n)$ het signaal uit vergelijking (2.2), waarbij

$$d(n) = d_1(n) + d_2(n) \quad (4.2)$$

Het uiteindelijke toegepaste algoritme wordt dan:

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \frac{\mathbf{f}(n)e(n)}{\mathbf{f}^T(n)\mathbf{f}(n) + \epsilon}, \quad (4.3)$$

met $\mathbf{f}(n)$ uit vergelijking (3.25). Een niet te verwaarlozen feit is de aanwezigheid van een duplicaat van het SLP-filter (*Stereo Lineaire Predictie*) oorspronkelijk gebruikt voor decorrelatie in (3.18), dat als ingangen de twee kanalen heeft geconvolveerd met de verschillende impulsresponsies van de dichte eind kamer [26]. Dit filter is nodig voor inversie van het origineel filter en garandeert de correcte werking van het adaptief algoritme, aangezien de van elkaar afgetrokken waarden $y_{\text{lpc}}(n)$ en $d_{\text{lpc}}(n)$ (zie verder) hetzelfde signaal moeten voorstellen. Een bijkomende vereiste hiervoor houdt in dat het toegepast

filter een lineaire operatie verricht, aangezien dit dan onderling omwisselbaar is met de eigenlijke en benaderde kamerimpulsresponsies. De operaties uitgevoerd door dit duplicaat SLP-filter zijn hieronder gegeven.

$$\tilde{d}_1(n) = \sum_{i=1}^K a_i d_1(n-i) + \sum_{i=1}^K b_i d_2(n-i) \quad (4.4)$$

$$\tilde{d}_2(n) = \sum_{i=1}^K c_i d_1(n-i) + \sum_{i=1}^K d_i d_2(n-i). \quad (4.5)$$

De bekomen waarden worden daarna afgetrokken van $d_1(n)$ en $d_2(n)$, wat leidt tot

$$d_{\text{lpc}}(n) = [d_1(n) - \tilde{d}_1(n)] + [d_2(n) - \tilde{d}_2(n)] = d(n) - \sum_{j=1}^2 \sum_{i=1}^K \mathbf{d}(n-i) \mathbf{A}_{K,i} \quad (4.6)$$

met $\mathbf{A}_{K,i}$ uit (3.21), $d(n)$ uit (4.2) en

$$\mathbf{d}(n) = \begin{bmatrix} d_1(n) & d_2(n) \end{bmatrix}. \quad (4.7)$$

de kolomvector bestaande uit $d_1(n)$ en $d_2(n)$. Net zoals bij het standaard LMS-algoritme wordt de fout tussen het ontvangen signaal en een benadering hiervoor geminimaliseerd. De fout wordt in dit geval gegeven door

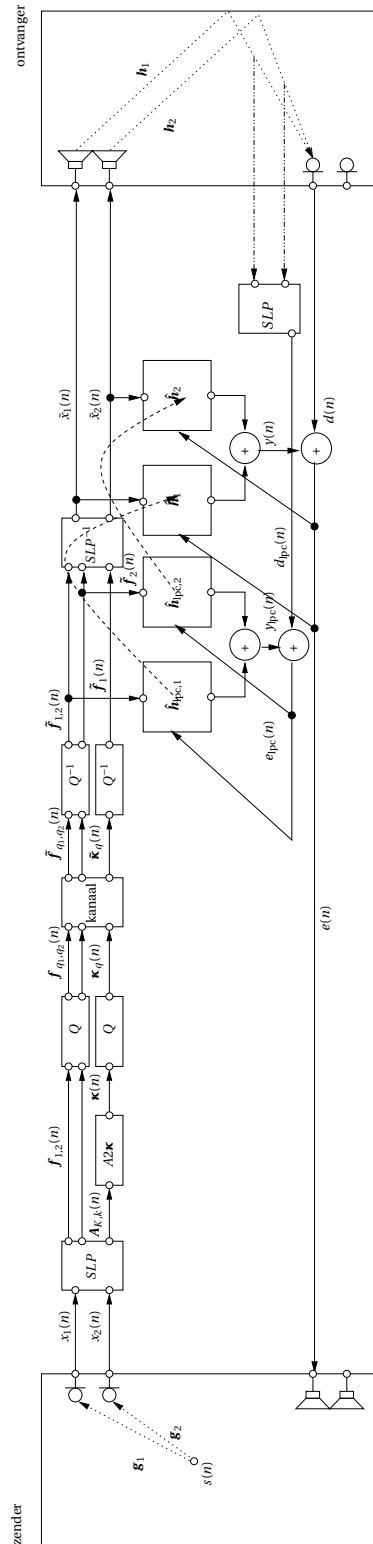
$$e_{\text{lpc}}(n) = d_{\text{lpc}}(n) - y_{\text{lpc}}(n), \quad (4.8)$$

met

$$y_{\text{lpc}}(n) = y_{\text{lpc},1}(n) + y_{\text{lpc},2}(n) = \mathbf{f}_1(n) * \hat{\mathbf{h}}_{\text{lpc},1}(n) + \mathbf{f}_2(n) * \hat{\mathbf{h}}_{\text{lpc},2}(n). \quad (4.9)$$

De opstelling besproken in voorgaande paragraaf bevat een onoverkomelijk nadeel: het duplicaat lineair predictief filter wordt toegepast op de convoluties van $\mathbf{x}_1(n)$ en $\mathbf{x}_2(n)$ met respectievelijk $\mathbf{h}_1(n)$ en $\mathbf{h}_2(n)$. Deze signalen zijn in realiteit niet zomaar voorhanden, wat maakt dat het echo-onderdrukkingsschema zoals tot nu toe besproken in praktijk niet bruikbaar is. Als oplossing voor dit probleem is volgende aanpassing aan het algoritme aangebracht: er wordt als het ware een dubbel LMS-algoritme uitgevoerd. Het al bestaand adaptief algoritme voor echo-onderdrukking uit vergelijking (4.3) wordt afgestemd op de fouten $e_{\text{lpc}}(n)$ gevonden door het verschil te nemen tussen de convolutie van de eigenlijke kamerimpulsresponsies en de voorwaartse predictiefouten uit vergelijking (3.25) - dit geeft het signaal $d_{\text{lpc}}(n)$ - en benaderde kamerimpulsresponsies en deze fouten - signaal $y_{\text{lpc}}(n)$. Daarnaast werkt een tweede LMS-algoritme simultaan op dezelfde recursief bepaalde filtercoëfficiënten met de standaard ingangen $\mathbf{x}(n)$. Door de aanwezigheid van het tweede standaard algoritme kan uit een aantal uitvoeringen hiervan een degelijke schatting van de kamerimpulsresponsies $\mathbf{h}_1(n)$ en $\mathbf{h}_2(n)$ gevonden worden. Deze schattingen worden nadien gebruikt als “reële” impulsresponsies door ze te convolueren met de ingangen $x_1(n)$ en $x_2(n)$, wat resulteert in $\hat{d}_1(n)$ en $\hat{d}_2(n)$. Hierop wordt dan het binnenste adaptief algoritme toegepast, te beginnen met een stereo lineaire predictie (SLP-blok in figuur 4.2). Voor dit gebeurt, wordt nog een verbetering aangebracht aan $\hat{d}_1(n)$ en $\hat{d}_2(n)$, om een nog betere schatting te bekomen. Hiervoor wordt $d_{\text{verschil}}(n)$ gedefinieerd als

$$d_{\text{verschil}}(n) = [\hat{d}_1(n) + \hat{d}_2(n)] - d(n) \quad (4.10)$$



Figuur 4.2: Geïntegreerd schema decorrelatie via predictiefouten

en wordt dit verschil gelijk verdeeld over de twee kanalen van de schatting

$$\hat{d}_1(n) = \hat{d}_1(n) - \frac{d_{\text{verschil}}(n)}{2}, \quad (4.11)$$

$$\hat{d}_2(n) = \hat{d}_2(n) - \frac{d_{\text{verschil}}(n)}{2}. \quad (4.12)$$

Uit het SLP-blok volgt dan het signaal $d_{\text{ipc}}(n)$. Concreet werd voor de volgende simulatieresultaten voor de eerste 100 monsters gewone LMS toegepast, dus het buitenste algoritme. Daarna worden de filtercoëfficiënten tot monster 10000 afwisselend door het buitenste en het binnenste algoritme bepaald. Vanaf 10000 monsters neemt het binnenste LMS-algoritme, met SLP-decorrelatie van de ingang, het volledig over. Figuur 4.2 geeft hieronder nog een samenvatting van het volledige schema.

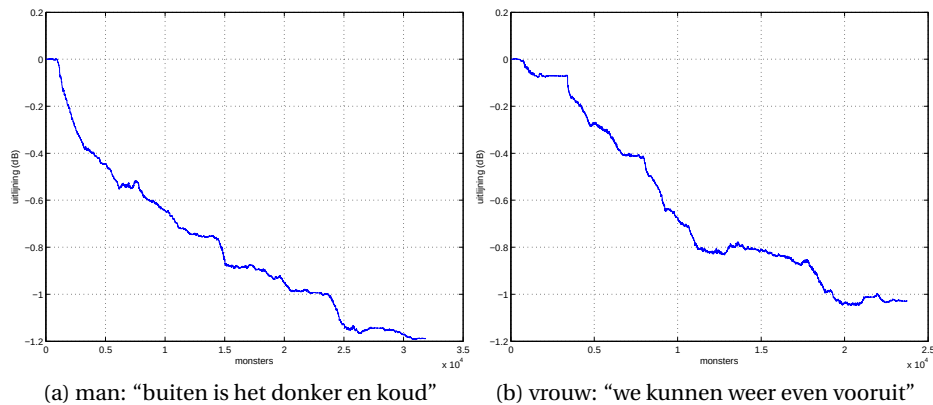
4.3.2 Simulaties

Voor de bekomen simulatieresultaten zijn dezelfde ingangen gebruikt als vroeger in deze tekst. De signalen die hier dienst doen als ingangen zijn de mannelijke en vrouwelijke stem uit sectie 2.3.6. Ze worden door kamerimpulsresponsies gestuurd die weer gemodelleerd zijn als onafhankelijke normaal verdeelde sequenties met gemiddelde 0 en variantie die afneemt als $1/n$, met n het tijdstip van de monsters. De lengte van de verre eind kamerimpulsresponsies L_F is hier gelijk gekozen aan 1024 en voor de dichte eind kamer zijn twee realistische impulsresponsies gebruikt, opgenomen in een middelgrote bureau-kamer, beiden met lengte $L_N = 2001$. In tegenstelling tot wat in hoofdstuk 2 wordt gezegd over de relatie tussen L_N en L_F , is hier wel een oplossing mogelijk voor de filtercoëfficiënten, aangezien deze recursief worden bepaald in dit schema. In de kamer zelf wordt ruis toegevoegd, zodat de signaal-ruisverhouding van het ontvangen signaal $d(n)$ herleid wordt naar 40dB.

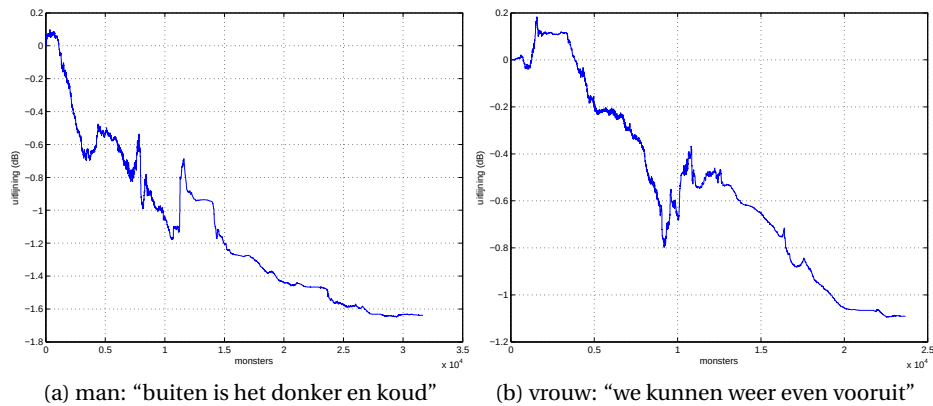
Als referentie wordt deze ingang eerst aangelegd op een standaard stereo LMS-algoritme, zonder enige bijgevoegde vorm van decorrelatie. De twee vectoren van filtercoëfficiënten zijn achter elkaar geplaatst om de uitlijning te bepalen en de optimale lengte van de adaptieve filters is experimenteel bepaald en is gelijk aan 600. De bekomen uitlijning voor deze opstelling wordt getoond in figuur 4.3.

Zoals kort aangehaald op het einde van hoofdstuk 2 zorgt stereo lineaire predictie voor een decorrelatie van het ingangssignaal en zou dit dus moeten leiden tot een snellere convergentie van de filtercoëfficiënten. Dit wordt ook proefondervindelijk vastgesteld door middel van figuren 4.4a voor de mannelijke stem en 4.4b voor de vrouwelijke. De predictieorde van het decorrelatiefilter werd hier gelijk aan 4 genomen, de optimale lengte van de adaptieve filters is 400 en de venstergrootte heeft nog steeds een lengte van 20 ms. De reden voor zo'n lage predictieorde ligt in de slechtere kwaliteit naarmate de orde groter wordt gekozen. Het exacte verloop wordt in volgende sectie beschouwd.

Indien ten slotte de vergelijking wordt gemaakt tussen dit voorgestelde algoritme en dat gebaseerd op decorrelatie via roosterimplementatie in de Newton/LMS-methode besproken in hoofdstuk 2, is op te merken dat de resultaten hier dezelfde performantie halen, terwijl de convergentiesnelheid voor audiosignalen zelfs beter is (vergelijk figuur 4.5 met figuur 2.31). Het gebruikte audiobestand hiervoor is [32] met dezelfde simulatieparameters als hierboven. Dit terwijl de uitvoeringssnelheid van het huidige schema ordes verschilt (in positieve zin) van die van het schema van hoofdstuk 2.



Figuur 4.3: Uitlijning van mannen- en vrouwenstem met toegevoegde ruis, bekomen via standaard LMS.

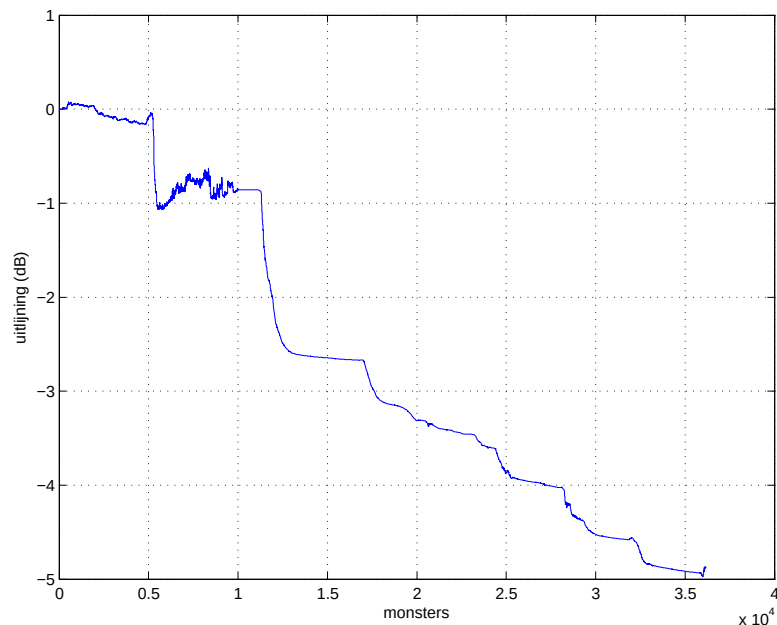


Figuur 4.4: Uitlijning van mannen- en vrouwenstem met toegevoegde ruis, bekomen via decorrelatie met stereo lineaire predictie.

4.3.3 Invloed van predictieorde

Vervolgens dient er rekening te worden gehouden met de predictieorde. In sectie 3.4.2 wordt de invloed van deze parameter op de resultante fout onderzocht. De conclusie hier hield in dat decorrelatie en predictieorde rechtevenredig verdeeld zijn. Hoewel een eenvoudige redenering hierbij zou leiden tot het maximaliseren van deze variabele, heeft dit weliswaar eveneens nadelen. Zo gaat een grotere predictieorde gepaard met een groter aantal coëfficiënten van de predictieveltermen en dus meer data die over het transmissiekanaal moet doorgestuurd worden. Op die manier wordt de te behalen compressiefactor dan ook verkleind. Daarnaast moet ook de rekencomplexiteit hier eveneens in rekening worden gebracht. Vergroten van de predictieorde zou namelijk meer berekeningen vergen en tegelijk meer rekentijd in beslag nemen.

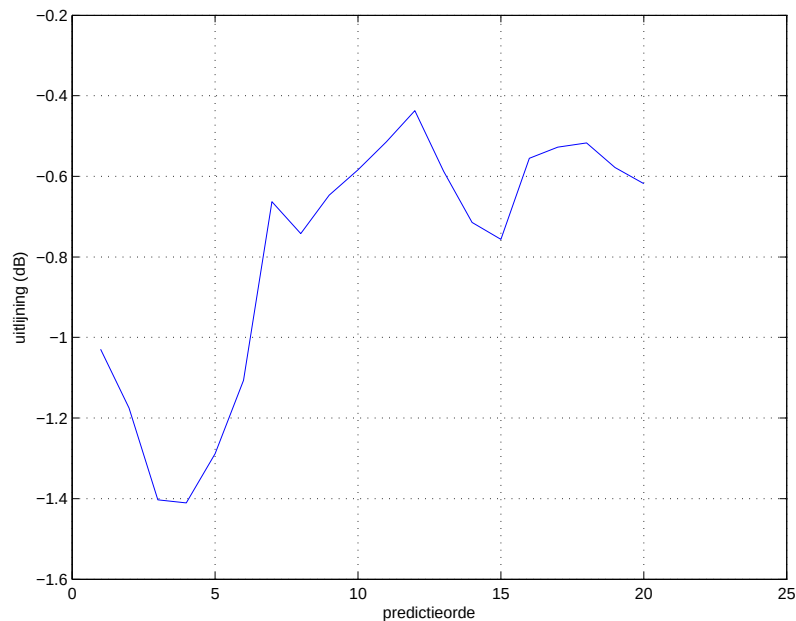
Om de lezer een beeld te verschaffen van de invloed van de predictieorde op de uit-



Figuur 4.5: Uitlijning van een audiosignaal met toegevoegde ruis, bekomen via decorrelatie met stereo lineaire predictie.

eindelijke uitlijning van de filtercoëfficiënten t.o.v. de kamerimpulsresponsies werd het voorgesteld algoritme verscheidene malen uitgevoerd, telkens met een andere waarde van de parameter. De resultaten zijn getoond in figuur 4.6 voor een kleinere dan de maximale stapgrootte, om bij elke beschouwde waarde van de predictieorde een convergerend resultaat te verkrijgen. In deze grafiek valt op dat voor kleine predictieorden de minimale uitlijning beter is dan voor grotere orden. Hierdoor moet een afweging worden gemaakt tussen de efficiëntie van codering (in termen van residuenergie) en de performantie van het echo-onderdrukingsalgoritme (in termen van convergentiesnelheid). Het eerste begrip is rechtevenredig met de predictieorde (sectie 3.4.2), het tweede omgekeerd evenredig.

Zoals eerder aangehaald is de uitlijning niet de enige belangrijke performantievariabele waarmee moet worden rekening gehouden. Afhankelijk van de toepassing waarvoor het huidige algoritme ingezet wordt, moet meer aandacht besteed worden aan bepaalde andere karakteristieken zoals bv. de rekencomplexiteit. Zo hangt deze term voor de in dit hoofdstuk beschreven methode af van de predictieorde K via het verband $3 * 8K * N + O(8K^2)$. N is hierbij het aantal monsters van het beschouwde ingangssignaal. De eerste term geeft het aantal gemaakte sommen en producten weer in het SLP-filter van de zender (vóór codering) en het duplicaat in de ontvanger, gecombineerd met de operaties van het inverse SLP-filter in de ontvanger – vandaar de factor 3. De tweede term staat in voor het aantal operaties in het blok-Levinson algoritme, gebruikt om de predictiecoëfficiënten te vinden. De gegeven complexiteitsbenadering dient, indien de concrete toepassing het vereist, afgewogen te worden tegenover de verandering in convergentiesnelheid.



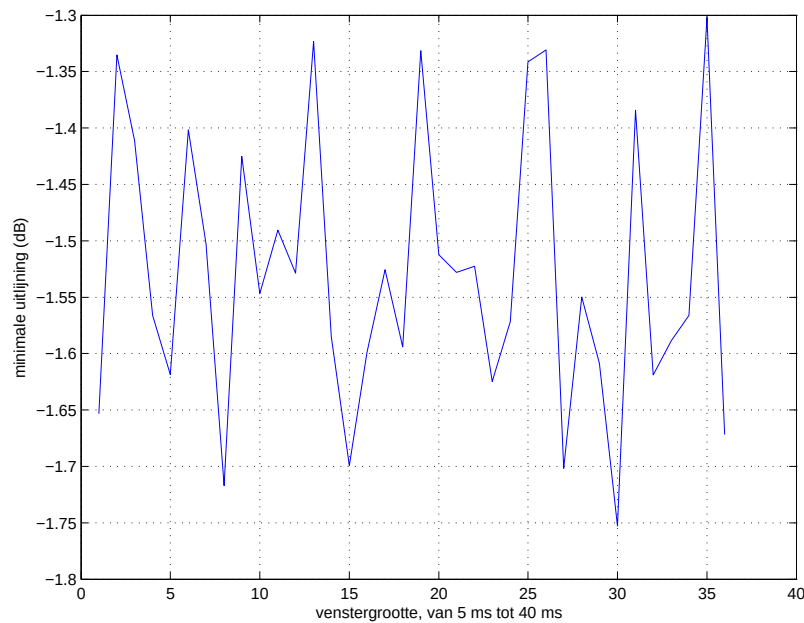
Figuur 4.6: Invloed van de predictieorde op de uitlijning na volledig doorlopen van het ingangssignaal.

Aangezien hier ten slotte slechts een algemene implementatiemogelijkheid naar voor wordt gebracht en vermits andere factoren zoals het type codering (verliesloos/benaderend) niet gekend zijn, beperkt deze tekst zich tot het geven van bovenstaande bevindingen en kan er geen uitspraak worden gedaan over waarden van specifieke performantiekenmerken.

4.3.4 Invloed van venstergrootte

Het is bekend dat tijdens het coderen in de zender het ingangssignaal opgesplitst wordt in vensters van bepaalde grootte (zie sectie 3.4.2 voor de gebruikte grootte en invloed van variatie hiervan) en voor de monsters in elk venster één set reflectiecoëfficiënten wordt berekend. Een wisselende venstergrootte heeft, net zoals een variërende predictieorde, een effect op de efficiëntie van decorrelatie en zo op de behaalde convergentiesnelheid.

Het resultaat van verschillende uitvoeringen van het voorgestelde algoritme met de mannenstem als ingangssignaal is weergegeven in figuur 4.7. De venstergrootte wordt hierin gevarieerd van 5 tot 40 ms tegenover de minimale uitlijning uitgesteld. De vrouwenstem als ingang geeft vergelijkbare bevindingen. Uit de resultaten is geen duidelijk verband tussen venstergrootte en convergentiesnelheid op te maken; alleen blijkt dat sommige vrij willekeurige waarden van de venstergrootte een betere performantie veroorzaken dan anderen.

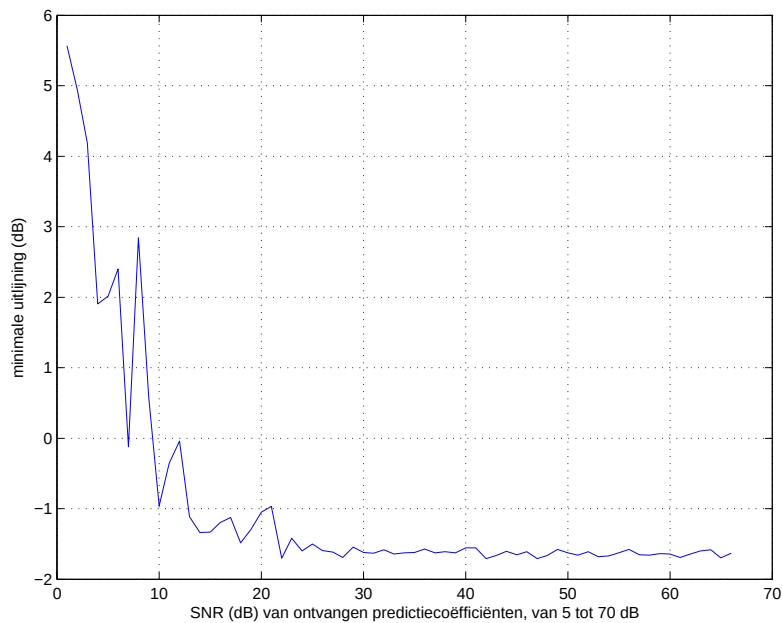


Figuur 4.7: Invloed van de venstergrootte op de uitlijning na volledig doorlopen van het ingangssignaal.

4.4 Niet-ideale transmissie

In dit laatste deel wordt een meer praktische opstelling getest. In de praktijk worden spraaksignalen vaak benaderend doorgestuurd over het kanaal. Hierdoor zijn kwantisatiefouten en kanaalruis niet weg te denken uit de opstelling en dit aspect wordt hier daarom gesimuleerd met weliswaar een vrij eenvoudige vorm van ruisadditie. De gevonden predictiecoëfficiënten van de lineaire predictieve veeltermen worden in realiteit, zoals vermeld in hoofdstuk 3, niet rechtstreeks maar in de vorm van reflectiecoëfficiënten doorgestuurd. Deze transformatie is voor de volgende simulaties niet uitgevoerd, aangezien hier vooral de aandacht is gevestigd op de convergentiekenarakteristieken van het voorgestelde algoritme en minder op de codering. Het soort codering hangt trouwens sterk af van de praktische toepassing, dus werd hier gekozen voor een additie van witte, Gaussiaanse ruis op de basisparameters waaruit elke codering moet starten, nl. de predictiecoëfficiënten zelf. De signaal-ruisverhouding die overblijft na deze ruisadditie wordt gevarieerd van 5 tot 70 dB en het resultaat op de minimale uitlijning bij convergentie (en dus op de convergentiesnelheid) wordt getoond in figuur 4.8.

De grafiek toont een duidelijke daling in de uitlijning na de volledige verwerking van het ingangssignaal bij stijgende signaal-ruisverhouding van de verzonden predictiecoëfficiënten. Daarnaast blijft de convergentiesnelheid vrij constant vanaf ongeveer 30 dB SNR. Aangezien deze coëfficiëntwaarden over het algemeen een slechte robuustheid vertonen tegenover perturbaties, zoals besproken in hoofdstuk 3, is het aangewezen ze te transformeren naar matrices met reflectiecoëfficiënten zoals uitgewerkt door het recursief algoritme in [41, p. 140-142].

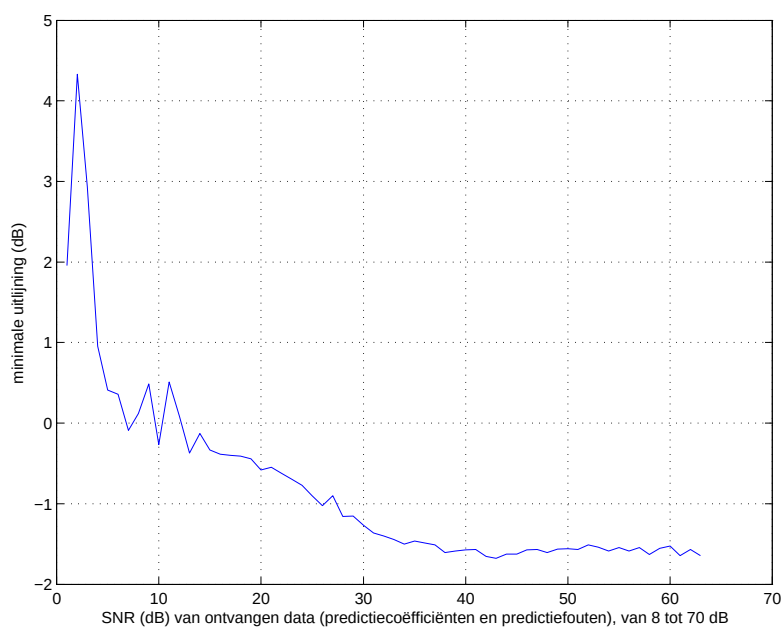


Figuur 4.8: Invloed van ruisadditie (in overgebleven SNR van de doorgestuurde predictiecoëfficiënten) op de convergentiesnelheid.

In het integratieschema worden twee karakteristieke waarden doorgestuurd: enerzijds de predictiecoëfficiënten, anderzijds de voorwaartse predictiefouten $f(n)$. In vorige simulatie werd van deze fouten verondersteld dat ze verliesloos gecodeerd en verzonden werden, zodat alleen de invloed van ruis op de predictiecoëfficiënten kon worden bepaald. Ter volledigheid wordt ten slotte in figuur 4.9 nog het resultaat getoond indien ook ruis wordt toegevoegd op de predictiefouten.

4.5 Samenvatting

In dit hoofdstuk werden twee alternatieve integratiemethoden voor een gecombineerde werking van lineair predictieve codering en stereo akoestische echo-onderdrukking gegeven. Hoewel het echo-onderdrukkingsgedeelte van beide alternatieven gebaseerd is op lineaire predictie (het coderingsgedeelte is identiek), leidt het verschil in realisatie tot uiteenlopende resultaten. Eerst werden de twee methoden afzonderlijk theoretisch uiteengezet, waarbij het meeste aandacht werd besteed aan de nieuw geïntroduceerde aspecten van elk schema. Daarna werden de mogelijkheid en de nodige voorwaarden voor een praktische implementatie onderzocht. Dit resulteerde in de conclusie dat het integratieschema dat berust op SAEO zoals uitgewerkt in hoofdstuk 2 in praktijk niet bruikbaar is. Het tweede schema daarentegen vertoonde wel bevredigende resultaten, dewelke uiteindelijk beter en sneller berekend zijn dan deze behaald via het algoritme uit sectie 2.5.2 zonder integratie. Als laatste zijn dan nog de invloeden van een aantal vooraanstaande parameters weergegeven en besproken.



Figuur 4.9: Invloed van ruisadditie (in overgebleven SNR van de doorgestuurde data) op de convergentiesnelheid.

In het eerste en meer theoretisch gedeelte van deze tekst werden twee verschillende processen aangehaald, die beiden voorkomen tijdens het typevoorbeeld dat diende als startpunt van de thesis, nl. het houden van een tele- of videoconferentie met stereofone spraak- en/of audiosignalen. Enerzijds worden de gegenereerde echo's in beide kamers onderdrukt door een adaptief algoritme, gebaseerd op genormaliseerde LMS. Anderzijds worden de opgenomen signalen verzonden over een niet-ideaal transmissiekanaal dat de verbinding vormt tussen de twee kamers. De recente ontwikkelingen inzake algoritmes en performantie van deze twee aspecten werden in vorige hoofdstukken vooreerst afzonderlijk onderzocht en eigen implementaties van de nieuwste methoden hieromtrent werden gerealiseerd en getoetst door middel van simulaties, eveneens beschreven in de desbetreffende hoofdstukken.

Het tweede gedeelte van de tekst handelde vervolgens over het samenzetten van de twee bovenvermelde begrippen, met de bedoeling om zo een efficiënter geheel te vormen dan wanneer beide delen apart en sequentieel zouden worden geïmplementeerd. Concreet omsloot dit stuk dus de uitwerking van onze vernieuwende bijdrage aan de onderwerpen besproken in vorige paragraaf. De beredeneerde vernieuwingen zijn uiteindelijk getest op performantie, zowel op zichzelf als in vergelijking met al bestaande technieken, en enkele resultaten op spraak- *en* audiosignalen werden getoond.

Als conclusie voor de volledige uiteenzetting worden de belangrijkste besluiten uit vorige hoofdstukken hieronder nog eens herhaald, waarbij voornamelijk aandacht wordt besteed aan het praktische nut ervan. Hierop verderbouwend worden ook suggesties gegeven voor toekomstig onderzoek en verder gebruik van de resultaten.

5.1 Stereo akoestische echo-onderdrukking

Zoals besproken in hoofdstuk 2 is het gebruik van “eenvoudige” aanpassingen als decoratiemethoden geen optie (meer) als een goeie performantie moet worden bekomen terwijl de signaalkwaliteit van het stereofoon signaal dient te worden bewaard. Als recente oplossing is dan het algoritme voorgesteld in [11] aangehaald, waarbij het idee van een efficiënt adaptief echo-onderdrukingsalgoritme gebaseerd is op een voorafgaande decoratiestap via lineaire predictie. De resultaten getoond in deze paper en de bevindingen bekomen door simulaties uitgevoerd op onze eigen implementatie zijn bevredigend als

theoretische experimenten, maar de manier waarop ze bekomen worden stellen teleur met het oog op een verdere integratie van deze methode met stereofone codering. De inwendige werking van het algoritme steunt namelijk op het recursief bepalen van de reflectiecoëfficiënten via de besproken roosterimplementatie met een enorm kleine stapgrootte. Het gevolg hiervan zijn waarden van deze reflectiecoëfficiënten die verre van overeenstemmen met de echte waarden. Indien, zoals ook uitgewerkt in hoofdstuk 2, de roosterpredictie wordt toegepast op een willekeurige ingang met een voldoende grote stapgrootte, treedt er een convergentie naar de realistische waarden van de reflectiecoëfficiënten op. Hierdoor werd oorspronkelijk aangenomen dat een integratie met dit algoritme een positief resultaat zou opleveren, wat uiteindelijk niet waar bleek te zijn. Omwille van deze reden is de methode uit [11] niet bruikbaar en werd op zoek gegaan naar een alternatief, dat gevonden werd in het principe van stereo lineaire predictie zelf, in de stap die stereo lineaire predictieve codering voorafgaat.

5.2 Stereo lineaire predictieve codering

Hoofdstuk 3 beschreef in detail het optimaal gebruik van lineaire predictieveltermen om door te zenden data te verkleinen en zo de compressiefactor van de transmissie te verbeteren, zowel voor monofone als voor stereofone signalen. Naast de theorie die in dit hoofdstuk uitgebreid aan bod kwam, zijn opnieuw resultaten getoond voor onze eigen implementatie van deze methode, toegepast op een stereofoon muzieksignaal. De weergegeven grafieken van de simulaties kwamen volledig overeen met de vooropgestelde verwachtingen, voor zowel de basistests (codering, decodering) als het onderzoek naar de invloed van belangrijke parameters die ook bij een latere integratie van belang zijn. Voor de effectieve resultaten hiervan wordt de lezer verwezen naar hoofdstuk 3 zelf. Door het bepalen van variaties op de orde van de lineaire predictie werd tegelijk een eerste concrete stap gezet richting integratie. De belangrijkste conclusie uit deze proeven hield in dat voor een betere decorrelatie, en als gevolg ook een snellere convergentie van het daaropvolgend adaptief algoritme, de predictieorde best zo groot mogelijk genomen werd. Een tegengestelde conclusie volgde verrassend genoeg uit simulaties uitgevoerd op het volledig geïntegreerd schema, maar daarover meer in volgende sectie.

5.3 Integratie

De theoretische achtergrond uiteengezet in [41] bood eveneens een gunstig alternatief voor het integratieprobleem uit sectie 5.1, aangezien de uitgangssignalen van het SLP-filter (zie hoofdstuk 4) meer gedecorreleerd horen te zijn dan de ingangen. Hierdoor kon dergelijk SLP-filter ook dienst doen als decorrelatiefilter in een adaptief echo-onderdrukingsalgoritme. Het feit dat bij integratie van beide delen de functie van dergelijk decorrelatiefilter al vervuld werd door de coderingsoperatie toegepast op het signaal vóór transmissie, was een bijkomend voordeel. Omwille van het volledig wegvallen van het decorrelatiefilter lag een integratie daarom voor de hand. Het enige aanzienlijk probleem dat hierbij nog de kop opstak, was de nood aan een duplicaat SLP-filter toegepast op de afzonderlijke kanalen van de echo in de dichte eind kamer. Deze kanalen zijn in

praktijk slechts beschikbaar als som van twee convoluties, dewelke op zich niet makkelijk te scheiden is. Om die reden werd in hoofdstuk 4 het tweevoudige LMS-algoritme voorgesteld, waarbij het algoritme uit het geïntegreerd schema voorafgegaan wordt door standaard LMS. Steunend op de veronderstelling dat de stereofone ingangssignalen sterk op elkaar gelijken - vermits ze beiden afkomstig zijn van hetzelfde bronsignaal in de verre eind kamer is deze veronderstelling gegrond - en dat hetzelfde geldt voor de twee kamerimpulsresponsies van de dichte eind kamer waarmee de ingangssignalen geconvolveerd worden, werd nog een extra aanpassing aan het tweevoudig algoritme toegevoegd. Zo wordt het verschil tussen de praktische benadering van het opgenomen signaal in de dichte eind kamer (bekomen door gebruik te maken van de adaptieve filters van het standaard LMS-algoritme als kamerimpulsresponsies van deze kamer) en de eigenlijke waarde hiervan evenredig verdeeld over de schatting van de kamerimpulsresponsies van beide kanalen. Het resultaat hiervan is een meer realistische waarde als uitgang van het duplicaat SLP-filter. Hoewel deze aanpassingen aan het originele LMS-algoritme een extra rekencomplexiteit in het geheel introduceert, blijft de methode significant sneller dan het oorspronkelijk echo-onderdrukingsalgoritme uit [11], zoals uit simulatieresultaten op spraak- en audiosignalen in hoofdstuk 4 volgt. Voor een aantal onderzochte audiosignalen werd eveneens een betere convergentiesnelheid bekomen.

5.4 Verder onderzoek

Zoals in vorige sectie vermeld, lijkt het voorgestelde algoritme een beter alternatief te zijn dan dat besproken in [11], dit op het gebied van echo-onderdrukking afzonderlijk en zeker wanneer een geïntegreerd schema beschouwd wordt. Hoewel de resultaten hier veelbelovend zijn, is de convergentiesnelheid van de methode nog te beperkt voor praktische toepassingen en zal hiervoor naar manieren moeten worden gezocht om de performantie verder te vergroten. Zo kan er nog een (tweede) decorrelatiefilter worden voorzien in de ontvanger, aangezien er in het integratieschema geen fysisch filter met die functie bestaat (decorrelatie zit intrinsiek verwerkt in de codering). Het toevoegen van akoestisch gemaskeerde ruis is een voorbeeld van dergelijke decorrelatie. Andere al bestaande, op zich minder efficiënte, decorrelatiemethoden kunnen hier ook aangewend worden, aangezien zij de aanwezige decorrelatie slechts hoeven te versterken. In elk geval zal de nodige performantie van het systeem afhankelijk zijn van de toepassing die ervan gebruik maakt, net zoals hierdoor een grens aan de overblijvende geluidskwaliteit wordt opgelegd. De specifieke eisen die aan de resulterende methode worden gesteld worden volledig door het praktisch gebruik bepaald en bepalen op hun beurt de ontwerpruimte aanwezig binnen het geïntegreerd schema.

Bijlagen

Bibliografie

- [1] G. E. Moore, "Cramming more components onto integrated circuits," *Proceedings of the IEEE*, vol. 38, no. 1, pp. 82–85, Apr. 1965.
- [2] M. Sondhi, D. Morgan, and J. Hall, "Stereophonic acoustic echo cancellation-an overview of the fundamental problem," *Signal Processing Letters, IEEE*, vol. 2, no. 8, pp. 148–151, 1995.
- [3] J. Benesty, D. Morgan, and M. Sondhi, "A better understanding and an improved solution to the specific problems of stereophonic acoustic echo cancellation," *Speech and Audio Processing, IEEE Transactions on*, vol. 6, no. 2, pp. 156–165, 1998.
- [4] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, Sep. 2001.
- [5] E. Hänsler and G. Schmidt, *Acoustic echo and noise control*. Wiley-IEEE, 2004.
- [6] J. G. Proakis and D. K. Manolakis, *Digital Signal Processing*, 4th ed. Prentice Hall, Apr. 2006.
- [7] E. A. Habets, "Single-and multi-microphone speech dereverberation using spectral enhancement," <http://library.tue.nl/catalog/LinkToVubis.csp?DataBib=6:627677>, 2007.
- [8] D. Morgan, J. Hall, and J. Benesty, "Investigation of several types of nonlinearities for use in stereo acoustic echo cancellation," *Speech and Audio Processing, IEEE Transactions on*, vol. 9, no. 6, pp. 686–696, 2001.
- [9] T. Gansler and J. Benesty, "New insights into the stereophonic acoustic echo cancellation problem and an adaptive nonlinearity solution," *Speech and Audio Processing, IEEE Transactions on*, vol. 10, no. 5, pp. 257–267, 2002.
- [10] L. W. Couch, *Digital and analog communication systems*, 7th ed. Pearson/Prentice Hall, 2007.
- [11] H. I. K. Rao and B. Farhang-Boroujeny, "Fast LMS/Newton algorithms for stereophonic acoustic echo cancellation," *IEEE Transactions on Signal Processing*, vol. 57, no. 8, pp. 2919–2930, 2009.
- [12] J. B. Allen and D. A. Berkley, "Image method for efficiently simulating small-room acoustics," *J. Acoust. Soc. Am*, vol. 65, no. 4, pp. 943–950, 1979.

-
- [13] S. Shimauchi and S. Makino, "Stereo projection echo canceller with true echo path estimation," in *Acoustics, Speech, and Signal Processing, 1995. ICASSP-95., 1995 International Conference on*, vol. 5, 1995.
 - [14] H. Fastl and E. Zwicker, *Psychoacoustics*. Springer, 2007.
 - [15] J. Johnston and K. Brandenburg, "Wideband coding-Perceptual considerations for speech and music," *Advances in Speech Signal Processing*, pp. 109–140, 1991.
 - [16] R. W. Chang, "A new equalizer structure for fast start-up digital communication(Automatic equalizer with fast startup time for digital communication systems including partial response and single sideband nyquist systems)," *Bell System Technical Journal*, vol. 50, pp. 1969–2014, 1971.
 - [17] C. F. N. Cowan, "Performance comparisons of finite linear adaptive filters," in *IEEE Proceedings F Communications, Radar and Signal Processing*, vol. 134, 1987, pp. 211–216.
 - [18] I. Newton, "Methodus fluxionum et serierum infinitarum," *Opuscula mathematica, philosophica et philologica*, vol. 1, 1671.
 - [19] K. Karhunen, *Ueber lineare Methoden in der Wahrscheinlichkeitsrechnung*. Kirjapaino oy. sana, 1947.
 - [20] M. Loeve, "Probability theory, vol. II," *Graduate Texts in Mathematics*, vol. 46, p. 387, 1978.
 - [21] I. T. Jolliffe, *Principal component analysis*. Springer, 2002.
 - [22] D. C. Lay, *Linear Algebra and Its Applications, 3rd Updated Edition*, 3rd ed. Addison Wesley, Sep. 2005.
 - [23] B. Widrow and E. Walach, *Adaptive inverse control*. Prentice Hall PTR, Dec. 1995.
 - [24] F. Beaufays and B. Widrow, "Two-layer linear structures for fast adaptive filtering," Ph.D. dissertation, to the Department of Electrical Engineering.Stanford University, 1995. [Online]. Available: <<http://www.speech.sri.com/people/francois/publications.html>>
 - [25] W. C. Chu, *Speech coding algorithms: foundation and evolution of standardized coders*. Wiley-Interscience, 2003.
 - [26] C. Breining, P. Dreiscitel, E. Hansler, A. Mader, B. Nitsch, H. Puder, T. Schertler, G. Schmidt, and J. Tilp, "Acoustic echo control. an application of very-high-order adaptivefilters," *IEEE Signal Processing Magazine*, vol. 16, no. 4, pp. 42–69, 1999.
 - [27] F. Ling and J. Proakis, "A generalized multichannel least squares lattice algorithm based on sequential processing stages," *Acoustics, Speech and Signal Processing, IEEE Transactions on*, vol. 32, no. 2, pp. 381–389, 1984.
 - [28] K. Mayyas, "Stereophonic acoustic echo cancellation using lattice orthogonalization," *Speech and Audio Processing, IEEE Transactions on*, vol. 10, no. 7, pp. 517–525, 2002.

-
- [29] B. Farhang-Boroujeny, "Fast LMS/Newton algorithms based on autoregressive modeling and their application to acoustic echo cancellation," *IEEE Transactions on Signal Processing*, vol. 45, no. 8, pp. 1987–2000, Aug. 1997.
- [30] N. Levinson, "The wiener RMS (root mean square) error criterion in filter design and prediction," *J. Math. Phys*, vol. 25, no. 4, pp. 261–278, 1947.
- [31] S. Narayan, A. Peterson, and M. Narasimha, "Transform domain LMS algorithm," *IEEE Transactions on Acoustics, Speech and Signal Processing*, vol. 31, no. 3, pp. 609–615, 1983.
- [32] K. Andersson and M. Deykers, "Seven (Martyn's seventh mix)," Sep. 2009. [Online]. Available: <<http://www.discogs.com/Fever-Ray-Seven/release/2061300>>
- [33] J. Herre, "From joint stereo to spatial audio coding-recent progress and standardization," in *Sixth International Conference on Digital Audio Effects (DAFX04), Naples, Italy*, 2004, pp. 157–162.
- [34] J. Geurts, L. L. M. Vogten, and A. Biswas, "Stereo linear predictive coding of audio," 2006.
- [35] K. Sayood, *Introduction to data compression*. Morgan Kaufmann, 2000.
- [36] L. Wood, "Graphics interchange format programming reference," <http://www.w3.org/Graphics/GIF/spec-gif89a.txt>, Jul. 1990. [Online]. Available: <<http://www.w3.org/Graphics/GIF/spec-gif89a.txt>>
- [37] T. Welch, "Technique for high-performance data compression." *Computer*, vol. 17, no. 6, pp. 8–19, 1984.
- [38] J. Bradbury, *Linear predictive coding*. Linear Predictive coding, 2000.
- [39] T. E. Tremain, "The government standard linear predictive coding algorithm: LPC-10," *Speech Technology*, vol. 1, no. 2, pp. 40–49, 1982.
- [40] J. H. Wilkinson, "The perfidious polynomial," *Studies in numerical analysis*, vol. 24, p. 28, 1984.
- [41] A. Biswas, "Advances in perceptual stereo audio coding using linear prediction techniques," <http://library.tue.nl/catalog/LinkToVubis.csp?DataBib=6:625934>, 2007.
- [42] T. Liebchen, "Lossless audio coding using adaptive multichannel prediction," *PREPRINTS-AUDIO ENGINEERING SOCIETY*, 2002.
- [43] T. van Waterschoot and M. Moonen, "Comparison of linear prediction models for audio signals," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2008, p. 2, 2008.

Fiche masterproef

Studenten: Martijn Hemeryck
Hans Van Herck

Titel: Geïntegreerde stereo-akoestische-echo-onderdrukking en spraakcodering voor tele-
/videoconferencingtoepassingen

Engelse titel: Integrated stereo acoustic echo cancellation and speech coding for remote
and video conferencing applications

UDC: 621.3

Korte inhoud:

Voor tele-/videoconferencingtoepassingen is het interessant om in een stereofone opstelling te werken o.w.v. de spatiale informatie. Problemen gerelateerd met akoestische echo-onderdrukking worden hierbij wel complexer. In een eerste deel worden de fundamentele problemen die met deze opstelling gepaard gaan opnieuw besproken evenals een aantal oplossingen. Meer specifiek oplossingen gebaseerd op (stereo) lineaire predictie worden bekeken. In een ander deel wordt dieper ingegaan op coderingstechnieken voor stereosignalen, met opnieuw een nadruk op lineaire predictie. Gezien voor beide signaalverwerkingstechnieken alternatieven bestaan op basis van lineaire predictie worden in een laatste deel twee concrete schema's voorgesteld voor een integratie.

Proefschrift voorgedragen tot het behalen van de graad van Master in de
ingenieurswetenschappen: elektrotechniek, optie Multimedia en signaalverwerking

Promotor: Prof. dr. ir. Marc Moonen

Assessors: Prof. dr. ir. Hugo Van hamme
Dr. ir. Ann Spriet

Begeleiders: Dr. ir. Toon van Waterschoot
Ir. Jose Manuel Gil-Cacho Lorenzo

Integrated stereo acoustic echo cancellation and speech coding for remote and video conferencing applications

Martijn Hemeryck, *Student Member, IEEE*, Hans Van Herck, Toon van Waterschoot, Pepe Gil Cacho, and Marc Moonen, *Fellow, IEEE*

Abstract—For videoconferencing applications it is interesting to have a stereophonic setting because of the spatial information. As a consequence, problems related to echo cancellation become more complex. In a first part, fundamental problems related to stereo acoustic echo cancellation are reviewed as are a number of possible solutions. More specifically, solutions based on linear prediction are mentioned. In another part, stereo coding techniques based on linear prediction are reviewed, i.e. the extension of linear prediction to multiple channels: stereo linear prediction [1]. Since for both techniques, alternatives based on stereo linear prediction exist, two integrated schemes are presented here. One of the schemes is based on an algorithm presented in [2]. This algorithm uses a recursive, sample based, estimation of the AR speech parameters in an LMS/Newton adaptive filter algorithm. The other scheme is based on a stereophonic extension of an algorithm discussed in [3], where an NLMS algorithm operates directly on the prediction error signals.

Index Terms—adaptive filters, stereo acoustic echo cancellation (SAEC), coding, linear prediction, integrated, video, conferencing

I. INTRODUCTION

ONE of the most used applications of adaptive signal processing is in the field of echo cancellation. Specifically for video conferencing applications a certain kind of acoustic echo cancellation scheme is required to remove the echoes caused by the coupling of the speaker signal into the microphone, or even worse, to prevent acoustic howling to occur. As more and more processing power [4] and bandwidth becomes readily available, it becomes interesting to extend the number of channels to increase the spatial realism. In this respect, one might argue that since the invention of the first telephone, there still isn't a communication system available that allows one to talk to another person with the same spatial realism as if they were in the same room.

Even though a great deal of (acoustic) echo cancellation schemes exist [5] and can be applied in case of a single channel, direct extension of this concept to multiple channels is not straightforward. In [6] the fundamental problem related to stereo acoustic echo cancellation is first explained and in [7] a more thorough mathematical analysis is made. As will be explained in part II, the fundamental problem is related to the intrinsic correlation that exists between the two channels. As such, most of the literature on solutions to the stereo acoustic echo cancellation problem employ a kind of decorrelation stage prior to an adaptive filtering algorithm.

Another important issue in any communication system is related to the coding system used for transmitting the actual data. Despite the increase in available bandwidth, the current explosion in telecommunications traffic would have been inconceivable without the proper codecs to compress all these data. Specifically for speech signals linear prediction based coding has been very successful, in part because of the relationship with physical properties of the human speech production system. An extension to two channels is also not straightforward and will be explained in part III.

In [5] it is shown that the residuals from linear prediction are orthogonal. The property of orthogonality is particularly interesting to the problem at hand for two major reasons. Firstly, as stated earlier and as will become more clear later on, a certain kind of decorrelation stage is necessary for an extension of the acoustic echo canceller to multiple channels. Secondly, since the speech codec is also based on linear prediction, it becomes interesting to investigate a possible integration of these two systems. This idea of integration is the main topic of part IV. Since echo cancellation on one hand and stereo coding on the other impose different constraints on the overall system, a certain tradeoff for specific parameters will be necessary.

II. STEREO ACOUSTIC ECHO CANCELLATION

A. Setup

The setup that is mostly assumed for the stereophonic acoustic echo cancellation problem is shown in figure 1. The left hand side shows the sending side or *far end* room, the right hand side shows the receiving side or the *near end* room. At the far end room, a speaker emits a source signal $s(n)$ that is transmitted through the room echo paths g_1 and g_2 , producing two microphone signals $x_1(n)$ and $x_2(n)$. The echo paths are commonly modelled as time-varying FIR filters [8], such that $x_i(n) = s(n) * g_i, i = 1, 2$. At the near end side, the stereophonic signals are passed through the room echo paths h_1 and h_2 , yielding two microphone signals $d_i(n) = x_i(n) * h_{1,i} + x_i(n) * h_{2,i}$. In the figure, only one of those desired signals is shown as the description for both channels is similar. To cancel the effect of the far end signals on the desired signals, two adaptive FIR filters $\hat{h}_1$ and $\hat{h}_2$ are defined, each corresponding to one of the echo paths in the near end room. As in monophonic acoustic echo cancellation, the resulting signal $y(n)$ is used to remove the far end signal from $d(n)$.

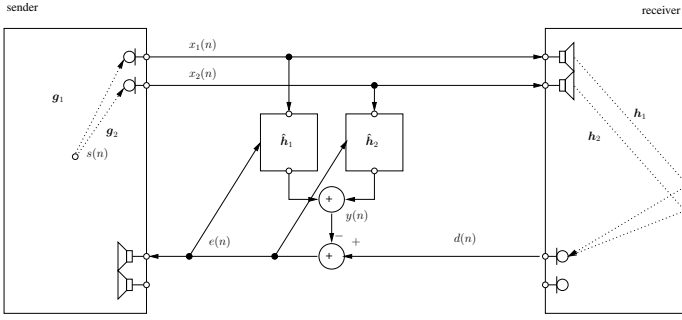


Fig. 1. General setup stereophonic acoustic echo cancellation

B. Fundamental problem

Extension of the monophonic acoustic echo cancellation problem to more channels is not straightforward due to some fundamental differences. These differences were first addressed in [6]. The main result there was that the intrinsic coupling between the channels result in an ill-conditioned set of equations from which to determine the adaptive filter coefficients. Here, it was already clear that the far end room echo paths also play a role in the determination of the adaptive filter for the near end room. Further mathematical details for this problem were provided by [7]. The effect for different lengths of the echo paths were taken into account: suppose the following notation L_M , L_F and L_N for respectively the lengths of the echo paths for the modelling adaptive filters, the far end room and the near end room. It was proved that for $L_M < L_F$ a unique solution exists, which is always the case since real echo paths have an infinite length [8]. It was also shown that for $L_M < L_N$, some form of *misalignment* will always exist, which is also always the case, again due to the fact that real echo paths have an infinite length. To deal with this misalignment, it was suggested that some sort of decorrelation stage needs to be applied to the far end signals $x(n)$, prior to the application of an adaptive filtering algorithm. To give a measure of the resulting correlation of such a method, the coherence function was suggested, which is basically a normalised measure of the cross-correlation in the frequency domain.

Virtually all proposed solutions do employ such a decorrelation stage. In [6] the following methods are proposed: addition of uncorrelated noise to (one of) the channels, applying certain frequency transformations such as inter channel frequency shifting and interleaved notch filtering. In [7] the application of a nonlinear function to the signal is proposed. A different number of such nonlinear functions is explored and further mathematical details of their effect on the coherence function are explained in [9]. A characteristic of all these solutions is the inherent tradeoff between signal quality and the decorrelation effectiveness. Since the goal of a stereophonic setup is to give a spatial impression, it is important to preserve this information. For the solution which uses the nonlinear function for the decorrelation, it has been reported that a good reduction in coherence is possible, with little loss of signal quality. Our own simulations confirm this for the simple case in which a half wave rectifier is used.

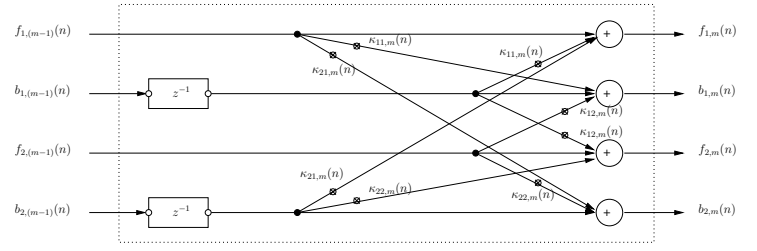


Fig. 2. Lattice predictor for recursive estimation of AR parameters

C. Linear prediction based decorrelation

From [5] it is known that the prediction error signals for different orders are orthogonal. This property makes it interesting to use it as part of a decorrelation stage in an SAEC scheme. Also, since linear prediction is one of the most widely used methods for parametric speech coding, a linear prediction based approach proves interesting in this context, as it provides the possibility of integrating the SAEC scheme with a stereo coding scheme. Here, two different approaches were examined.

One of the algorithms is an extension of results from the literature, more specifically the algorithm proposed in [2]. The AR parameters in this algorithm are determined in a recursive fashion using a recursive lattice predictor, as first proposed by Mayyas [10]. A single cell of this lattice predictor is shown in figure 2. The following equations define the calculation of the backward, respectively forward prediction errors

$$\mathbf{b}_m(n) = \mathbf{b}_{m-1}(n-1) + \boldsymbol{\kappa}_m(n) \mathbf{f}_{m-1}(n) \quad (1)$$

$$\mathbf{f}_m(n) = \mathbf{f}_{m-1}(n) + \boldsymbol{\kappa}_m^T(n) \mathbf{b}_{m-1}(n-1) \quad (2)$$

which are initialised as

$$\mathbf{b}_0(n) = \mathbf{f}_0(n) = [x_1(n) \ x_2(n)]^T \quad (3)$$

The reflection coefficients κ_{ij} are also calculated recursively as

$$\kappa_{ij,m}(n+1) = \kappa_{ij,m}(n) - \frac{2\mu_r}{\tilde{P}_{ij,m-1}(n) + \epsilon} \cdot (f_{j,m}(n)b_{i,m-1}(n-1) + f_{j,m-1}(n)b_{i,m}(n)) \quad (4)$$

$$\text{for } (i, j) = \{(1, 1), (1, 2), (2, 1), (2, 2)\}$$

where μ_r is a step size constant, ϵ is a constant to compensate for numerical instability and the $\tilde{P}_{ij,m}$ are an estimation of the power in the signal, used as a normalisation. The last factors are calculated as

$$\begin{aligned} \tilde{P}_{ij,m}(n) &= \beta \tilde{P}_{ij,m}(n-1) + \\ &0.5(1-\beta) (b_{i,m-1}^2(n-1) + f_{j,m-1}^2(n)) \end{aligned} \quad (5)$$

for $(i, j) = \{(1, 1), (1, 2), (2, 1), (2, 2)\}$

where $0 \leq \beta \leq 1$.

The AR parameters calculated here are used in an LMS/Newton algorithm, which is an LMS algorithm in which the step size μ is adjusted with respect to the inverse of (an instantaneous estimation of) the autocorrelation matrix of the input signals [11], [12]. Instead of using a direct estimate of the

autocorrelation matrix, a factorisation of this matrix based on the prediction error signals is used, which results in a drastic reduction of required operations, as the order of the linear prediction can be taken much lower than the modeling filter length L_M . For further details concerning this algorithm, the reader is referred to the original paper [2].

The other algorithm is based on direct application of an NLMS algorithm on the residual from (stereo) linear prediction, which was first proposed for the monophonic case in [3]. For the monophonic case, the use of linear prediction is useful as a means to lower the eigenvalue spread, as it is known from [5] that a lower eigenvalue spread yields a better performance for a stochastic gradient type adaptive algorithm.

A further treatment of each of these algorithms can be found in section IV, where they are discussed in context with a stereo coding scheme.

III. STEREO CODING: STEREO LINEAR PREDICTION

The stereo coding scheme used here is a two channel extension of linear prediction for the monophonic case, i.e. stereo linear prediction (SLP) as proposed by [1]. As a summary, the basic equations for this algorithm are given here. Assume a stereophonic signal $\mathbf{x}(n)$, consisting of two channels $x_1(n)$, $x_2(n)$, the predicted signals $\tilde{x}_1(n)$, $\tilde{x}_2(n)$ can be represented as

$$\begin{aligned}\tilde{x}_1(n) &= \sum_{i=1}^{K_a} a_i x_1(n-i) + \sum_{i=1}^{K_b} b_i x_2(n-i) \\ \tilde{x}_2(n) &= \sum_{i=0}^{K_c} c_i x_1(n-i) + \sum_{i=1}^{K_d} d_i x_2(n-i)\end{aligned}\quad (6)$$

Here K_a , K_c and K_b , K_d are the prediction orders for the autocorrelation, respectively cross-correlation terms of both channels. In [13], it is proved that in order to maintain a stable analysis/synthesis scheme, all of these orders need to be the same, say K . Derivation of the optimal coefficients a_i , b_i , c_i , d_i is also provided in [1]. For a symmetric scheme, i.e. the orders are all kept the same and the number of samples on which each estimation is based is the same, the system of equations defined in (7) can be solved efficiently using the block Levinson algorithm, similar to the monophonic case where the LPC parameters can be determined more efficiently using the Levinson-Durbin algorithm [14], [15]. Similar to the monophonic case, the stereo Yule-Walker equations are defined, using the correlation signals $r_{pq}(k) = \sum_n x_p(n)x_q(n-k)$

$$\begin{aligned}r_{11}(l) &= \sum_{i=1}^K a_i r_{11}(l-i) + \sum_{i=1}^K b_i r_{12}(l-i), \quad 1 \leq l \leq K \\ r_{21}(m) &= \sum_{i=1}^K a_i r_{21}(m-i) + \sum_{i=1}^K b_i r_{22}(m-i), \quad 1 \leq m \leq K \\ r_{12}(l) &= \sum_{i=1}^K c_i r_{11}(l-i) + \sum_{i=1}^K d_i r_{12}(l-i), \quad 1 \leq l \leq K \\ r_{22}(m) &= \sum_{i=1}^K c_i r_{21}(m-i) + \sum_{i=1}^K d_i r_{22}(m-i), \quad 1 \leq m \leq K\end{aligned}\quad (7)$$

For further details regarding e.g. the (different implementations of) the block-Levinson algorithm, the reader is again referred to the original thesis [1].

IV. INTEGRATED STEREO ACOUSTIC ECHO CANCELLATION STEREO CODING SCHEME

A. Linear Prediction Based Joint Process Estimation – LMS/Newton

In section II a short description of the algorithm as proposed by [2] was given. A scheme for the integration with an SLP encoding/decoding stage is given in figure 6. A major difficulty in integrating any coding scheme with an adaptive filtering scheme, is that coding schemes mostly work on a block of samples, while adaptive filtering schemes work on a per sample basis. An integration of the SAEC scheme would imply that the κ_{ij} from equations (1) and (2) would be replaced by those sent over the channel. To overcome the problem related to the difference in update window, we suggest keeping the transmitted κ_{ij} constant for the window for which they are defined, and calculating the related prediction error signals recursively on a per sample basis using the lattice predictor equations. The forward prediction error residual signal of the highest order K is transmitted in order to obtain a lossless transmission of the speech signal, as the original speech signal can be regenerated based on the SLP parameters and this residual signal [16]. A tradeoff in the window size must be made with respect to coding and to SAEC. Coding requires the window size to be approximately the time for which a speech signal is assumed to be stationary, i.e. 20ms [16]. For SAEC, a per sample update would be ideal, in face of the integration with the recursive lattice predictor. Note however that in general, the required update rate for an AEC algorithm depends on the rate of change of the room echo paths.

Despite this tradeoff, simulations showed that this wasn't the major issue for an integrated scheme. Tests on an implementation of the recursive lattice predictor showed that it is possible to extract the AR parameters for a signal that is synthesised using known AR parameters. For a real speech signal, where the variation in the AR parameters is considerably more rapid, our implementation was still able to extract the AR parameters, which were estimated using a window based linear prediction scheme. As the variation in AR parameters increased, simulations showed that it was harder to keep track of the window based estimation. Nevertheless, for a proper functioning of the overall SAEC algorithm, the step size for the recursive lattice predictor had to be taken very low, in the order of 10^{-6} . This means that, for our implementation, the SAEC algorithm did not rely on the actual AR parameters in the signal for the decorrelation stage. Even though fairly good results could be obtained for specific input signals, since there is no longer a direct connection with the AR parameters in the signal, we concluded it is not possible to have a functional integration with a coding scheme.

B. Linear Prediction Residual Based Decorrelation – NLMS

Another setup would consist of using the prediction error signals directly as input to an adaptive filtering algorithm, i.e.

NLMS. As explained in section II-C, one of the properties of a linear prediction analysis filter is its whitening property. This property is useful in a monophonic setting to decrease the eigenvalue spread and thus to improve the tracking behaviour of the adaptive filter. For the stereophonic extension, it is interesting since it removes the correlation between the different channels. Mathematically, the update equation for the adaptive filter coefficients $\hat{\mathbf{h}}(n)$ is now written as

$$\hat{\mathbf{h}}(n+1) = \hat{\mathbf{h}}(n) + \mu \frac{\mathbf{f}(n)}{\mathbf{f}^T(n)\mathbf{f}(n) + \epsilon} e(n) \quad (8)$$

where the input to the adaptive filter is replaced by $\mathbf{f}(n)$, the forward prediction error residual signal. The reason why the forward prediction error signals are used, is due to the fact that calculation of all required parameters can start as soon as an initial window of K values are available, whereas the backward prediction error signal is based on future samples and thus imposes an inherent delay.

As the input to the adaptive filter is now based on linear prediction, some adjustments in the calculation of the error signal e_{lpc} are also required. One way of looking at this, is that the inverse filtering operation should be done on the output of the near end room. More precisely, the desired signal d_{lpc} should be calculated as the prediction error residual of the output signals of the near end room, i.e.

$$d_{\text{lpc}} = d(n) - \tilde{d}(n) \quad (9)$$

where $\tilde{d}(n) = \tilde{d}_1(n) + \tilde{d}_2(n)$ is the desired signal after the SLP operation

$$\begin{aligned} \tilde{d}_1(n) &= \sum_{i=1}^K a_i d_1(n-i) + \sum_{i=1}^K b_i d_2(n-i) \\ \tilde{d}_2(n) &= \sum_{i=1}^K c_i d_1(n-i) + \sum_{i=1}^K d_i d_2(n-i) \end{aligned} \quad (10)$$

similar to the SLP equations (6). Note that in this context, the index i in d_i now stands for the index in the original far end signal, i.e. *not* the microphone. The parameters a_i , b_i , c_i and d_i are those transmitted over the channel, calculated earlier in the parameterisation of the original signal. The error signal e_{lpc} in this case now needs to be calculated as

$$e_{\text{lpc}}(n) = d_{\text{lpc}}(n) - y_{\text{lpc}}(n) \quad (11)$$

where

$$y_{\text{lpc}} = \mathbf{f}_1(n) * \hat{\mathbf{h}}_{\text{lpc},1}(n) + \mathbf{f}_2(n) * \hat{\mathbf{h}}_{\text{lpc},2}(n) \quad (12)$$

A problem with this scheme is that the desired signals $d_1(n)$, $d_2(n)$ are generally not known, since they are calculated as the sum of the convolution of $\mathbf{x}_i$ with the $\mathbf{h}_i$. Our proposed solution consists of duplicating the NLMS filter, where one loop is based on the forward prediction error signals and another is based on the actual far end signals. The loop based on the standard far end signals $\mathbf{x}_i(n)$ is used to get an initial estimate of the near end room echo paths. Once an estimation has been obtained, it can be used in the loop based on the prediction error signals. To get an even better estimation, the difference between the “real” desired signal

$d(n)$ and the sum of those calculated through the estimated near end room echo paths $\hat{d}_1(n)$, $\hat{d}_2(n)$ is distributed between $\hat{d}_1(n)$, $\hat{d}_2(n)$. To this extent, the difference signal $d_{\text{diff}}(n)$ is defined as

$$d_{\text{diff}}(n) = [\hat{d}_1(n) + \hat{d}_2(n)] - d(n) \quad (13)$$

such that $\hat{d}_1(n)$, $\hat{d}_2(n)$ are adapted as

$$\begin{aligned} \hat{d}_1(n) &= \hat{d}_1(n) - \frac{d_{\text{diff}}(n)}{2} \\ \hat{d}_2(n) &= \hat{d}_2(n) - \frac{d_{\text{diff}}(n)}{2} \end{aligned} \quad (14)$$

The proposed scheme is shown in figure 7.

V. SIMULATIONS

In this section we'll present some simulation results for the LMS/Newton algorithm of II and the integrated scheme presented in part IV-B. The discussion is done in terms of the misalignment as is commonly done in the literature [6], [7], [2] and the mean squared error (MSE) $e(n)$. As a benchmark, the results for the standard NLMS algorithm are also presented.

a) *Setup parameters:* As input, two Dutch speaker signals are used, a male and a female speaker. The signals are sampled at 44.1kHz and resampled to 16kHz, the resolution is 16bit/sample. The male speaker utters the Dutch words “buiten is het donker en koud”, the female speaker “we kunnen weer even vooruit”. The echo paths for the far end room were chosen to be two independent Gaussian sequences, with zero mean and a variance that decays as a function of the sample index $1/n$ and a length $L_F = 1024$. The echo paths for the near end room were chosen to be two real echo paths, measured in a midsize office at a sampling rate of 16kHz given a length of $L_N = 2001$. The length of the adaptive filters was chosen to be $L_M = 400$. In the near end room, noise is added to the signals such that the resulting signal has an SNR of 40dB. As parameters for the integrated scheme, the symmetric prediction order K was chosen to be 4. The window length on which the SLP scheme works was chosen to be 20ms.

b) *Misalignment:* The minimal misalignment that is reached for the integrated schemes is lower than that for the standard NLMS scheme, because of the decorrelation properties of using the prediction error signals. The prediction order is explicitly kept very low, since simulations showed that higher prediction orders yielded worse results. Figure 3 presents the misalignment for both the application of the standard NLMS algorithm and the integrated scheme. The full blue line represents the results of the former, the red dashed line stands for the latter. On one hand the first figure, which contains the results for the male voice as input signal, shows a clear difference in the convergence speed of the two methods. The proposed algorithm has a better performance of about 0.4 dB after 30000 samples (less than 2 seconds). Although the second plot on the other hand, which has the female voice as input, doesn't show a clear difference, other tested inputs usually gave results comparable to the first graph.

In comparison to the echo cancelling algorithm in [2] our proposed integration scheme delivers similar results for

speech input signals. For the tested audio signals however, the integrated scheme obtains a slightly faster convergence. An example simulation is shown in figure 4, where the proposed algorithm eventually acquires a better misalignment.

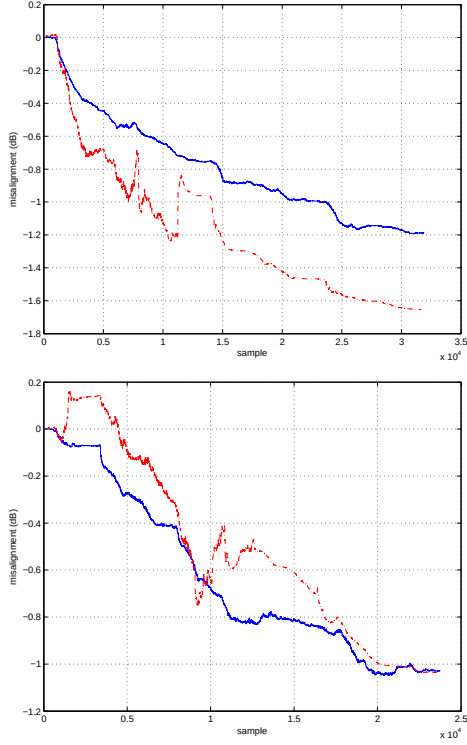


Fig. 3. Misalignment for male 3(a) and female 3(b) speech signals, using the standard NLMS algorithm (full blue line) and the proposed integrated scheme (red dashed line)

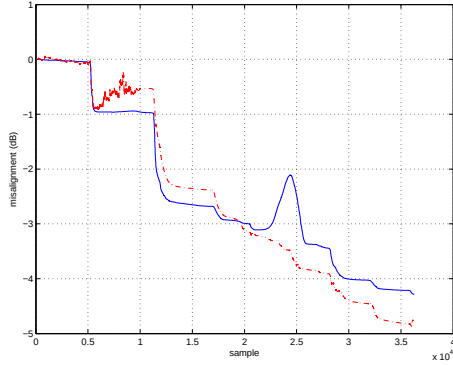


Fig. 4. Misalignment for audio signal [17], using the LMS/Newton algorithm (full blue line) and the proposed integrated scheme (red dashed line)

c) MSE: As a reference, the MSE signals for the proposed schemes are provided in figure 5.

VI. CONCLUSION

In the introduction, a motivation for an integrated scheme combining stereo acoustic echo cancellation on the one hand and stereo coding on the other, was presented. In the two following sections, a short review of both signal processing

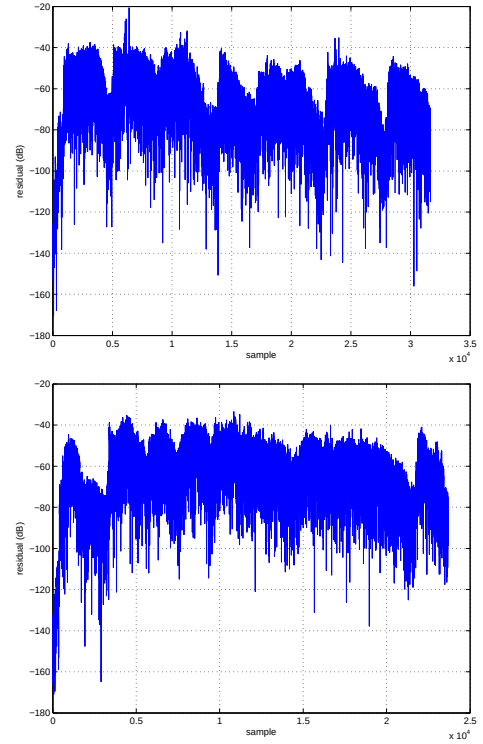


Fig. 5. MSE for male 5(a) and female 5(b) speech signals, with added noise, using the proposed integrated scheme

techniques was provided, with an emphasis on linear prediction based schemes. For the SAEC problem, the fundamental problem was explained and a number of solutions were mentioned. For stereo coding, the basic principles of stereo linear prediction from [1] were reviewed. In the next section, two integrated schemes were proposed, one based on an LMS/Newton algorithm from [2] another based on direct application of an NLMS algorithm on the forward prediction error signals. Since simulations revealed that the AR parameters used in the LMS/Newton algorithm did not correspond to the actual speech parameters, we found that the last scheme working directly at SLP parameters, is the algorithm of choice for an integrated scheme. In the final section, a number of simple simulations were shown, indicating the advantage of using the proposed scheme over direct application of an NLMS algorithm without any decorrelation.

APPENDIX A

FIGURES INTEGRATED SCHEMES

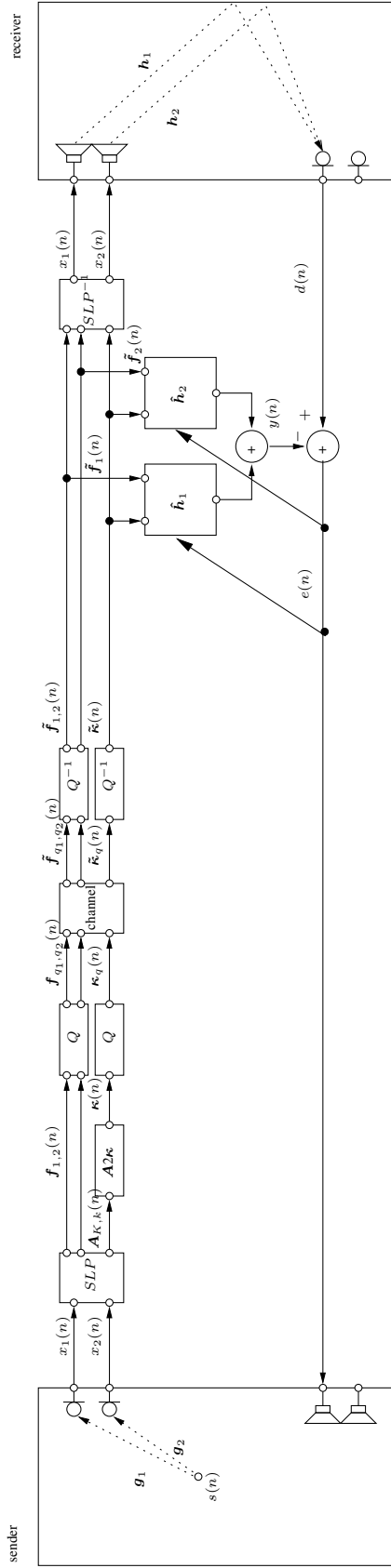


Fig. 6. Integrated scheme using joint process estimation – LMS/Newton

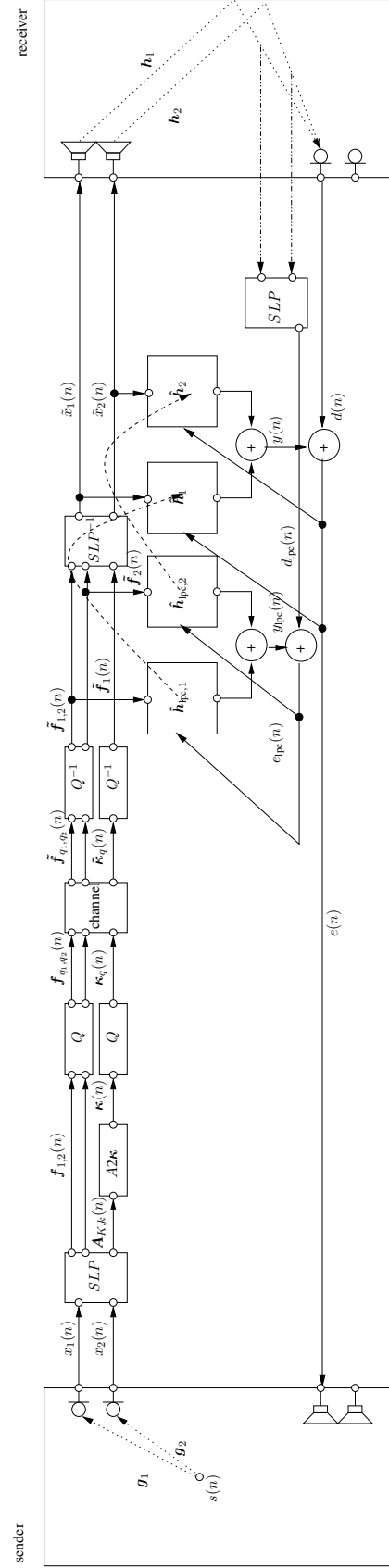


Fig. 7. Integrated scheme using linear prediction residual based decorrelation

REFERENCES

- [1] A. Biswas, "Advances in perceptual stereo audio coding using linear prediction techniques," <http://library.tue.nl/catalog/LinkToVubis.csp?DataBib=6:625934>, 2007.
- [2] H. I. K. Rao and B. Farhang-Boroujeny, "Fast LMS/Newton algorithms for stereophonic acoustic echo cancelation," *IEEE Transactions on Signal Processing*, vol. 57, no. 8, pp. 2919–2930, 2009.
- [3] C. Breining, P. Dreiscitel, E. Hansler, A. Mader, B. Nitsch, H. Puder, T. Schertler, G. Schmidt, and J. Tilp, "Acoustic echo control. an application of very-high-order adaptivefilters," *IEEE Signal Processing Magazine*, vol. 16, no. 4, pp. 42–69, 1999.
- [4] G. E. Moore, "Cramming more components onto integrated circuits," *Proceedings of the IEEE*, vol. 38, no. 1, pp. 82–85, Apr. 1965.
- [5] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, Sep. 2001.
- [6] M. Sondhi, D. Morgan, and J. Hall, "Stereophonic acoustic echo cancellation-an overview of the fundamental problem," *Signal Processing Letters, IEEE*, vol. 2, no. 8, pp. 148–151, 1995.
- [7] J. Benesty, D. Morgan, and M. Sondhi, "A better understanding and an improved solution to the specific problems of stereophonic acoustic echo cancellation," *Speech and Audio Processing, IEEE Transactions on*, vol. 6, no. 2, pp. 156–165, 1998.
- [8] E. A. Habets, "Single-and multi-microphone speech dereverberation using spectral enhancement," <http://library.tue.nl/catalog/LinkToVubis.csp?DataBib=6:627677>, 2007.
- [9] D. Morgan, J. Hall, and J. Benesty, "Investigation of several types of nonlinearities for use in stereo acoustic echo cancellation," *Speech and Audio Processing, IEEE Transactions on*, vol. 9, no. 6, pp. 686–696, 2001.
- [10] K. Mayyas, "Stereophonic acoustic echo cancellation using lattice orthogonalization," *Speech and Audio Processing, IEEE Transactions on*, vol. 10, no. 7, pp. 517–525, 2002.
- [11] R. W. Chang, "A new equalizer structure for fast start-up digital communication(Automatic equalizer with fast startup time for digital communication systems including partial response and single sideband nyquist systems)," *Bell System Technical Journal*, vol. 50, pp. 1969–2014, 1971.
- [12] C. F. N. Cowan, "Performance comparisons of finite linear adaptive filters," in *IEE Proceedings F Communications, Radar and Signal Processing*, vol. 134, 1987, pp. 211–216.
- [13] A. Biswas, T. Selten, A. C. den Brinker, and P. N. Laboratorium, "Stability of the synthesis filter in stereo linear prediction," in *Proc. 15th ProRISC*, 2004, p. 230237.
- [14] N. Levinson, "The wiener RMS (root mean square) error criterion in filter design and prediction," *J. Math. Phys.*, vol. 25, no. 4, pp. 261–278, 1947.
- [15] J. Durbin, "The fitting of time-series models," *Revue de l'Institut International de Statistique*, p. 233244, 1960.
- [16] W. C. Chu, *Speech coding algorithms: foundation and evolution of standardized coders*. Wiley-Interscience, 2003.
- [17] K. Andersson and M. Deykers, "Seven (Martyn's seventh mix)," Sep. 2009. [Online]. Available: <http://www.discogs.com/Fever-Ray-Seven/release/2061300>